SNOWFLAKE DATA ENGINEERING PROJECT

Healthcare Intelligence Hub

Patient, Cost & Care Analytics

Complete Project Report

Final report · Phases 1–8 · Appendices

Student	Aashi Bindal
Enrollment No.	0827CI241004
Team ID	[Team ID]
Institution	Acropolis Institute of Technology and Research · Department of CSIT
Faculty Guide	[Faculty Guide]
Academic Session	[Academic Session]
Document date	October 2026

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

Abstract

This project builds a Snowflake data-engineering pipeline and a Streamlit-in-Snowflake dashboard. They address a clear problem: hospital administrators and finance teams need a consolidated, queryable view of patient visits (cost, length of stay (LOS), satisfaction and readmissions) that today sits fragmented across raw operational records. The project answers which facilities and doctors perform well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk. Six CSV sources (14,059 rows) are loaded into `HEALTH_DB`, cleaned in `STAGING`, modelled as a visit-grain star schema in `CURATED`, and summarised by ten `MART` views. Fifty-five automated checks were executed against a local replica of the pipeline: 50 pass, and 5 expose genuine data or logic issues (two `INITCAP` label changes, a doctor-grouping grain error and two source-data anomalies). Snowflake-side and live-UI tests are scripted but not yet executed.

Keywords: Snowflake, data warehouse, ELT, star schema, Streamlit, healthcare analytics, readmission, data quality.

About this report

Part I follows the institution's Final Report Template (sections 1–11). Part II contains the eight phase documents (Phase 7 is Part I). Part III holds the full source code and data dictionary. Every technical statement was checked against the uploaded project files. Tests marked "Not Executed" have not been run, and figures labelled as local reproductions are not screenshots of the deployed app.

Contents

Part I — Final Report 1

1. INTRODUCTION 1
2. IDEATION PHASE 1
3. REQUIREMENT ANALYSIS 2
4. PROJECT DESIGN 3
5. PROJECT PLANNING & SCHEDULING 4
6. FUNCTIONAL AND PERFORMANCE TESTING 5
7. RESULTS 5
8. ADVANTAGES & DISADVANTAGES 7
9. CONCLUSION 7
10. FUTURE SCOPE 8
11. APPENDIX 8

Part II — Detailed Phase Documents 9

Phase 1 — Ideation Phase 10

1.1 Ideation Summary 10
1.2 Define the Problem Statements 12
1.3 Empathy Map Canvas 13
1.4 Brainstorm & Idea Prioritization 13

Phase 2 — Requirement Analysis 15

2.1 Problem Statement and Business Objectives 15
2.2 Customer Journey Map 15
2.3 Solution Requirements (Functional & Non-functional) 16
2.4 Data Flow Diagram & User Stories 19
2.5 Technology Stack (Architecture & Stack) 21

Phase 3 — Project Design Phase 23

3.1 Problem – Solution Fit 23
3.2 Proposed Solution (Project Proposal) 23
3.3 Solution Architecture 24
3.4 Data Layers and Pipeline Design 26
3.5 Snowflake Database and Schema Design 27
3.6 Fact and Dimension Model 28
3.7 Data Transformation Design 29
3.8 Application and Dashboard Design 29

3.9 Security, Access Control and Data Privacy 30

Phase 4 — Project Planning Phase 32

4.1 Project Planning Template 32

4.2 Work Breakdown Structure 34

4.3 Phases, Milestones and Deliverables 34

4.4 Resource and Tool Requirements 35

4.5 Roles and Responsibilities 35

4.6 Risk Assessment and Mitigation 35

4.7 Testing and Documentation Plan 35

4.8 Progress Tracking and Completion Criteria 36

Phase 5 — Project Development Phase 37

5.1 Development Environment and Setup 37

5.2 Verified Technology Stack and Dependencies 38

5.3 Source Data Collection and Ingestion 38

5.4 Raw-Layer Implementation 38

5.5 Data Cleaning and Standardisation (STAGING) 38

5.6 Data Transformation and Modelling (CURATED) 39

5.7 Snowflake SQL Object Inventory 39

5.8 Analytical Queries and KPIs (MART) 40

5.9 Streamlit Application Implementation 40

5.10 Integration Between Components 41

5.11 Error Handling, Logging and Validation 41

5.12 Business Questions and Visualisation Report 41

5.13 Dashboard Design 46

5.14 Story Design 47

5.15 Completed versus Planned Functionality 47

Phase 6 — Performance Testing 49

6.1 Project Performance Testing (template) 49

6.2 Testing Objectives and Scope 49

6.3 Test Strategy and Methodology 50

6.4 Executed Test Cases (local replica) 50

6.5 Snowflake Test Cases (Not Executed) 52

6.6 Dashboard Functional and Filter Tests (Not Executed, manual) 53

6.7 Query Performance and Resource Usage 53

6.8 Defects, Limitations and Corrective Actions 54

Phase 8 — Project Demonstration 55

8.1 Presentation-Ready Overview 55
8.2 Demonstration of Proposed Features 55
8.3 Team Involvement in Demonstration 56
8.4 Project Demo Planning 57
8.5 Communication 57
8.6 Step-by-Step Setup and Execution (for the demo machine) 58
8.7 Demonstration Script (≈ 9 minutes) 58
8.8 Suggested Screenshots to Capture 59
8.9 Frequently Asked Technical Questions 59
8.10 Known Limitations 60
8.11 Scalability & Future Plan 60

Part III — Appendices 62

Appendix A — query.sql (complete) 63

Appendix B — Streamlit application and deployment files 69

B.1 Health/streamlit_app.py 69
B.2 Health/snowflake.yml 72
B.3 Health/pyproject.toml 72

Appendix C — Data Dictionary 73

List of Figures and Tables

List of Figures

Figure 7.1 — DFD Level 1 (see also Level 0 in Phase 2) 3
Figure 7.2 — Solution architecture 4
Figure 7.3 — Monthly visit trend 6
Figure 7.4 — Average cost breakdown by visit type 6
Figure 7.5 — 30-day readmission rate by diagnosis 7
Figure 2.1 — Context diagram (DFD Level 0) 20
Figure 2.2 — DFD Level 1: processes 1.0–5.0 and data stores D1–D4, as implemented in query.sql and streamlit_app.py 20
Figure 3.1 — Solution architecture as implemented. The dashed load arrow marks the one step that is not scripted in the repository. 25
Figure 3.2 — Layered data pipeline with the validation queries run at each layer 26
Figure 3.3 — ER diagram of the CURATED star schema (keys are logical: no PK/FK constraints are declared in query.sql) 28
Figure 3.4 — Proposed role hierarchy and masking (dashed = proposed, not implemented) 31

Figure 4.1 — Planned (ideal) burndown for 60 story points over 24 working days. No actual daily progress data was recorded, so only the ideal line is shown. Plot your actual remaining points against it. 34
Figure 5.1 — Monthly visit trend (local reproduction from CSVs) 42
Figure 5.2 — Visits by visit type (bar form used for print; the app uses a donut) 42
Figure 5.3 — Top 10 diagnoses by visit volume 43
Figure 5.4 — 30-day readmission rate by diagnosis (all 15 shown; the app shows the top 10) 43
Figure 5.5 — Visits and average cost per visit by facility 44
Figure 5.6 — Average cost breakdown by visit type 44
Figure 5.7 — Treatment and admission status 45
Figure 5.8 — Insurance coverage share by provider (additional analysis, not in app) 45
Figure 5.9 — Distribution of patient satisfaction scores (additional analysis, not in app) 46

List of Tables

Table 7.1 — Test summary 5
Table 7.2 — Headline KPI results (full dataset, local replica of VW_EXECUTIVE_KPI) 5
Table 1.1 — Challenges identified during ideation 10
Table 1.2 — Stakeholders 11
Table 1.3 — Customer problem statements 12
Table 1.4 — Empathy map: Hospital administrator / finance manager 13
Table 1.5 — Brainstormed ideas 13
Table 1.6 — Idea prioritisation 14
Table 2.0 — Problem-statement questions and how they are answered 15
Table 2.1 — Customer journey map 16
Table 2.2 — Functional requirements 16
Table 2.3 — Non-functional requirements 17
Table 2.4 — User stories 20
Table 2.5 — Technology stack 21
Table 3.1 — Problem–Solution Fit canvas (CC BY-NC-ND 4.0 canvas by Daria Nepriakhina / Amaltama) 23
Table 3.2 — Proposed solution 24
Table 3.3 — Architecture components 25
Table 3.4 — Layer responsibilities 26
Table 3.5 — Keys, grain and relationships 28
Table 3.6 — Transformation rules by layer 29
Table 3.7 — Dashboard layout (wireframe of the implemented page) 30
Table 4.1 — Product backlog and sprint schedule 32
Table 4.2 — Project tracker (relative schedule; actual dates to be filled in) 33
Table 4.3 — Milestones (relative timeline, estimated) 34
Table 4.4 — Risk register 35
Table 5.1 — Dependencies found in the code 38

Table 5.2 — Objects created by query.sql (36 objects incl. the Streamlit app) 39
Table 5.3 — MART views and their measures 40
Table 5.4 — Implementation status (verified against code) 47
Table 6.1 — Performance test template 49
Table 6.2 — Test levels and environments 50
Table 6.3 — Executed checks (local replica, 21 Oct 2026) 50
Table 6.4 — Snowflake platform tests 52
Table 6.5 — Dashboard UI test cases 53
Table 6.6 — Defect log 54
Table 8.1 — Proposed features and status (implementation status verified against code) 55
Table 8.2 — Demo plan (10 minutes) 57

Part I — Final Report

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Template covered:** Project Report Format (Final Report Template). Sections 1–11 below follow the template order exactly. Each section summarises its phase and cross-references the detailed phase document.

Field	Value
Student	Aashi Bindal
Enrollment Number	0827CI241004
Team ID	[Team ID]
Institution	Acropolis Institute of Technology and Research, Department of Computer Science & Information Technology (CSIT)
Faculty Guide	[Faculty Guide]
Academic Session	[Academic Session]

Abstract. This project builds a Snowflake data-engineering pipeline and a Streamlit-in-Snowflake dashboard. They address a clear problem: hospital administrators and finance teams need a consolidated, queryable view of patient visits (cost, length of stay (LOS), satisfaction and readmissions) that today sits fragmented across raw operational records. The project answers which facilities and doctors perform well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk. Six CSV sources (14,059 rows) are loaded into `HEALTH_DB`, cleaned in `STAGING`, modelled as a visit-grain star schema in `CURATED`, and summarised by ten `MART` views. Fifty-five automated checks were executed against a local replica of the pipeline: 50 pass, and 5 expose genuine data or logic issues (two `INITCAP` label changes, a doctor-grouping grain error and two source-data anomalies). Snowflake-side and live-UI tests are scripted but not yet executed.

Keywords: Snowflake, data warehouse, ELT, star schema, Streamlit, healthcare analytics, readmission, data quality.

1. INTRODUCTION

1.1 Project Overview

The Healthcare Intelligence Hub is an end-to-end analytics solution built on Snowflake. Its components, all verified in the repository:

- `query.sql`: creates database `HEALTH_DB` with schemas `RAW`, `STAGING`, `CURATED` and `MART`; a CSV file format and an internal stage; 6 raw tables, 6 staging tables, 5 dimensions, 1 fact table and 10 views; and validation queries.
- `Health/streamlit_app.py`: a single-page dashboard with 6 KPI cards, 8 charts, 2 tables and 5 filters, reading `CURATED.FACT_PATIENT_VISITS` through the active Snowpark session.
- `Health/snowflake.yml` and `pyproject.toml`: deployment definition for Streamlit in Snowflake (warehouse `COMPUTE_WH`, Python 3.11).
- Six CSV data files covering January 2024 to August 2026.

1.2 Purpose

To replace fragmented raw operational records with a structured warehouse: a governed, re-runnable pipeline and a self-service dashboard. The aim is that visit volume, cost, LOS, satisfaction and readmission metrics are defined once and can be compared across facilities, doctors and diagnoses.

2. IDEATION PHASE

(Full detail: Phase 1)

2.1 Problem Statement

> Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.

The customer view of this problem is captured in PS-1 to PS-3 (Phase 1 Table 1.3). The three questions are answered as follows (detail in Phase 2 Table 2.0):

#	Question in the problem statement	How the solution answers it	Finding (full dataset, local replica)
Q1	Which facilities and doctors are performing well?	`VW_FACILITY_ANALYTICS` , `VW_DOCTOR_ANALYTICS` ; app facility chart and table, doctor leaderboard (visits, avg cost, satisfaction, readmissions)	Facility volume 958–1,039 visits; City General Hospital busiest (1,039). Highest doctor satisfaction: Dr. Neha Brown, D002 (4.02)
Q2	Which diagnoses drive the longest and most expensive stays?	`VW_DIAGNOSIS_ANALYTICS` (avg cost, avg LOS); app filter Visit Type = Inpatient with diagnosis charts	Inpatient stays: longest Fracture (8.60 days avg), Anemia (8.32), Diabetes (8.07); most expensive Kidney Disease (14,816.82 avg), Hypertension (14,640.75), Asthma (14,549.67)
Q3	Where is patient satisfaction at risk?	Avg Satisfaction KPI; satisfaction columns in facility and doctor tables and in `VW_DIAGNOSIS_ANALYTICS`	Lowest facility: Sunrise Hospital (3.78; 13.9% of visits score 1–2). Lowest diagnosis: Asthma (3.75). Lowest doctors: Dr. Rohan Singh, D060 (3.65), Dr. Rohan Gupta, D004 (3.69)

2.2 Empathy Map Canvas

The combined administrator / finance persona *says* "I need one agreed number", *thinks* "are my joins right?", *does* monthly VLOOKUP consolidation, and *feels* overwhelmed and anxious about accuracy. The persona gains trust and speed from a single governed dataset (Phase 1 Table 1.4).

2.3 Brainstorming

Ten ideas were grouped into four themes. The three selected and implemented: (1) a layered Snowflake ingestion and cleaning pipeline; (2) a star schema with MART KPI views; (3) a Streamlit-in-Snowflake dashboard, chosen over Tableau to keep data and UI on one platform. Automation, RBAC and ML were deferred (Phase 1 Table 1.6).

3. REQUIREMENT ANALYSIS

(Full detail: Phase 2)

3.1 Customer Journey Map

Five stages: need → gather → prepare → present → follow-up. Each is mapped from today's manual pain points to the Hub's corresponding layer (Phase 2 Table 2.1).

3.2 Solution Requirement

Six functional epics: Data Access & Integration, Data Processing, Business Analysis, Business Insights, User Interaction, Data Update & Reporting. Six NFRs: usability, security, reliability, performance, availability, scalability (Phase 2 Tables 2.2–2.3). Data-quality requirements DQ-1 to DQ-6 were defined and then tested in Phase 6.

3.3 Data Flow Diagram

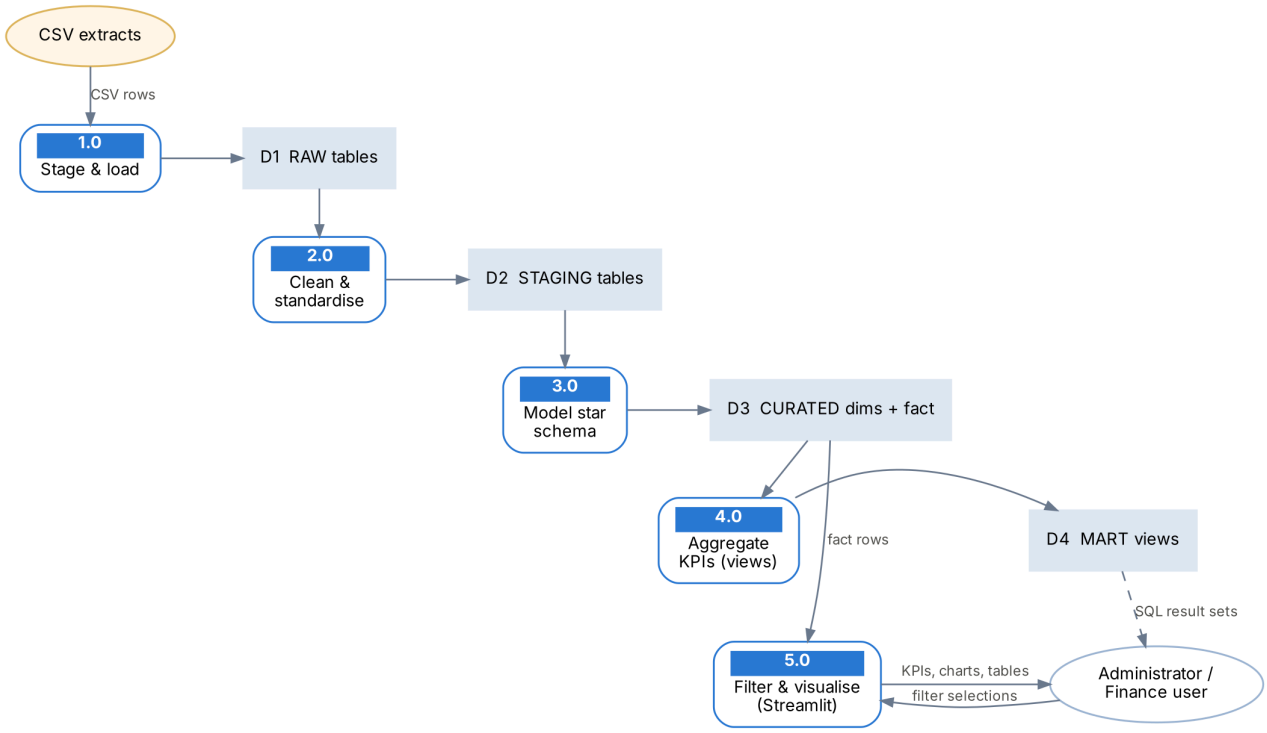


Figure 7.1 — DFD Level 1 (see also Level 0 in Phase 2)

3.4 Technology Stack

Snowflake SQL and Python 3.11; Snowflake `HEALTH_DB` with `COMPUTE_WH` ; internal stage and file format for ingestion; Snowflake SQL (CTAS, views) and pandas for transformation; Altair and Streamlit for visualisation; Streamlit in Snowflake as the interface (Phase 2 Table 2.5).

4. PROJECT DESIGN

(Full detail: Phase 3)

4.1 Problem Solution Fit

The canvas links the customer segment (administrators and finance), constraints (time, skills, privacy), root cause (no integrated model or KPI definitions) and triggers (month-end reviews, cost or readmission spikes) to the solution and its online and offline channels (Phase 3 Table 3.1).

4.2 Proposed Solution

A Snowflake-native pipeline with a dashboard. Its novelty is a single governed platform with no credentials in the app. Its impact is earlier detection of high-readmission diagnoses and transparent out-of-pocket cost (≈27.7%). It scales by warehouse resizing, with SQL push-down proposed for large data (Phase 3 Table 3.2).

4.3 Solution Architecture

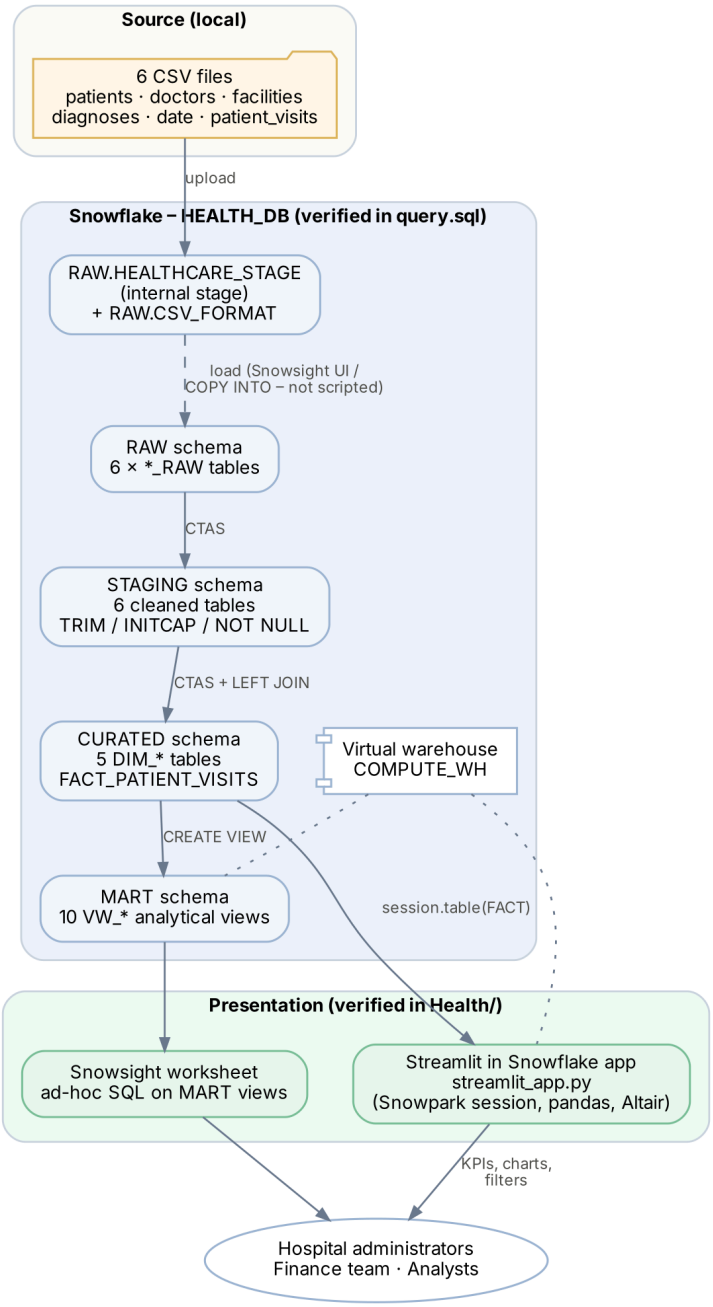


Figure 7.2 — Solution architecture

The star schema has `FACT_PATIENT_VISITS` (grain: one visit; 10,000 rows) and dimensions for patient (3,000), doctor (60), facility (10), diagnosis (15) and date (974). Keys are logical: no constraints are declared, and `DIM_DATE` is not yet joined (Phase 3 §3.6).

5. PROJECT PLANNING & SCHEDULING

(Full detail: Phase 4)

5.1 Project Planning

Four 6-day sprints of 15 story points each (60 in total; planned velocity 2.5 points per day). Sprint 1: setup and ingestion. Sprint 2: staging, curated model, MART views and validation. Sprint 3: KPI and chart stories. Sprint 4: filters, deployment, testing and documentation. Actual dates were not available in the project files and are left as placeholders.

6. FUNCTIONAL AND PERFORMANCE TESTING

(Full detail: Phase 6 · TEST_CASES.md)

6.1 Performance Testing

Table 7.1 — Test summary

Suite	Cases	Executed	Pass	Fail	Not Executed
Local replica (`tests/local_validation.py`)	55	55	50	5	0
Snowflake (`tests/snowflake_validation.sql`)	9	0	—	—	9
Dashboard UI (manual)	12	0	—	—	12

Defects (Phase 6 Table 6.6): D-01 doctor-view grain merges two doctors; D-02/D-03 `INITCAP` produces `Ent` and `Follow-Up`; D-04 "Self Pay" visits show insurance coverage; D-05 admitted emergency visits with 0-day LOS; D-06 redundant view; D-07 unused `DIM_DATE`; D-08 currency label; D-09 load not scripted. Local timings (≈0.3 s end to end) are not Snowflake measurements.

7. RESULTS

7.1 Output Screenshots

> The figures below were regenerated from the project CSVs with the same aggregation logic as the app and MART views. **They are not screenshots of the deployed Snowflake app.** Insert live screenshots at the marked placeholders (capture list: Phase 8 §8.8).

[Screenshot placeholder — Streamlit app, unfiltered: header, sidebar filters and KPI row]

Table 7.2 — Headline KPI results (full dataset, local replica of VW_EXECUTIVE_KPI)

Total visits	Unique patients	Doctors	Facilities	Total cost	Insurance coverage	Patient payment	Avg satisfaction	Avg LOS	Readmissions
10,000	2,889	60	10	82,818,797.00	59,918,913.84	22,899,883.16	3.83	0.97	1,203 (12.03%)

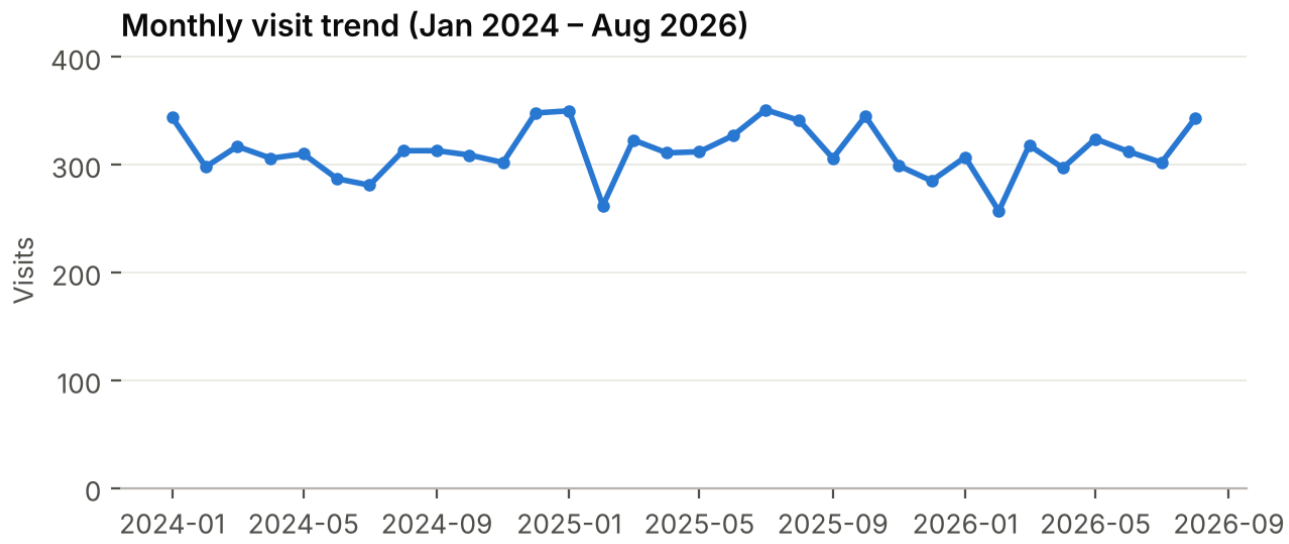


Figure 7.3 — Monthly visit trend

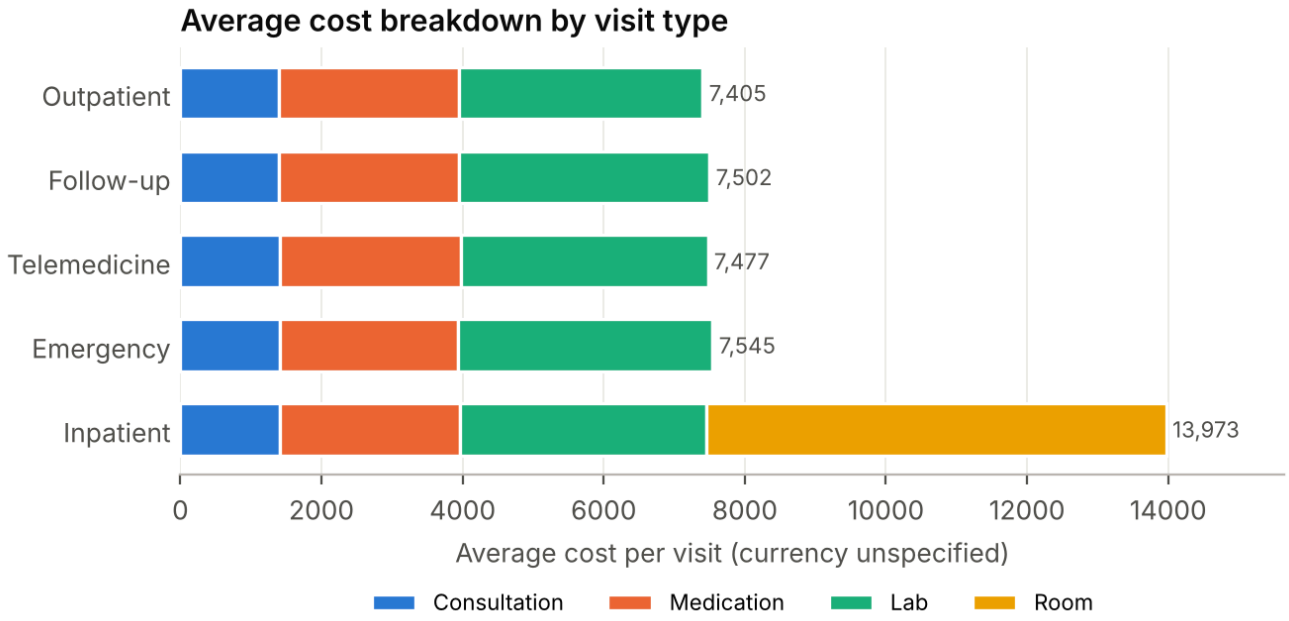


Figure 7.4 — Average cost breakdown by visit type

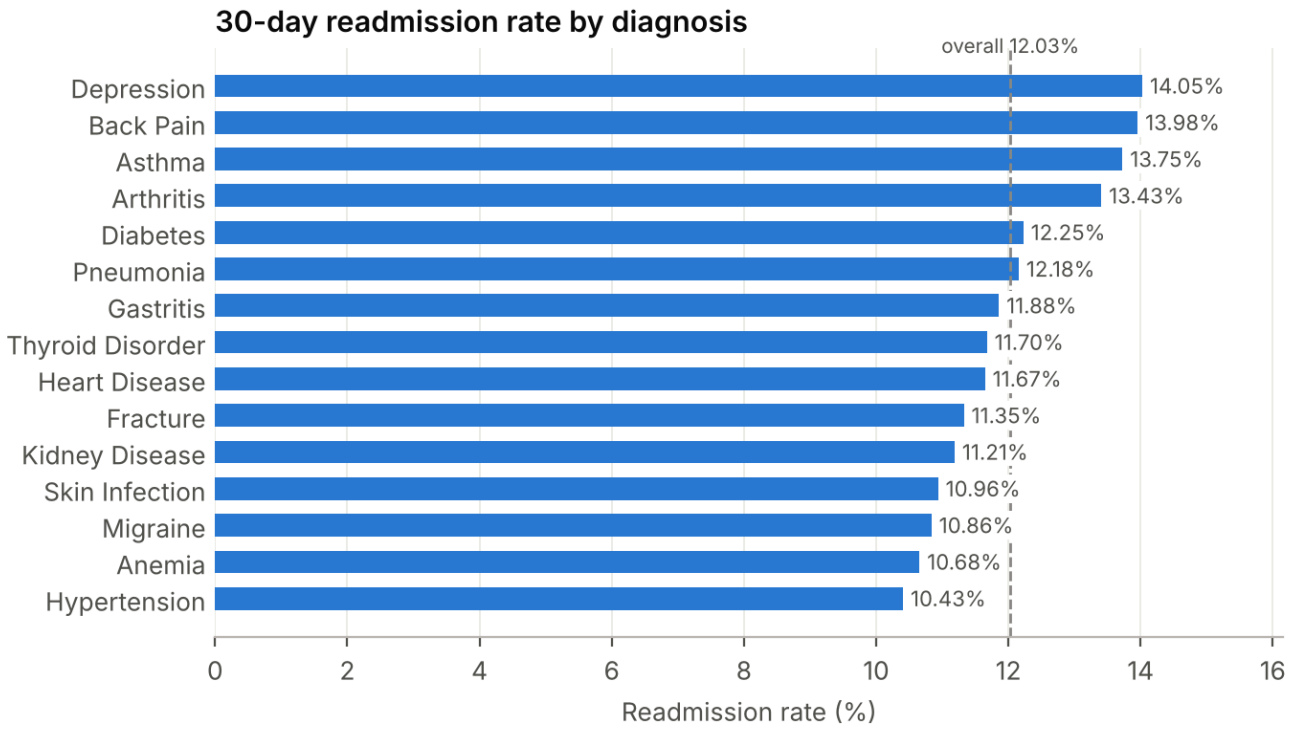


Figure 7.5 — 30-day readmission rate by diagnosis

Discussion. Inpatient visits (12.6% of volume) cost about 1.9× other types because only they incur room charges. Lab is the largest cost component (42.3%). Readmissions cluster in Depression, Back Pain, Asthma and Arthritis (13.4–14.1%, against 12.03% overall). Facility differences in volume, cost and satisfaction are small (cost per visit within about 4%), which is consistent with a synthetic dataset. Two source anomalies, insured "Self Pay" visits and zero-LOS admissions, mean that payer and LOS insights should be confirmed with the data owner before any operational use.

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages / limitations
Whole stack in one platform: no data movement, no credentials in the app	Data load step not scripted; refresh is manual
Layered schemas make every metric traceable and rebuildable (`CREATE OR REPLACE`)	No RBAC or masking; patient names flow into the fact table and app
KPI logic centralised in 10 SQL views	App recomputes KPIs in pandas instead of using MART views, which duplicates logic
Interactive filters with caching; zero-row guards	Full fact table loaded into memory, which limits scale
Data verified clean: 0 NULL or duplicate keys, 0 orphans, costs reconcile	INITCAP label side effects; doctor-view grain bug; unused `DIM_DATE` ; redundant view
Low cost: X-Small warehouse suffices	Snowflake and UI tests not yet executed; synthetic data limits real-world conclusions

9. CONCLUSION

The project meets its core objectives. It consolidates six healthcare sources into a validated star schema in Snowflake, defines visit, cost, LOS, satisfaction and readmission KPIs in reusable views, and presents them through a filterable Streamlit-in-Snowflake dashboard. Verification against a local replica confirmed complete, unique and referentially intact data and correct KPI arithmetic. It also surfaced specific, fixable defects that the original validation queries did not catch. With the corrective actions in Phase 6 and the execution of the prepared Snowflake and UI tests, the solution would be ready for a pilot on real, governed data.

10. FUTURE SCOPE

See Phase 8 §8.11 for the roadmap. In summary: scripted `COPY INTO` with Snowpipe or Tasks for automated refresh; RBAC and masking policies; SQL push-down in the app; weekday and seasonality analysis via `DIM_DATE`; insurer and payer dashboards; SCD Type 2 dimensions; readmission-risk modelling with Snowpark ML; CI checks using the provided test scripts.

11. APPENDIX

Source Code: `query.sql` (complete SQL; key excerpts in Phase 5) · `Health/streamlit_app.py` (dashboard) · `Health/snowflake.yml`, `Health/pyproject.toml` (deployment) · `tests/local_validation.py`, `tests/snowflake_validation.sql` (tests added with this documentation). Data dictionary: `DATA_DICTIONARY.md`.

Dataset Link: the six CSVs are included in the repository root. Original dataset source: **[provide the source or "provided by institution"]**.

GitHub & Project Demo Link: GitHub: **[https://github.com/<username>/<repository>]** · Demo video: **[link]**

References

1. Snowflake Inc. *COPY INTO <table>*. Snowflake Documentation. https://docs.snowflake.com/en/sql-reference/sql/copy-into-table
2. Snowflake Inc. *CREATE STAGE.* https://docs.snowflake.com/en/sql-reference/sql/create-stage
3. Snowflake Inc. *CREATE FILE FORMAT.* https://docs.snowflake.com/en/sql-reference/sql/create-file-format
4. Snowflake Inc. *INITCAP.* https://docs.snowflake.com/en/sql-reference/functions/initcap
5. Snowflake Inc. *About Streamlit in Snowflake.* https://docs.snowflake.com/en/developer-guide/streamlit/about-streamlit
6. Snowflake Inc. *Understanding Dynamic Data Masking.* https://docs.snowflake.com/en/user-guide/security-column-ddm-intro
7. Snowflake Inc. *Overview of Access Control.* https://docs.snowflake.com/en/user-guide/security-access-control-overview
8. Snowflake Inc. *QUERY_HISTORY view.* https://docs.snowflake.com/en/sql-reference/account-usage/query_history
9. Snowflake Inc. *Introduction to tasks.* https://docs.snowflake.com/en/user-guide/tasks-intro
10. Streamlit. *st.cache_data.* https://docs.streamlit.io/develop/api-reference/caching-and-state/st.cache_data
11. Vega-Altair: Declarative Visualization in Python. https://altair-viz.github.io/
12. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
13. Centers for Medicare & Medicaid Services. *Hospital Readmissions Reduction Program (HRRP).* https://www.cms.gov/medicare/payment/prospective-payment-systems/acute-inpatient-pps/hospital-readmissions-reduction-program-hrrp

Part II — Detailed Phase Documents

The chapters below are the complete institutional phase documents that Part I summarises. Section numbers restart within each phase (e.g., “Phase 3 §3.6”).

Phase 1 — Ideation Phase

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Define the Problem Statements (2 marks) · Empathy Map Canvas (4 marks) · Brainstorm & Idea Prioritization (4 marks)

> **Evidence basis.** Everything in this document was checked against the uploaded project archive (`query.sql` , `Health/streamlit_app.py` , `Health/snowflake.yml` , `Health/pyproject.toml` and six CSV files). Items marked *Proposed* are not present in the code. Team details are placeholders because they were not supplied.

1.1 Ideation Summary

1.1.1 Abstract

Hospitals record every patient visit across several operational sources: patient registration, doctor rosters, facility master data, diagnosis coding and billing. Administrators and finance teams rarely see these together. The Healthcare Intelligence Hub loads six CSV extracts into a Snowflake database (`HEALTH_DB`) and moves them through four schemas: `RAW` , `STAGING` , `CURATED` and `MART` . Along the way it cleans and standardises the data, models it as one visit-level fact table (`FACT_PATIENT_VISITS`) with five dimensions, and exposes ten analytical views. A Streamlit-in-Snowflake application reads the fact table and gives users six KPI cards, eight charts, two tables and five filters. The dataset covers 10,000 visits by 2,889 distinct patients across 10 facilities, 60 doctors and 15 diagnoses, from January 2024 to August 2026.

Keywords: Snowflake, data engineering, medallion layers, star schema, healthcare analytics, Streamlit in Snowflake, readmission rate, length of stay, cost analytics.

1.1.2 Problem statement

> **Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.**

(Official problem statement from the project workspace.)

The data needed to answer these questions already exists in operational records: patients, doctors, facilities, diagnoses, a calendar, and visit-level billing and outcomes. It is not organised for analysis. Each question today means joining raw extracts by hand, so answers are slow, inconsistent and hard to compare across facilities or doctors. The project's purpose is to build that structured warehouse and put an interactive view on top of it.

1.1.3 Existing challenges in healthcare analytics

Table 1.1 — Challenges identified during ideation

#	Challenge	Effect on decision-makers
1	Data is held in separate sources: patients, doctors, facilities, diagnoses and visits/billing	No single view of a visit with its cost and outcome
2	Inconsistent text values (case, stray spaces) between sources	Group-by results split one category into several
3	Manual spreadsheet consolidation	Reports go stale; numbers differ between teams
4	No common KPI definitions (e.g., readmission rate)	Facilities and doctors cannot be compared fairly
5	Cost components (consultation, medication, lab, room) and payer split (insurance and patient) are not analysed together	Finance cannot see what drives cost or out-of-pocket burden
6	Static reports with no filtering	Users cannot narrow by date, facility, specialty, visit type or diagnosis category

1.1.4 Proposed solution and innovation

The proposed solution is a cloud data-warehouse pipeline with an interactive application on top:

1. **Ingest:** a named CSV file format and an internal stage in Snowflake, with typed `*_RAW` tables.

2. **Clean:** `STAGING` tables apply `TRIM`, `INITCAP` and NOT NULL key filters.
3. **Model:** a `CURATED` star schema of five `DIM_*` tables and a visit-grain `FACT_PATIENT_VISITS`.
4. **Serve:** ten `MART` views that hold the KPI logic (executive KPIs, monthly trend, diagnosis, facility, doctor, visit type, treatment, admission and readmission analytics).
5. **Explore:** a Streamlit app that runs inside Snowflake, so data never leaves the platform. It uses the active Snowpark session with no stored credentials and offers five filters.

What makes it innovative: the whole stack (storage, transformation, KPI logic and user interface) runs inside a single governed platform. There is no separate BI server and no credential handling in the app (`get_active_session()`). The layered schemas also make every number traceable back to its raw row.

1.1.5 Target users and stakeholders

Table 1.2 — Stakeholders

Stakeholder	Interest	Primary outputs used
Hospital administrator	Visit volumes, length of stay, facility and doctor workload, satisfaction	KPI cards, monthly trend, facility performance, doctor leaderboard
Finance team	Total cost, cost breakdown, insurance coverage and patient payment	KPI view totals, cost-breakdown chart, facility cost table
Clinical quality lead	30-day readmissions, treatment outcomes	Readmission-by-diagnosis chart, treatment status chart
Data engineer	Reliable, auditable pipeline	RAW, STAGING, CURATED and MART schemas; validation queries
Faculty evaluator	Correctness and completeness of the project	This documentation set

1.1.6 Business value and expected benefits

- One trusted, queryable source for visit, cost and outcome metrics.
- KPI definitions written once in SQL views and reused, instead of being recalculated in spreadsheets.
- Faster ad-hoc analysis: filters and charts recompute in the app, with no export step.
- A basis for quality improvement. For example, the analysis shows Depression (14.05%) and Back Pain (13.98%) with the highest readmission rates, against 12.03% overall.

1.1.7 Scope, assumptions, constraints and feasibility

In scope: the six supplied CSV files, the Snowflake objects in `query.sql`, the Streamlit app in `Health/`, data-quality checks, KPIs and dashboards.

Out of scope: live EHR/HIS integration, real-time streaming, predictive models, patient-facing features, and production RBAC (proposed only).

Assumptions: - The CSV data is synthetic or anonymised. Names repeat across records (20 first names and 15 surnames across 3,000 patients), which suggests generated data. - Monetary values share one currency. The source does not name it, and the app labels values with `$`. - `READMISSION_WITHIN_30_DAYS` is pre-computed upstream.

Constraints: a Snowflake account with a warehouse (`COMPUTE_WH`); Streamlit in Snowflake with the packages its runtime allows; academic timeline.

Feasibility:

Dimension	Assessment
Technical	**High.** All required objects are standard Snowflake SQL. The app uses `streamlit`, `pandas`, `altair` and Snowpark, all available in Streamlit in Snowflake.
Data	**High.** The data is complete: no NULL or duplicate keys, and every foreign key resolves (verified, see Phase 6).
Economic	**High.** A data volume of about 1.4 MB of CSV fits comfortably in an X-Small warehouse or a trial account.
Operational	**Medium.** Loading is a manual step today, and scheduling and RBAC are proposed.

1.1.8 Proposed technology stack (verified)

Layer	Technology	Verified in
Storage / warehouse	Snowflake (`HEALTH_DB` ; schemas `RAW` , `STAGING` , `CURATED` , `MART`)	`query.sql`
Ingestion	Snowflake file format `RAW.CSV_FORMAT` and internal stage `RAW.HEALTHCARE_STAGE`	`query.sql`
Transformation	Snowflake SQL (CTAS, views)	`query.sql`
Application	Streamlit in Snowflake, Python 3.11, pandas, Altair, Snowpark	`Health/*.py` , `pyproject.toml` , `snowflake.yml`
Deployment descriptor	Snowflake CLI project definition v2 (`snowflake.yml`)	`Health/snowflake.yml`

1.1.9 Expected outcomes and success criteria

#	Success criterion	Measure	Status
SC-1	All six sources load into RAW with no row loss	RAW count = CSV rows	Verified locally (Phase 6, TC-ING-01…06)
SC-2	Zero NULL or duplicate business keys after staging	Null/duplicate checks = 0	Verified locally
SC-3	Every visit links to patient, doctor, facility, diagnosis and date	Orphan count = 0	Verified locally
SC-4	KPIs reproducible from source	View KPIs = independent pandas result	Verified locally (TC-KPI-01…06)
SC-5	Dashboard responds to all five filters	Filtered counts correct	Logic verified locally; live app **Not Executed**

1.2 Define the Problem Statements

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	2 Marks

Customer Problem Statement Template: Create a problem statement to understand your customer's point of view.

Table 1.3 — Customer problem statements

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	a hospital administrator responsible for 10 facilities and 60 doctors	see which facilities and doctors are performing well on visit volume, cost, length of stay and satisfaction	visit, doctor and facility data sit fragmented across raw operational records that I must join by hand	there is no structured warehouse linking each visit to its doctor and facility	frustrated and unsure whether the comparisons I present are correct
PS-2	a finance team member	find which diagnoses drive the longest and most expensive stays, and how that cost is split between insurance and the patient	length of stay and cost components are buried in visit-level records with no diagnosis view	no consolidated, queryable model aggregates cost and length of stay by diagnosis	under pressure, because each cost review takes days to rebuild
PS-3	a hospital administrator responsible for patient experience	find where patient satisfaction is at risk, and where readmissions are high, by facility, doctor and diagnosis	satisfaction scores and readmission flags are recorded per visit but never summarised	there is no shared KPI definition or filterable view	worried that dissatisfied patients and preventable readmissions go unnoticed

1.3 Empathy Map Canvas

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	4 Marks

An empathy map is a simple, easy-to-digest visual that captures knowledge about a user's behaviours and attitudes. The persona here is a **hospital administrator / finance manager** (combined persona). The entries are design assumptions derived from the project objectives, not interview transcripts.

Table 1.4 — Empathy map: Hospital administrator / finance manager

SAYS	THINKS
"I need one number for total visits that everyone agrees on." "Which facility is the most expensive per visit?" "Why are readmissions going up?"	Are my spreadsheets joined correctly? Is a cost increase driven by room charges or by medication? Are some doctors overloaded?
DOES	**FEELS**
Exports CSVs from separate systems every month Uses VLOOKUP to combine visits with doctor and facility names Builds static charts for review meetings	Overwhelmed by manual work Anxious about presenting wrong figures Wants confidence and control
PAINS	**GAINS**
Inconsistent category labels Slow turnaround No drill-down by date, facility, specialty or diagnosis	One consolidated, governed dataset KPIs refresh with filters in seconds Comparable facility, doctor and diagnosis metrics

1.4 Brainstorm & Idea Prioritization

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	4 Marks

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process that leads to problem solving.

Step 1: Brainstorm and Idea Listing

Table 1.5 — Brainstormed ideas

S.No	Team Member	Idea / Suggestion	Category	Group No.
1	[Team Member 1]	Load all six CSVs into Snowflake through a stage and file format	Data ingestion	1
2	[Team Member 2]	Separate RAW, STAGING, CURATED and MART schemas for traceability	Data architecture	1
3	[Team Member 3]	Standardise text with TRIM/INITCAP and drop rows with NULL keys	Data quality	1
4	[Team Member 4]	Star schema: one visit fact plus patient, doctor, facility, diagnosis and date dimensions	Data modelling	2
5	[Team Member 1]	SQL views for KPIs: visits, cost, LOS, satisfaction, readmission rate	Analytics	2

S.No	Team Member	Idea / Suggestion	Category	Group No.
6	[Team Member 2]	Interactive Streamlit app inside Snowflake with sidebar filters	Visualisation	3
7	[Team Member 3]	Tableau Public dashboard on extracted data	Visualisation	3
8	[Team Member 4]	Predict readmission risk with an ML model	Advanced analytics	4
9	[Team Member 1]	Automate refresh with Snowflake Streams and Tasks	Automation	4
10	[Team Member 2]	Role-based access and masking of patient names	Security	4

Step 2: Idea Prioritization

Table 1.6 — Idea prioritisation

Group No.	Final Idea	Feasibility (High/Medium/Low)	Importance (High/Medium/Low)	Priority	Selected (Yes/No)
1	Layered Snowflake ingestion and cleaning pipeline (RAW → STAGING)	High	High	1	Yes (implemented)
2	Star-schema CURATED layer and MART KPI views	High	High	2	Yes (implemented)
3	Streamlit-in-Snowflake interactive dashboard (chosen over Tableau, to keep data and app in one platform)	High	High	3	Yes (implemented)
4	Automation, RBAC/masking and ML readmission prediction	Medium	Medium	4	No: deferred to future scope (Phase 8)

Phase 2 — Requirement Analysis

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Customer Journey Map · Solution Requirements (4 marks) · Data Flow Diagram & User Stories (4 marks) · Technology Stack (2 marks)

> **Template note.** The Drive folder lists `Customer Journey Map.pdf`, but that file was not among the uploads. Section 2.2 therefore follows the conventional journey-map layout (stages × actions, touchpoints, emotions, pain points, opportunities). Replace it with the official layout if it differs. The other three templates were received; their headings and table columns are reproduced exactly. They carry the printed label "Project Design Phase-II".

2.1 Problem Statement and Business Objectives

Problem statement (official): Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.

The statement poses three questions. Each maps to project components as follows:

Table 2.0 — Problem-statement questions and how they are answered

#	Question in the problem statement	How the solution answers it	Finding (full dataset, local replica)
Q1	Which facilities and doctors are performing well?	`VW_FACILITY_ANALYTICS`, `VW_DOCTOR_ANALYTICS`; app facility chart and table, doctor leaderboard (visits, avg cost, satisfaction, readmissions)	Facility volume 958–1,039 visits; City General Hospital busiest (1,039). Highest doctor satisfaction: Dr. Neha Brown, D002 (4.02)
Q2	Which diagnoses drive the longest and most expensive stays?	`VW_DIAGNOSIS_ANALYTICS` (avg cost, avg LOS); app filter Visit Type = Inpatient with diagnosis charts	Inpatient stays: longest Fracture (8.60 days avg), Anemia (8.32), Diabetes (8.07); most expensive Kidney Disease (14,816.82 avg), Hypertension (14,640.75), Asthma (14,549.67)
Q3	Where is patient satisfaction at risk?	Avg Satisfaction KPI; satisfaction columns in facility and doctor tables and in `VW_DIAGNOSIS_ANALYTICS`	Lowest facility: Sunrise Hospital (3.78; 13.9% of visits score 1–2). Lowest diagnosis: Asthma (3.75). Lowest doctors: Dr. Rohan Singh, D060 (3.65), Dr. Rohan Gupta, D004 (3.69)

The project's business objectives are:

ID	Objective	Implemented by
BO-1	Consolidate patient, doctor, facility, diagnosis and visit data	`CURATED.FACT_PATIENT_VISITS` (LEFT JOIN of staging tables)
BO-2	Analyse visit volumes, treatment costs and LOS	`VW_EXECUTIVE_KPI`, `VW_MONTHLY_VISITS`, `VW_VISIT_TYPE_DISTRIBUTION`; app KPI cards and trend
BO-3	Compare facilities, doctors and diagnoses	`VW_FACILITY_ANALYTICS`, `VW_DOCTOR_ANALYTICS`, `VW_DIAGNOSIS_ANALYTICS`; app facility chart and table, doctor leaderboard, top-10 diagnoses
BO-4	Monitor satisfaction and 30-day readmission	`VW_READMISSION_ANALYTICS`; app readmission-rate card and chart, satisfaction card
BO-5	Analyse insurance coverage, patient payment and cost breakdown	`VW_EXECUTIVE_KPI` totals; app cost-breakdown chart. **Partial:** no per-insurer view or chart exists
BO-6	Present insights through an interactive, filterable Streamlit app	`Health/streamlit_app.py` (5 filters)

2.2 Customer Journey Map

(Official template not received; conventional structure used.) Persona: hospital administrator / finance manager preparing the monthly performance review.

Table 2.1 — Customer journey map

Stage	1. Need arises	2. Gather data	3. Prepare & analyse	4. Present & decide	5. Follow up
Actions (today)	Month-end review is scheduled	Requests CSV exports from registration, HR roster, facility master, coding and billing	Joins files in spreadsheets; fixes inconsistent labels; computes averages	Shares static charts in a meeting	Repeats the cycle next month
Touchpoints	Calendar, email	Source systems, IT team	Excel / Sheets	Slides	Email
Emotions	Neutral	Impatient	Stressed	Anxious about accuracy	Fatigued
Pain points	Short deadline	Waiting on several teams	Manual joins, label mismatches	Cannot answer follow-up "what if" questions live	No reuse of prior work
Opportunity (with the Hub)	Self-service access	CSVs staged once into `HEALTH_DB.RAW`	STAGING cleans; CURATED joins; MART computes KPIs	Streamlit filters answer questions live	Re-run `query.sql` (or a future Task) to refresh
Emotion (with the Hub)	Confident	Relieved	In control	Credible	Efficient

2.3 Solution Requirements (Functional & Non-functional)

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	4 Marks

Functional Requirements: Following are the functional requirements of the proposed solution.

Table 2.2 — Functional requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)	Priority (High/Medium/Low)
1	Data Access & Integration	Access six business datasets: PATIENTS (8 cols, 3,000 rows), DOCTORS (5 cols, 60), FACILITIES (4 cols, 10), DIAGNOSES (3 cols, 15), DATE (8 cols, 974) and PATIENT_VISITS (19 cols, 10,000). Integrate them in Snowflake `HEALTH_DB` through file format `RAW.CSV_FORMAT` (header skip, NULL_IF, `YYYY-MM-DD` dates) and internal stage `RAW.HEALTHCARE_STAGE` into typed `*_RAW` tables.	High
2	Data Processing	Trim all text keys and attributes; standardise case of categorical fields with `INITCAP` ; remove rows with NULL business keys; build five deduplicated dimensions (`SELECT DISTINCT`); build `FACT_PATIENT_VISITS` at visit grain by LEFT JOIN to patient, doctor, facility and diagnosis attributes; run NULL-key, duplicate-key, row-count and orphan checks.	High
3	Business Analysis	Compute total visits, unique patients, doctors, facilities; total and average cost; total insurance coverage and patient payment; average satisfaction; average LOS; readmission count and rate (% of visits flagged "Yes"); monthly visit and cost trend; aggregates by diagnosis, facility, doctor, visit type, treatment status and admission status; average cost component by visit type.	High
4	Business Insights & Analytical Outputs	Ten MART views (`VW_EXECUTIVE_KPI` … `VW_READMISSION_ANALYTICS`) and a Streamlit dashboard with 6 KPI cards, monthly trend line, visit-type and admission-status donuts, top-10 diagnoses bar, facility bar plus table, treatment-status bar, readmission-rate-by-diagnosis bar, cost-breakdown stacked bar and a doctor leaderboard table.	High
5	User Interaction & Analysis	Sidebar filters for visit date range, facility, doctor specialty, visit type and diagnosis category, with an "All" option on each. Every KPI, chart and table recomputes on the filtered set. Chart tooltips show the underlying values. Tables are sortable (Streamlit `st.dataframe`).	High

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)	Priority (High/Medium/Low)
6	Data Update & Reporting	Re-running `query.sql` rebuilds STAGING and CURATED (`CREATE OR REPLACE`); MART views always read current CURATED data; the app caches the fact table for 600 s (`@st.cache_data(ttl=600)`), then re-reads it. *Proposed:* scripted `COPY INTO` and scheduled Tasks.	Medium

Non-functional Requirements: Following are the non-functional requirements of the proposed solution.

Table 2.3 — Non-functional requirements

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	Single-page wide layout; filters grouped in the sidebar; every chart titled with axis labels; KPI cards formatted with thousands separators and units ("/ 5", "%", "days"); a footer shows the filtered visit count and source table.
NFR-2	Security	App uses the caller's Snowflake session (`get_active_session()`), with no credentials in code or config. Data stays inside Snowflake. *Proposed:* least-privilege roles and masking policies for patient names and date of birth (see Phase 3 §3.9).
NFR-3	Reliability	Deterministic, idempotent rebuild (`CREATE … IF NOT EXISTS` / `CREATE OR REPLACE`); built-in validation queries for NULLs, duplicates, row counts and orphans; app guards divide-by-zero when a filter returns no rows.
NFR-4	Performance	MART queries and app filters should return in under 5 s on an X-Small warehouse for the current 10,000-row volume. The fact table is cached in the app for 10 minutes. *Measured locally only (Phase 6); Snowflake timings Not Executed.*
NFR-5	Availability	Availability is inherited from the Snowflake service and warehouse auto-resume; the app is reachable to authorised Snowsight users whenever the warehouse can start.
NFR-6	Scalability	Compute scales by resizing the warehouse independently of storage. *Limitation:* the app loads the whole fact table into pandas, so very large volumes should push aggregation to SQL/MART views (Phase 8).

2.3.1 Stakeholder requirements

Stakeholder	Requirement
Administrator	Facility and doctor comparison on visits, cost, LOS and satisfaction; date filtering
Finance	Total cost, insurance versus patient share, cost by component and visit type
Quality lead	Readmission rate by diagnosis and doctor; treatment status mix
Data engineer	Re-runnable scripts; layer separation; validation output

2.3.2 User roles and use cases

UC	Actor	Use case	Main flow
UC-1	Data engineer	Load and transform data	Upload CSVs to stage → load RAW → run STAGING, CURATED and MART sections of `query.sql` → review validation results
UC-2	Administrator	Review overall performance	Open app → read KPI cards → inspect monthly trend
UC-3	Administrator	Compare facilities	Select date range → view facility bar and table → sort by average satisfaction
UC-4	Finance analyst	Analyse cost drivers	Select visit type → read cost-breakdown chart → query `VW_EXECUTIVE_KPI` for insurance and patient totals
UC-5	Quality lead	Investigate readmissions	Filter diagnosis category → view readmission-rate chart and doctor leaderboard readmissions

2.3.3 Source data and data requirements

Dataset	Grain	Key	Rows	Required fields
patients.csv	patient	patient_id	3,000	gender, date_of_birth, city, insurance_provider, registration_date
doctors.csv	doctor	doctor_id	60	doctor_name, specialty, experience_years, department
facilities.csv	facility	facility_id	10	facility_name, city, facility_type

Dataset	Grain	Key	Rows	Required fields
diagnoses.csv	diagnosis	diagnosis_id	15	diagnosis_name, diagnosis_category
date.csv	calendar day	date	974 (2024-01-01 → 2026-08-31)	year, quarter, month, month_name, week, day_of_week, is_weekend
patient_visits.csv	visit	visit_id	10,000	4 FKs, visit_date, visit_type, statuses, LOS, 5 cost fields, insurance_coverage, patient_payment, satisfaction, readmission flag

The complete column list is in `DATA_DICTIONARY.md`.

2.3.4 Business questions the solution answers

1. How many visits, unique patients, doctors and facilities are there in a period?
2. What is the monthly trend in visits, cost and satisfaction?
3. Which visit types are most frequent and most expensive?
4. Which diagnoses have the highest volume, cost and LOS?
5. Which facilities handle the most visits, and how do they compare on average cost, satisfaction and LOS?
6. Which doctors have the most visits, and what are their average cost, satisfaction and readmissions?
7. What is the 30-day readmission rate overall and by diagnosis?
8. How is total cost split between consultation, medication, lab and room, and between insurance and the patient?
9. What is the mix of treatment outcomes and admission status?

2.3.5 Data quality requirements

DQ-ID	Dimension	Rule
DQ-1	Completeness	Business keys are never NULL in STAGING
DQ-2	Uniqueness	Each business key is unique per table
DQ-3	Referential integrity	Every visit FK matches a dimension row; every visit date exists in DATE
DQ-4	Validity	Satisfaction 1–5; LOS ≥ 0; costs ≥ 0; readmission flag ∈
DQ-5	Consistency	TOTAL_COST = sum of the four components; INSURANCE_COVERAGE + PATIENT_PAYMENT = TOTAL_COST
DQ-6	Conformity	Categorical labels consistently cased (`INITCAP`), with acronyms preserved

2.3.6 Hardware and software requirements

Type	Requirement
Client hardware	Any laptop or desktop with a modern browser and 4 GB+ RAM
Cloud platform	Snowflake account (trial is sufficient) with a role able to create databases and Streamlit apps; warehouse `COMPUTE_WH` (X-Small sufficient)
Streamlit runtime	Streamlit in Snowflake; Python `~=3.11.0` ; dependency `streamlit[snowflake]` (from `pyproject.toml`); `snowflake.yml` names compute pool `SYSTEM_COMPUTE_POOL_CPU`
Optional local tools	Snowflake CLI (to deploy with `snow streamlit deploy`); Python 3.11 with pandas to run `tests/local_validation.py`

2.3.7 Constraints, risks, assumptions and acceptance criteria

Type	Item
Constraint	Data limited to supplied CSVs; no live hospital-system feed
Constraint	Streamlit in Snowflake only allows packages available to its runtime
Risk	Manual loading step may be skipped or repeated (duplicates). Mitigation: `CREATE OR REPLACE` RAW tables before each load; row-count checks
Risk	Patient names in fact table expose PII. Mitigation (proposed): masking policy, analyst role on MART only
Assumption	Readmission flag is calculated correctly upstream

Type	Item
Acceptance	All DQ-1 to DQ-5 checks return 0 failures; KPI view matches an independent recomputation; all 5 filters change all visuals; app loads without error in Snowsight

2.4 Data Flow Diagram & User Stories

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	4 Marks

Data Flow Diagrams: A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. It shows how data enters and leaves the system, what changes the information, and where data is stored.

DFD Symbol Legend

Symbol	Name	Description
Oval / Rounded shape	External Entity	A person, organisation, or system outside the project that sends or receives data (here: CSV source extracts; administrator / finance user).
Rectangle with numbered header	Process	An activity that transforms incoming data into outgoing data (1.0 Stage & load … 5.0 Filter & visualise).
Rectangle (solid fill, no number)	Data Store	A place where data is held for later use (D1 RAW … D4 MART).
Labeled arrow	Data Flow	The movement of data between entities, processes and data stores.

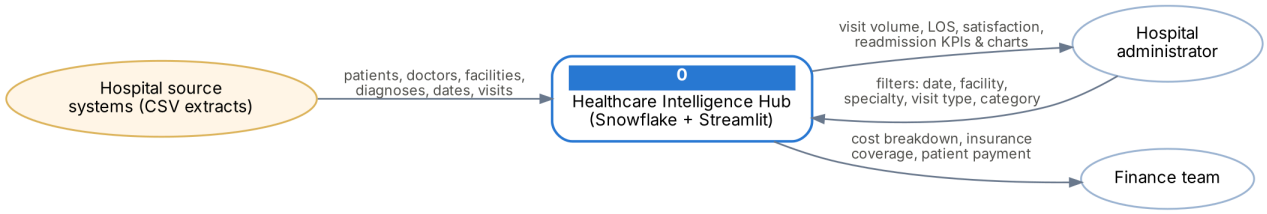


Figure 2.1 — Context diagram (DFD Level 0)

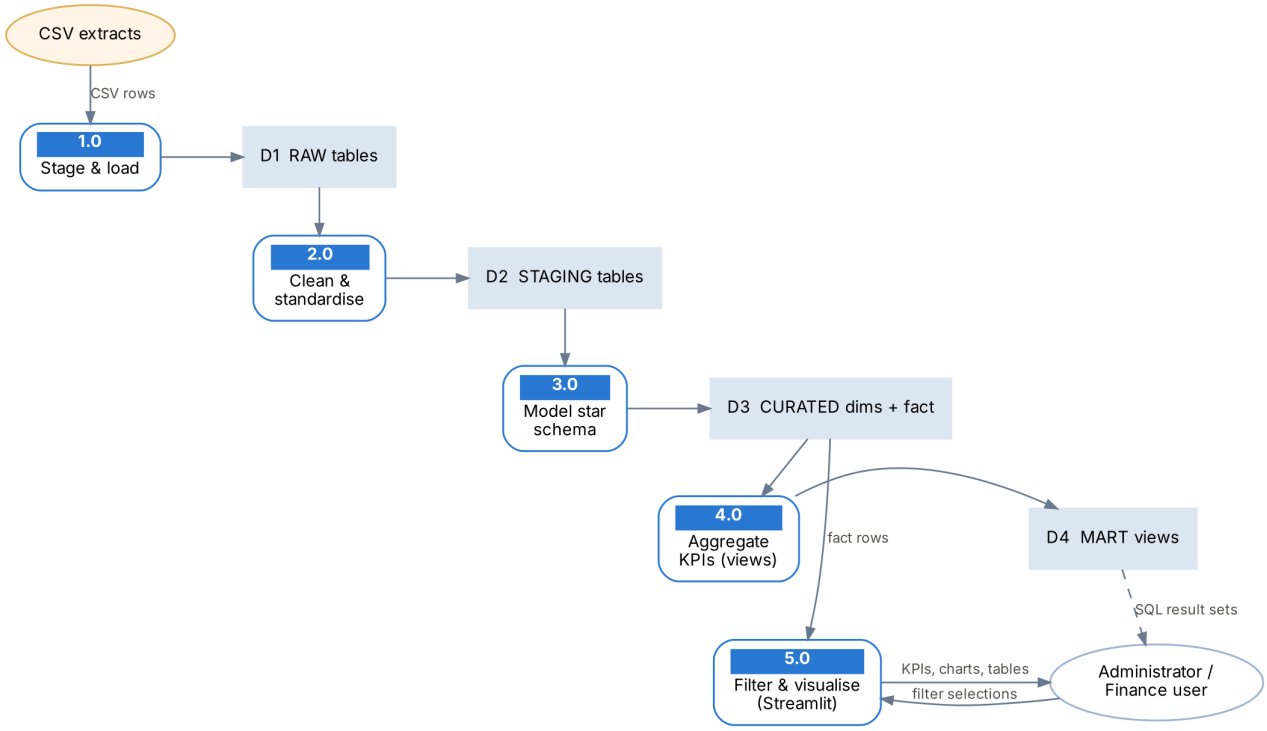


Figure 2.2 — DFD Level 1: processes 1.0–5.0 and data stores D1–D4, as implemented in query.sql and streamlit_app.py

User Stories: Use the below template to list all the user stories for the product.

Table 2.4 — User stories

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Data Engineer	Data Access & Integration	USN-1	As a data engineer, I can create `HEALTH_DB` with RAW, STAGING, CURATED and MART schemas.	`SHOW SCHEMAS IN DATABASE HEALTH_DB` lists all four	High	Sprint-1
Data Engineer	Data Access & Integration	USN-2	As a data engineer, I can define a CSV file format and an internal stage, and load the six CSVs into typed RAW tables.	RAW row counts = 3,000 / 60 / 10 / 15 / 974 / 10,000	High	Sprint-1
Data Engineer	Data Processing	USN-3	As a data engineer, I can build cleaned STAGING tables with trimmed, case-standardised values and non-NULL keys.	Null-key and duplicate-key checks return 0	High	Sprint-2
Data Engineer	Data Processing	USN-4	As a data engineer, I can build five dimensions and a visit-grain fact table.	FACT rows = 10,000; orphan checks return 0	High	Sprint-2
Hospital Administrator	Business Analysis	USN-5	As an administrator, I can see total visits, unique patients, total cost, average satisfaction, readmission rate and average LOS.	Six KPI cards render, matching `VW_EXECUTIVE_KPI`	High	Sprint-3

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Hospital Administrator	Business Insights	USN-6	As an administrator, I can view the monthly visit trend and visit-type split.	Line chart and donut render	High	Sprint-3
Hospital Administrator	Business Insights	USN-7	As an administrator, I can compare facilities on visits, average cost and satisfaction.	Facility bar and table render, sorted by visits	High	Sprint-3
Hospital Administrator	Business Insights	USN-8	As an administrator, I can see a doctor leaderboard with visits, average cost, satisfaction and readmissions.	Table lists every doctor	Medium	Sprint-3
Finance Analyst	Business Analysis	USN-9	As a finance analyst, I can see average consultation, medication, lab and room cost by visit type.	Stacked bar renders	High	Sprint-3
Finance Analyst	Business Analysis	USN-10	As a finance analyst, I can query total insurance coverage and patient payment.	`VW_EXECUTIVE_KPI` returns both totals	Medium	Sprint-3
Quality Lead	Business Insights	USN-11	As a quality lead, I can see readmission rate by diagnosis.	Top-10 bar of readmission % renders	High	Sprint-3
All dashboard users	User Interaction	USN-12	As a user, I can filter by date range, facility, specialty, visit type and diagnosis category.	All KPIs and charts change with filters	High	Sprint-4
Data Engineer	Data Update & Reporting	USN-13	As a data engineer, I can deploy the app with `snowflake.yml`.	App opens in Snowsight	Medium	Sprint-4
Administrator (security)	Data Update & Reporting	USN-14	As an administrator, I can restrict access by role and mask patient names.	*Proposed: not implemented*	Low	Future

2.5 Technology Stack (Architecture & Stack)

Field	Value
Date	31 January 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	2 Marks

Define Your Technology Stack: Identify the technologies, platforms, databases, BI tools, and integrations used to build the Business Intelligence solution.

Table 2.5 — Technology stack

S.No	Component	Description	Technology	Justification / Purpose
1.	Programming / Query Language	Language used to query, filter, transform, aggregate, and analyse data.	**SQL (Snowflake dialect)**; **Python 3.11**	SQL implements all layers and KPI views (`query.sql`); Python drives the Streamlit app and in-app aggregations (pandas).
2.	Database / Data Warehouse	Warehouse used to store and manage structured business data for analysis and reporting.	**Snowflake**: database `HEALTH_DB` ; schemas `RAW` , `STAGING` , `CURATED` , `MART` ; warehouse `COMPUTE_WH`	Separates storage and compute, gives full SQL, and runs Streamlit natively.
3.	Data Ingestion	Used to load data from source files or external systems into the analytical environment.	**Snowflake internal stage** `RAW.HEALTHCARE_STAGE` + **file format** `RAW.CSV_FORMAT` ; CSV source files	Standard, typed CSV loading. *Note:* the `PUT` / `COPY INTO` commands are not in `query.sql` , so the load was presumably done through the Snowsight UI (not verifiable from the files).

S.No	Component	Description	Technology	Justification / Purpose
4.	Data Transformation and Business Logic	Tools/logic used to clean, join, filter, aggregate data and calculate KPIs, metrics and business rules.	**Snowflake SQL**: CTAS (`TRIM` , `INITCAP` , `WHERE … IS NOT NULL` , `SELECT DISTINCT` , `LEFT JOIN`), views (`COUNT` , `SUM` , `AVG` , `ROUND` , `CASE` , `DATE_TRUNC`); **pandas** in the app	Logic is versionable SQL, close to the data; pandas recomputes metrics after filtering.
5.	Data Visualization / Analytics	Used to create interactive dashboards, charts, reports, KPIs and visual analysis.	**Altair** (line, arc/donut, bar, stacked bar) + **Streamlit** `st.metric` , `st.dataframe`	Declarative charts with tooltips, bundled with Streamlit.
6.	Front-End / Interface	User interface through which users interact with dashboards, filters, parameters and visual insights.	**Streamlit in Snowflake** (`streamlit_app.py` , deployed by `snowflake.yml` , title "HEALTH")	Native to Snowflake, no separate hosting, uses the user's Snowflake session. Tableau was considered but not used.

Phase 3 — Project Design Phase

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Problem – Solution Fit (2 marks) · Project Proposal (Proposed Solution) · Solution Architecture (4 marks)

> **Template note.** `Project Proposal (Proposed Solution) template.docx` appears in the Drive folder but was not uploaded. Section 3.2 uses the widely used six-parameter "Proposed Solution" format: problem, idea, novelty, social impact, business model, scalability. Replace it with the official layout if it differs. The diagrams show only objects that exist in `query.sql` or `Health/`, unless a component is labelled *proposed* and drawn dashed.

3.1 Problem – Solution Fit

Field	Value
Date	[Submission Date] *(template shows 19 September 2022)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	2 Marks

Problem – Solution Fit Template: The Problem-Solution Fit simply means that you have found a problem with your customer and that the solution you have realised for it actually solves the customer's problem.

Purpose: Solve complex problems in a way that fits the state of your customers. Succeed faster and increase solution adoption by tapping into existing channels of behaviour. Sharpen communication with the right triggers. Understand the existing situation in order to improve it.

Table 3.1 — Problem–Solution Fit canvas (CC BY-NC-ND 4.0 canvas by Daria Nepriakhina / Amaltama)

<table>
<tr><th>1. CUSTOMER SEGMENT(S) — CS</th><th>6. CUSTOMER CONSTRAINTS — CC</th><th>5. AVAILABLE SOLUTIONS — AS</th></tr>
<tr><td>Hospital administrators and finance teams in a multi-facility hospital network (10 facilities, 60 doctors); secondary: clinical quality leads.</td><td>Limited analyst time; no data-engineering staff; data spread across systems; patient privacy obligations; budget for tools.</td><td>Monthly spreadsheet exports joined with VLOOKUP; static slide charts; vendor HIS reports. Pros: familiar. Cons: manual, slow, inconsistent, no drill-down.</td></tr>
<tr><th>2. JOBS-TO-BE-DONE / PROBLEMS — J&P</th><th>9. PROBLEM ROOT CAUSE — RC</th><th>7. BEHAVIOUR — BE</th></tr>
<tr><td>Get a consolidated, queryable view of visits, spanning cost, LOS, satisfaction and readmissions. Answer: which facilities and doctors are performing well; which diagnoses drive the longest and most expensive stays; where patient satisfaction is at risk.</td><td>Visit data sits fragmented across raw operational records, and there is no structured warehouse linking each visit to patient, doctor, facility and diagnosis. KPI definitions are not shared, and labels are inconsistent between sources.</td><td>Request extracts from IT; clean data by hand; build pivot tables; email static reports before review meetings.</td></tr>
<tr><th>3. TRIGGERS — TR</th><th rowspan="2">10. YOUR SOLUTION — SL</th><th>8. CHANNELS OF BEHAVIOUR — CH</th></tr>
<tr><td>Month-end or quarterly reviews; rising costs; an unexpected readmission spike; board or regulator questions.</td><td>8.1 Online: Snowsight worksheets and the Streamlit app; email for sharing extracts. 8.2 Offline: review meetings, printed reports, discussions with department heads.</td></tr>
<tr><th>4. EMOTIONS: BEFORE / AFTER — EM</th><td rowspan="2">A Snowflake pipeline (RAW → STAGING → CURATED → MART) that consolidates six sources into a visit-grain star schema, with ten KPI views and a Streamlit-in-Snowflake dashboard offering six KPIs, eight charts, two tables and five filters. Runs in one platform, needs no credentials in the app, and can be rebuilt by re-running one script.</td><td></td></tr>
<tr><td>Before: overwhelmed, unsure of accuracy, reactive.
After: confident, in control, able to answer follow-up questions live.</td><td></td></tr>
</table>

3.2 Proposed Solution (Project Proposal)

Field	Value
Date	[Submission Date]

Field	Value
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	[as per template]

Table 3.2 — Proposed solution

S.No.	Parameter	Description
1	Problem Statement (Problem to be solved)	Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.
2	Idea / Solution description	Load six CSV sources into Snowflake `HEALTH_DB` ; clean them in `STAGING` ; model a visit-grain fact table and five dimensions in `CURATED` ; encode KPIs in ten `MART` views; deliver an interactive Streamlit-in-Snowflake dashboard with date, facility, specialty, visit-type and diagnosis-category filters.
3	Novelty / Uniqueness	End-to-end inside one governed platform: storage, transformation, KPI logic and UI. The app authenticates through the active Snowpark session, with no secrets. Every metric traces back through layered schemas. Built-in validation queries run at each layer.
4	Social Impact / Customer Satisfaction	Faster identification of high-readmission diagnoses (e.g., Depression at 14.05%) and low-satisfaction areas supports better patient care. Transparency on out-of-pocket payment (about 27.7% of total cost) supports fairer financial counselling.
5	Business Model (Revenue Model)	Internal analytics capability: value comes from reduced reporting effort and better cost and quality decisions. Running cost is Snowflake consumption, which is pay-per-use with auto-suspended warehouses. It could be packaged as a template for other hospital groups.
6	Scalability of the Solution	Snowflake scales compute (warehouse size, multi-cluster) separately from storage. New facilities or months need only new rows. To scale the app beyond about 10^6 rows, move filtering and aggregation into SQL (MART views or Snowpark) instead of loading the full fact table into pandas (Phase 8).

3.3 Solution Architecture

Field	Value
Date	19 September 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	4 Marks

Solution Architecture: Solution architecture bridges the gap between business problems and technology solutions. Its goals are to find the best technical solution, describe the structure and behaviour of the software to stakeholders, define features and phases, and provide specifications for delivery.

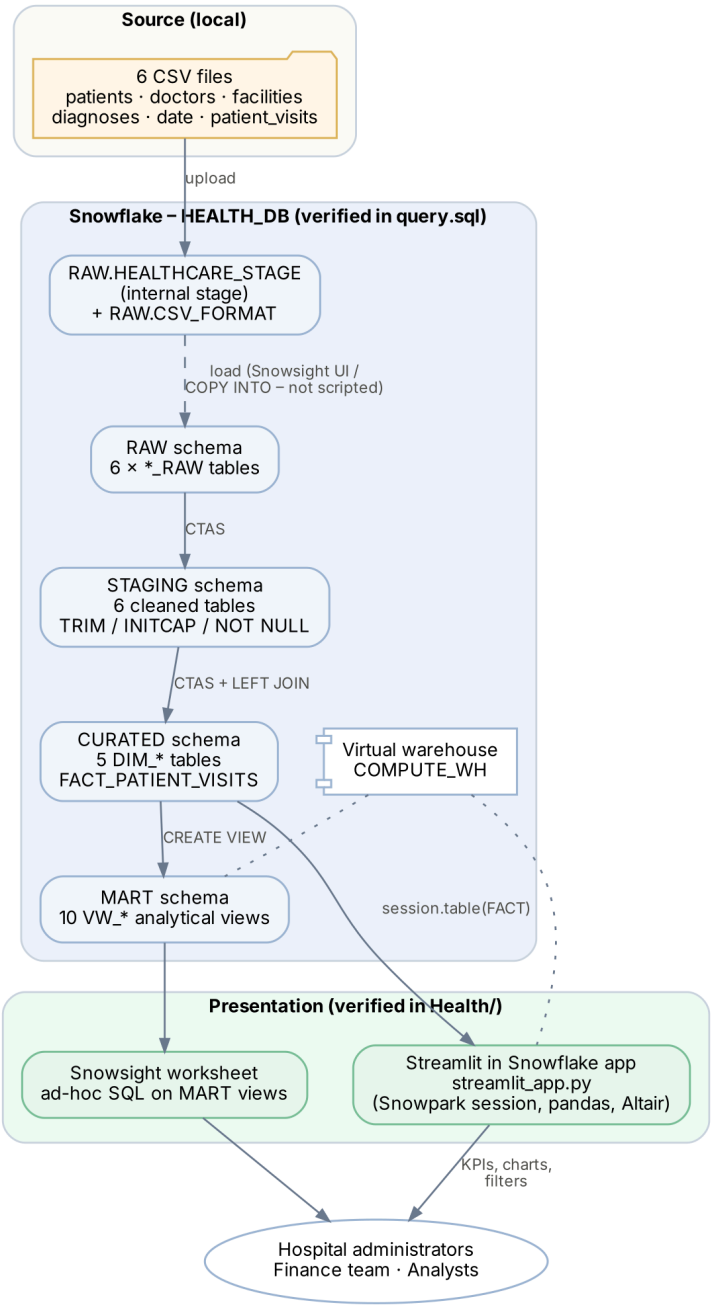


Figure 3.1 — Solution architecture as implemented. The dashed load arrow marks the one step that is not scripted in the repository.

Component Description Table

Table 3.3 — Architecture components

Component Name	Description / Role in Architecture	Technology Used
Source CSV files	Six extracts: patients, doctors, facilities, diagnoses, date and patient_visits (about 1.4 MB, 14,059 data rows)	CSV (UTF-8, header row, ISO dates)
CSV file format	Parsing rules: `SKIP_HEADER = 1` , `FIELD_OPTIONALLY_ENCLOSED_BY = '"'` , `NULL_IF = ('NULL','null','')` , `EMPTY_FIELD_AS_NULL` , `DATE_FORMAT = 'YYYY-MM-DD'`	Snowflake `FILE FORMAT HEALTH_DB.RAW.CSV_FORMAT`
Internal stage	Landing area for uploaded files	Snowflake `STAGE HEALTH_DB.RAW.HEALTHCARE_STAGE`
RAW layer	Typed copies of each source, unmodified	6 tables `*_RAW` in schema `RAW`
STAGING layer	Cleaned and standardised tables; NULL-key rows removed	6 CTAS tables in schema `STAGING`

Component Name	Description / Role in Architecture	Technology Used
CURATED layer	Star schema: 5 dimensions + visit fact with denormalised descriptive attributes	`DIM_PATIENT` , `DIM_DOCTOR` , `DIM_FACILITY` , `DIM_DIAGNOSIS` , `DIM_DATE` , `FACT_PATIENT_VISITS`
MART layer	Reusable KPI and aggregate logic	10 views `VW_*` in schema `MART`
Virtual warehouse	Compute for SQL and app queries	`COMPUTE_WH`
Streamlit app	Interactive dashboard: reads FACT through Snowpark, filters and aggregates in pandas, charts in Altair	Streamlit in Snowflake; `streamlit_app.py` ; object `HEALTH_DB.RAW.ICYNJI7QNBXR2IUI` , title "HEALTH"
Deployment descriptor	Declares app artefacts, warehouse and compute pool	`snowflake.yml` (definition_version 2), `pyproject.toml`

3.4 Data Layers and Pipeline Design

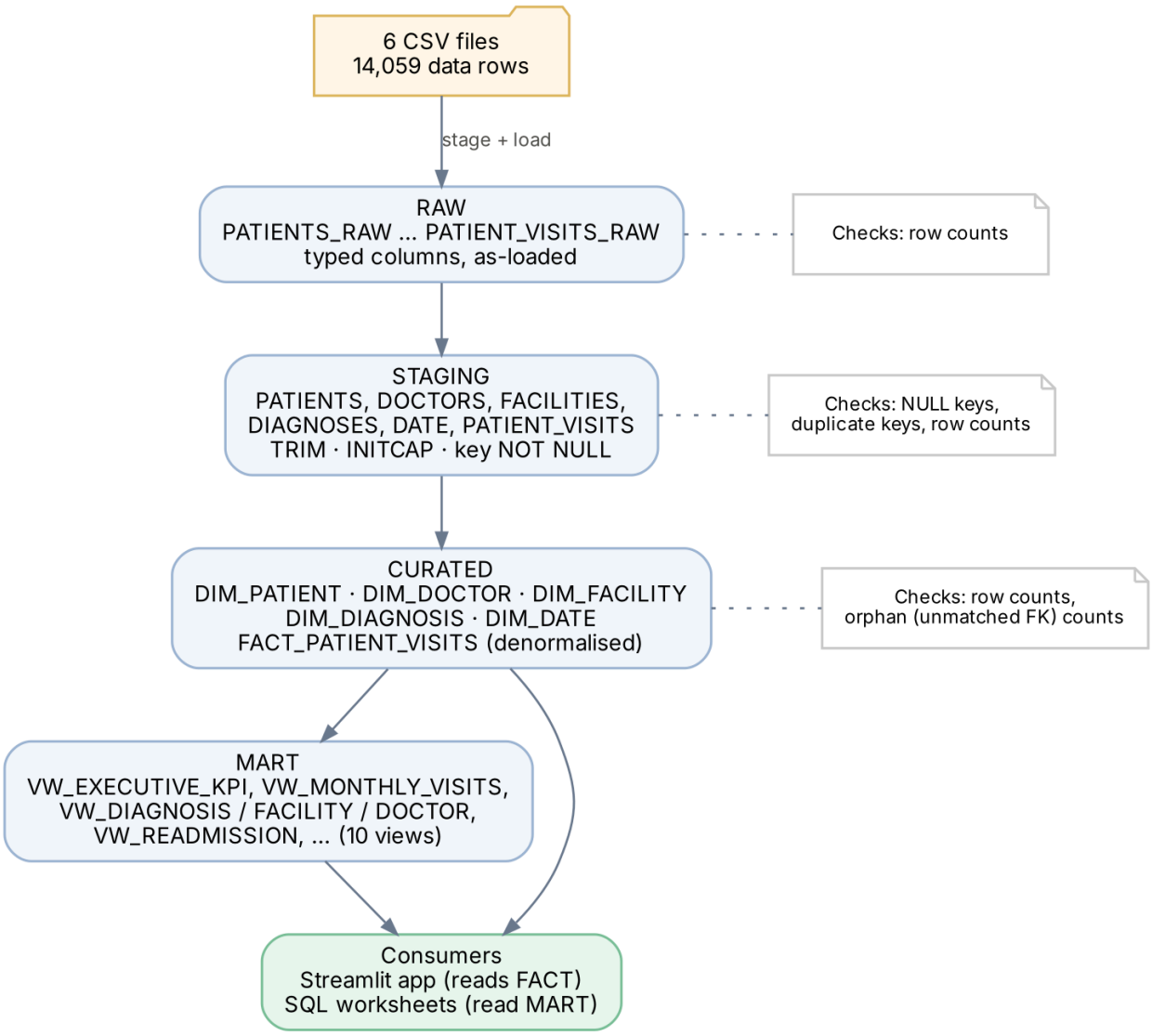


Figure 3.2 — Layered data pipeline with the validation queries run at each layer

Table 3.4 — Layer responsibilities

Layer	Schema	Objects	Responsibility	Materialisation
Source	—	6 CSV files	System-of-record extracts	Files

Layer	Schema	Objects	Responsibility	Materialisation
Raw	`RAW`	6 `*_RAW` tables, `CSV_FORMAT`, `HEALTHCARE_STAGE`	Land data with types; no business logic	Tables (`CREATE OR REPLACE`)
Staging (cleaned)	`STAGING`	`PATIENTS`, `DOCTORS`, `FACILITIES`, `DIAGNOSES`, `DATE`, `PATIENT_VISITS`	Trim, case-standardise, drop NULL keys	CTAS tables
Curated (analytical model)	`CURATED`	5 `DIM_*`, `FACT_PATIENT_VISITS`	Conformed dimensions; visit-grain fact	CTAS tables
Mart (consumption)	`MART`	10 `VW_*`	KPI and aggregate definitions	Views

3.5 Snowflake Database and Schema Design

```
HEALTH_DB
├── RAW       PATIENTS_RAW, DOCTORS_RAW, FACILITIES_RAW, DIAGNOSES_RAW, DATE_RAW,
│             PATIENT_VISITS_RAW, CSV_FORMAT (file format), HEALTHCARE_STAGE (stage),
│             ICYNJI7QNBXR2IUI (Streamlit app, per snowflake.yml)
├── STAGING   PATIENTS, DOCTORS, FACILITIES, DIAGNOSES, DATE, PATIENT_VISITS
├── CURATED   DIM_PATIENT, DIM_DOCTOR, DIM_FACILITY, DIM_DIAGNOSIS, DIM_DATE,
│             FACT_PATIENT_VISITS
└── MART      VW_EXECUTIVE_KPI, VW_MONTHLY_VISITS, VW_DIAGNOSIS_ANALYTICS,
              VW_FACILITY_ANALYTICS, VW_VISIT_TYPE_DISTRIBUTION, VW_TREATMENT_ANALYTICS,
              VW_ADMISSION_STATUS_ANALYTICS, VW_PATIENT_VISIT_TYPE_ANALYSIS,
              VW_DOCTOR_ANALYTICS, VW_READMISSION_ANALYTICS
```

Design observation: the Streamlit app object is created in the `RAW` schema (per `snowflake.yml`). A dedicated `APP` or `MART` schema would keep the raw layer free of consumption objects and simplify grants.

3.6 Fact and Dimension Model

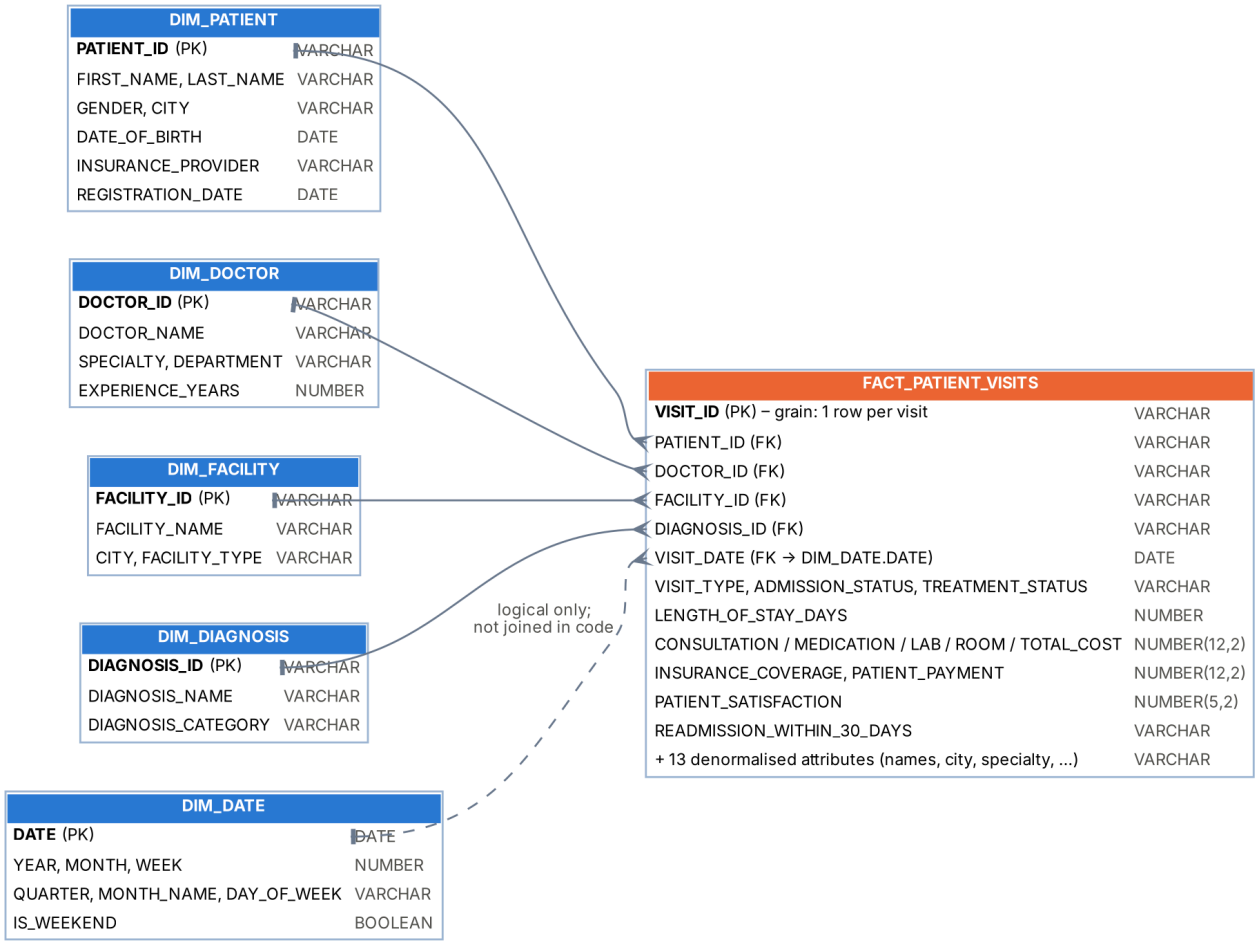


Figure 3.3 — ER diagram of the CURATED star schema (keys are logical: no PK/FK constraints are declared in query.sql)

Table 3.5 — Keys, grain and relationships

Table	Grain	Primary key (logical)	Foreign keys	Rows	Relationship to fact
`FACT_PATIENT_VISITS`	One row per patient visit	`VISIT_ID`	`PATIENT_ID`, `DOCTOR_ID`, `FACILITY_ID`, `DIAGNOSIS_ID`, `VISIT_DATE`	10,000	—
`DIM_PATIENT`	One row per patient	`PATIENT_ID`	—	3,000	1 : N (2,889 patients have ≥1 visit)
`DIM_DOCTOR`	One row per doctor	`DOCTOR_ID`	—	60	1 : N
`DIM_FACILITY`	One row per facility	`FACILITY_ID`	—	10	1 : N
`DIM_DIAGNOSIS`	One row per diagnosis	`DIAGNOSIS_ID`	—	15	1 : N
`DIM_DATE`	One row per calendar day	`DATE`	—	974	1 : N via `VISIT_DATE` (logical only)

Model characteristics (verified from code):

1. **Denormalised fact.** `FACT_PATIENT_VISITS` carries 13 descriptive attributes copied from the dimensions (patient name, gender, city, insurer, doctor name, specialty, department, facility name, city and type, diagnosis name and category). This lets the app and views read one table with no joins. The cost is redundancy, and dimension changes require a fact rebuild.
2. **DIM_DATE is built but unused.** No view or app code joins to it. Monthly grouping uses `DATE_TRUNC` on `VISIT_DATE`, so weekday and weekend attributes are not yet exploited.

3. **No declared constraints.** Snowflake records but does not enforce PK and FK constraints. Declaring them (`ALTER TABLE … ADD PRIMARY KEY`) would still help documentation and BI tools.
4. **Dimensions are rebuilt in full** (`CREATE OR REPLACE` , Type 1). There is no history (SCD Type 2).

3.7 Data Transformation Design

Table 3.6 — Transformation rules by layer

Step	Source → Target	Rule	Columns
T1	CSV → `*_RAW`	Parse with `CSV_FORMAT` ; empty and 'NULL' strings become NULL; dates parsed from `YYYY-MM-DD`	all
T2	`*_RAW` → `STAGING.*`	`TRIM()` on every text column	IDs, names, categorical values
T3	`*_RAW` → `STAGING.*`	`INITCAP(TRIM())` on categorical columns	GENDER, CITY, SPECIALTY, DEPARTMENT, FACILITY_TYPE, DIAGNOSIS_CATEGORY, VISIT_TYPE, ADMISSION_STATUS, TREATMENT_STATUS, READMISSION_WITHIN_30_DAYS
T4	`*_RAW` → `STAGING.*`	`WHERE <key> IS NOT NULL`	each business key
T5	`STAGING.*` → `DIM_*`	`SELECT DISTINCT` (removes exact duplicate rows)	all
T6	`STAGING.*` → `FACT_PATIENT_VISITS`	`LEFT JOIN` visits to patients, doctors, facilities and diagnoses on their IDs; rename `CITY` to `PATIENT_CITY` / `FACILITY_CITY`	19 visit columns + 13 attributes
T7	`FACT` → `MART.VW_*`	`COUNT` , `COUNT DISTINCT` , `SUM` , `AVG` , `ROUND(…,2)` , `CASE WHEN flag='Yes'` , `DATE_TRUNC('MONTH')` , `GROUP BY` , `ORDER BY`	measures

Design risks found during verification (details and evidence in Phase 6):

- T3 `INITCAP` lower-cases letters after the first and treats `-` as a word break. Snowflake will therefore produce `Ent` (from `ENT`) and `Follow-Up` (from `Follow-up`). Fix: exclude acronyms, or apply `INITCAP` only where the source is inconsistent.
- `VW_DOCTOR_ANALYTICS` and the app leaderboard group by `DOCTOR_NAME, SPECIALTY, DEPARTMENT` . Doctors D018 and D020 share all three values ("Dr. Ananya Reddy", Orthopedics, Specialty Care), so they merge into one row: 59 rows for 60 doctors. Fix: add `DOCTOR_ID` to the grouping.
- `VW_VISIT_TYPE_DISTRIBUTION` and `VW_PATIENT_VISIT_TYPE_ANALYSIS` return the same columns and differ only in sort order (redundant).

3.8 Application and Dashboard Design

The app is a single Streamlit page (`layout="wide"` , sidebar expanded) built in `streamlit_app.py` .

Table 3.7 — Dashboard layout (wireframe of the implemented page)

<table>
<tr><td rowspan="8">Sidebar: Filters
Date Range (date_input)
Facility (selectbox)
Doctor Specialty
Visit Type
Diagnosis Category
each defaults to "All"</td><td colspan="2">Header: "Healthcare Intelligence Hub", caption "Patient, Cost & Care Analytics"</td></tr>
<tr><td colspan="2">KPI row (6 × st.metric): Total Visits · Unique Patients · Total Cost · Avg Satisfaction (/5) · Readmission Rate (%) · Avg Length of Stay (days)</td></tr>
<tr><td>Monthly Visit Trend: line + points (Altair)</td><td>Visit Type Split: donut</td></tr>
<tr><td>Top 10 Diagnoses by Volume: horizontal bar</td><td>Admission Status: donut</td></tr>
<tr><td>Facility Performance: bar coloured by facility type</td><td>Facility table: visits, avg cost, avg satisfaction</td></tr>
<tr><td>Treatment Status: bar</td><td>Readmission Rate by Diagnosis: top-10 horizontal bar</td></tr>
<tr><td colspan="2">Average Cost Breakdown by Visit Type: stacked bar (consultation, medication, lab, room)</td></tr>
<tr><td colspan="2">Doctor Performance Leaderboard: table · footer: filtered visit count and source table</td></tr>
</table>

Data access design: `load_fact()` reads `HEALTH_DB.CURATED.FACT_PATIENT_VISITS` once through `session.table(...).to_pandas()`, upper-cases column names, converts `VISIT_DATE` to datetime, and is cached for 600 seconds. All filtering and aggregation then happen in pandas. The MART views are **not** used by the app. They serve SQL users and the documentation of KPI logic.

3.9 Security, Access Control and Data Privacy

Implemented: - No credentials in code. The app uses `get_active_session()`, which runs with the owner's or caller's Snowflake role, according to the Streamlit-in-Snowflake model. - Data stays inside Snowflake. There are no external APIs or network calls.

Not implemented (gaps): - No `CREATE ROLE` or `GRANT` statements in `query.sql`; development presumably used a high-privilege role. - Patient first and last names and date of birth are carried into the fact table and app data frame. The data appears synthetic, but a real deployment would need protection.

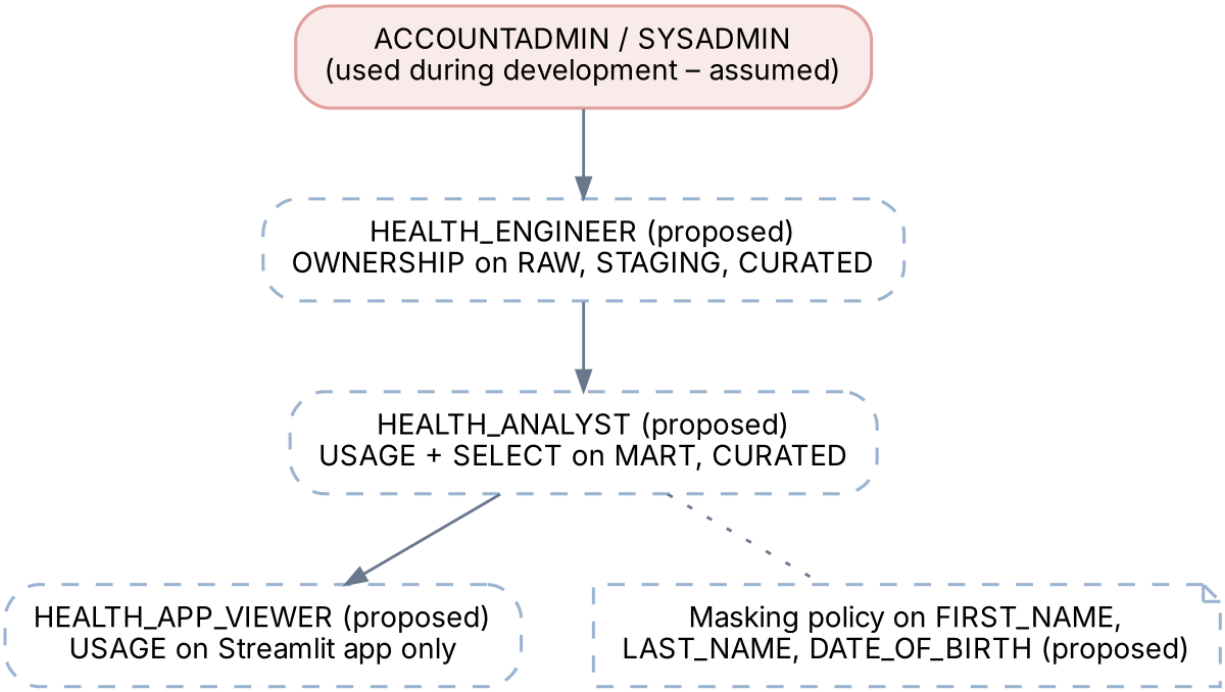


Figure 3.4 — Proposed role hierarchy and masking (dashed = proposed, not implemented)

Proposed controls: least-privilege roles (`HEALTH_ENGINEER` , `HEALTH_ANALYST` , `HEALTH_APP_VIEWER`); `SELECT` on `MART` only for analysts; dynamic data-masking policies on `FIRST_NAME` , `LAST_NAME` and `DATE_OF_BIRTH` ; dropping patient names from `FACT_PATIENT_VISITS` because no visual uses them; moving the app out of `RAW` . A real deployment must also comply with applicable health-data regulations (e.g., India's Digital Personal Data Protection Act, 2023, or HIPAA in the US).

Phase 4 — Project Planning Phase

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Template covered:** Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points), 8 marks

> **Dates and effort.** The project files contain no commit history, timestamps or sprint records. Calendar dates are therefore shown as **relative days (estimates)**. Story points are planning estimates. "Completion evidence" refers only to artefacts that exist in the repository. Fill the bracketed fields with your actual dates before submission.

4.1 Project Planning Template

Field	Value
Date	18 October 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	8 Marks

4.1.1 Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Table 4.1 — Product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Planning & Requirements	T-1	Study problem, profile six CSVs (row counts, keys, value domains)	3	High	[Member 1]
Sprint-1	Planning & Requirements	T-2	Ideation and requirement documents (Phases 1–2)	5	High	[Member 2]
Sprint-1	Data Access & Integration	USN-1	Create `HEALTH_DB` and the RAW, STAGING, CURATED and MART schemas	2	High	[Member 3]
Sprint-1	Data Access & Integration	USN-2	Define `CSV_FORMAT` and `HEALTHCARE_STAGE` ; create six typed RAW tables; load CSVs	5	High	[Member 3]
Sprint-2	Data Processing	USN-3	Build STAGING tables (TRIM, INITCAP, NOT NULL keys)	5	High	[Member 4]
Sprint-2	Data Processing	USN-4	Build five dimensions and `FACT_PATIENT_VISITS`	5	High	[Member 4]
Sprint-2	Business Analysis	T-3	Create ten MART KPI views	3	High	[Member 1]
Sprint-2	Data Processing	T-4	Validation queries: row counts, NULLs, duplicates, orphans	2	High	[Member 2]
Sprint-3	Business Analysis	USN-5	KPI cards: visits, patients, cost, satisfaction, readmission %, LOS	3	High	[Member 1]
Sprint-3	Business Insights	USN-6	Monthly trend and visit-type donut	2	High	[Member 2]
Sprint-3	Business Insights	USN-7	Facility performance chart and table	3	High	[Member 3]
Sprint-3	Business Insights	USN-8	Doctor leaderboard	2	Medium	[Member 4]
Sprint-3	Business Analysis	USN-9	Cost breakdown by visit type	2	High	[Member 1]
Sprint-3	Business Analysis	USN-10	Insurance and patient payment totals (KPI view)	1	Medium	[Member 2]
Sprint-3	Business Insights	USN-11	Readmission rate by diagnosis	2	High	[Member 3]

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-4	User Interaction	USN-12	Sidebar filters: date range, facility, specialty, visit type, diagnosis category	5	High	[Member 4]
Sprint-4	Data Update & Reporting	USN-13	Package and deploy app (`snowflake.yml` , `pyproject.toml`)	3	Medium	[Member 1]
Sprint-4	Testing	T-5	Data-quality, transformation, KPI and dashboard tests (Phase 6)	4	High	[Member 2]
Sprint-4	Documentation	T-6	Final report, README and demonstration preparation (Phases 7–8)	3	High	[All]
Backlog	Security	USN-14	Roles, grants and masking policies	5	Low	—

4.1.2 Project Tracker, Velocity & Burndown Chart (4 Marks)

Table 4.2 — Project tracker (relative schedule; actual dates to be filled in)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)	Completion evidence in repository
Sprint-1	15	6 Days	Day 1 [date]	Day 6 [date]	[actual]	[date]	`CREATE DATABASE/SCHEMA` , `CSV_FORMAT` , `HEALTHCARE_STAGE` , six `*_RAW` tables in `query.sql` ; CSVs present. *Load command not scripted.*
Sprint-2	15	6 Days	Day 7 [date]	Day 12 [date]	[actual]	[date]	STAGING, CURATED and MART sections and validation queries in `query.sql`
Sprint-3	15	6 Days	Day 13 [date]	Day 18 [date]	[actual]	[date]	KPI, chart and table code in `streamlit_app.py`
Sprint-4	15	6 Days	Day 19 [date]	Day 24 [date]	[actual]	[date]	Filters in `streamlit_app.py` ; `snowflake.yml` ; `tests/local_validation.py` ; this documentation

Velocity:

With a 6-day sprint and a planned velocity of 15 story points per sprint, the team's average velocity (AV) per iteration unit is:

> AV = Sprint velocity ÷ Sprint duration = 15 ÷ 6 = **2.5 story points per day**

Across 4 sprints the plan covers 60 points. If the actual completed points differ, recompute AV as (sum of completed points ÷ number of sprints) ÷ 6.

Burndown Chart: A burndown chart is a graphical representation of work left to do versus time.

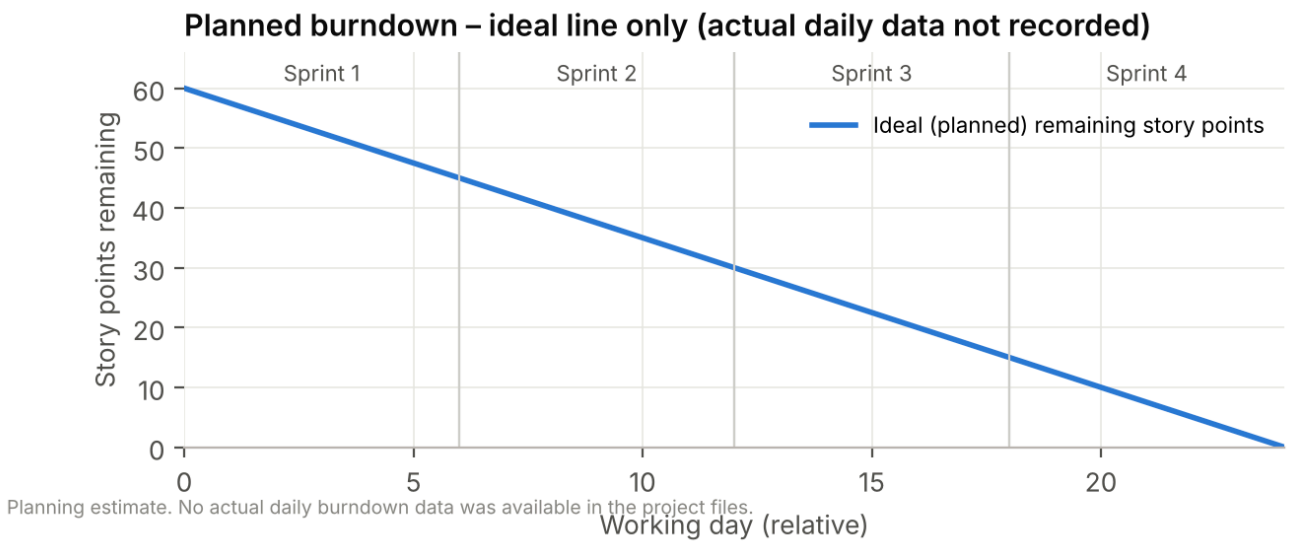


Figure 4.1 — Planned (ideal) burndown for 60 story points over 24 working days. No actual daily progress data was recorded, so only the ideal line is shown. Plot your actual remaining points against it.

References (from template): Atlassian Agile guides on project management, epics, sprints, estimation and burndown charts; Visual Paradigm: Scrum burndown chart.

4.2 Work Breakdown Structure

```
1  Healthcare Intelligence Hub
├── 1.1 Initiation & Ideation ........ problem statements, empathy map, brainstorming
├── 1.2 Requirement Analysis ......... FR/NFR, DFD, user stories, tech stack, journey map
├── 1.3 Design ....................... architecture, layers, star schema, app wireframe, security
├── 1.4 Data Engineering
│   ├── 1.4.1 Database & schemas
│   ├── 1.4.2 File format, stage, RAW tables, data load
│   ├── 1.4.3 STAGING cleaning
│   ├── 1.4.4 CURATED dimensions & fact
│   └── 1.4.5 MART KPI views + validation queries
├── 1.5 Application .................. KPI cards, 8 charts, 2 tables, 5 filters, deployment
├── 1.6 Testing ...................... DQ, transformation, KPI, dashboard, performance
└── 1.7 Documentation & Demo ......... report, README, demo script, presentation
```

4.3 Phases, Milestones and Deliverables

Table 4.3 — Milestones (relative timeline, estimated)

#	Milestone	Planned (relative)	Deliverable	Dependency	Status (evidence)
M1	Ideation complete	Day 2	Phase 1 document	—	Documented
M2	Requirements baselined	Day 4	Phase 2 document	M1	Documented
M3	Design approved	Day 6	Phase 3 document, diagrams	M2	Documented
M4	Data loaded to RAW	Day 6	RAW tables populated	M3	DDL present; load not scripted
M5	Curated model built	Day 12	STAGING, CURATED, MART	M4	Present in `query.sql`
M6	Dashboard feature-complete	Day 22	`streamlit_app.py`	M5	Present
M7	Testing complete	Day 23	Phase 6 report, test scripts	M5, M6	Local tests executed; Snowflake tests Not Executed
M8	Documentation and demo	Day 24	Phases 7–8, README	M7	This set

Critical path: M2 → M3 → M4 → M5 → M6 → M7 → M8. Every analytic output depends on the curated fact table.

4.4 Resource and Tool Requirements

Resource	Purpose
Snowflake account, warehouse `COMPUTE_WH`	Storage, compute, Streamlit hosting
Snowsight	Worksheets, file upload, app editor
Snowflake CLI (optional)	`snow streamlit deploy` using `snowflake.yml`
Python 3.11 + pandas (local, optional)	`tests/local_validation.py`
Git / GitHub	Version control and publication
Graphviz, Markdown	Diagrams and documentation

4.5 Roles and Responsibilities

Team composition was not supplied. Assign names to the roles below.

Role	Responsibilities	Assigned to
Team lead / coordinator	Planning, tracking, integration, demo lead	[Name]
Data engineer	Snowflake objects, loading, STAGING and CURATED SQL	[Name]
Analytics engineer	MART views, KPI definitions, validation queries	[Name]
App developer	Streamlit dashboard, filters, deployment	[Name]
QA and documentation	Test cases, report, README	[Name]

4.6 Risk Assessment and Mitigation

Table 4.4 — Risk register

ID	Risk	Likelihood	Impact	Mitigation	Status
R1	Snowflake trial credits expire	Medium	High	Use X-Small warehouse with auto-suspend; keep `query.sql` re-runnable	Open
R2	Manual data load is repeated, causing duplicate RAW rows	Medium	Medium	`CREATE OR REPLACE` RAW tables; duplicate-key checks in STAGING	Mitigated by design
R3	Inconsistent category labels	Low	Medium	TRIM/INITCAP in STAGING	Mitigated, but INITCAP side effects found (Phase 6)
R4	App slow on large data (full table into pandas)	Low now / High at scale	Medium	Cache (`ttl=600`); future SQL push-down	Accepted for current volume
R5	PII exposure	Medium	High	Proposed roles and masking	Open
R6	Team availability	Medium	Medium	Clear role ownership; documented scripts	[to update]

4.7 Testing and Documentation Plan

Activity	When	Owner	Output
Local replica tests (`tests/local_validation.py`)	After each SQL change	QA	Console report / JSON
Snowflake validation (`tests/snowflake_validation.sql`)	After full rebuild in Snowflake	Data engineer	Results recorded in `TEST_CASES.md`
Dashboard functional tests (manual checklist)	Before demo	App developer	Screenshots, pass/fail

Activity	When	Owner	Output
Documentation	Throughout; final in Sprint-4	All	`docs/` , README, PDFs

4.8 Progress Tracking and Completion Criteria

Track progress in the tracker table (4.1.2) and the burndown chart. The project counts as complete when:

1. Every High-priority story in Table 4.1 has its acceptance criterion met (Phase 2, Table 2.4).
2. `tests/snowflake_validation.sql` has run in Snowflake with results recorded.
3. The app opens in Snowsight and all five filters work.
4. All eight phase documents and the README are committed to the repository.

Phase 5 — Project Development Phase

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Business Question and Visualisation Report (5 marks) · Dashboard Design (5 marks) · Story Design (5 marks), from the uploaded "Tableau" template folder

> **Template adaptation.** The development templates are written for Tableau. The implemented project uses **Streamlit in Snowflake** with Altair charts, and no Tableau workbook exists in the archive. The template headings and requirements (at least 8 business questions with visualisations, dashboard outcomes, story observations, published link) are kept, and each answer points to the Streamlit or SQL component that delivers it. Where the template asks for a Tableau Public link, the field is marked *not applicable* with the Streamlit equivalent.
>
> **Figures.** Charts in this phase were regenerated from the project CSVs with the same aggregation logic as the app and views (see `tests/local_validation.py`). They are **not screenshots of the deployed Snowflake app**. Section 5.14 lists the screenshots to capture from your live app to replace or accompany them.

5.1 Development Environment and Setup

5.1.1 Repository contents (as uploaded)

```
<repo root>/                      (uploaded folder name: Nautanki)
├── query.sql                     Snowflake DDL/DML for all four layers + checks (440 lines)
├── patients.csv  doctors.csv  facilities.csv  diagnoses.csv  date.csv  patient_visits.csv
└── Health/
    ├── streamlit_app.py          Streamlit dashboard (230 lines)
    ├── snowflake.yml             Snowflake CLI project definition (v2)
    ├── pyproject.toml            Python ~=3.11, dependency streamlit[snowflake]
    └── .streamlit/config.toml    Theme placeholder (comments only)
```

No notebooks, dbt project, orchestration code, tests or README were present. The `__MACOSX/` folder in the ZIP holds macOS metadata only and is not part of the project.

5.1.2 Setup steps (Snowsight)

1. **Create objects.** Open a SQL worksheet, then run `query.sql` from the top down to and including `CREATE OR REPLACE STAGE …` (database, four schemas, six RAW tables, file format, stage).
2. **Upload files.** Upload the six CSVs to `HEALTH_DB.RAW.HEALTHCARE_STAGE`, either in Snowsight (*Data → Databases → HEALTH_DB → RAW → Stages → HEALTHCARE_STAGE → + Files*) or with SnowSQL/CLI `PUT`.
3. **Load RAW.** `query.sql` contains no load statement. Use the Snowsight *Load Data* wizard, or run the commands below. They are a **recommended addition**, not part of the original script.

```
-- Recommended load commands (not in the original query.sql)
COPY INTO HEALTH_DB.RAW.PATIENTS_RAW       FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*patients[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.DOCTORS_RAW        FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*doctors[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.FACILITIES_RAW     FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*facilities[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.DIAGNOSES_RAW      FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*diagnoses[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.DATE_RAW           FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*date[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
COPY INTO HEALTH_DB.RAW.PATIENT_VISITS_RAW FROM @HEALTH_DB.RAW.HEALTHCARE_STAGE PATTERN = '.*patient_visits[.]csv.*'
ON_ERROR = 'ABORT_STATEMENT';
```

The stage was created with `FILE_FORMAT = HEALTH_DB.RAW.CSV_FORMAT`, so `COPY INTO` inherits it.

4. **Transform.** Run the rest of `query.sql` (RAW count check → STAGING → CURATED → MART).
5. **Deploy the app.** Either create a Streamlit app in Snowsight (*Projects → Streamlit → + Streamlit App*, database `HEALTH_DB`, warehouse `COMPUTE_WH`) and paste `streamlit_app.py`, or from `Health/` run `snow streamlit deploy --replace` with a configured Snowflake CLI connection. The `snowflake.yml` identifier `ICYNJI7QNBXR2IUI` is an auto-generated name, which suggests the app was first created in Snowsight. Live deployment could not be verified from the files.

6. **Validate.** Run `tests/snowflake_validation.sql` (added for this documentation). Locally, `python tests/local_validation.py` validates the CSVs and the SQL logic without Snowflake.

5.2 Verified Technology Stack and Dependencies

Table 5.1 — Dependencies found in the code

Item	Version / setting	Source
Python	`requires-python = "~=3.11.0"`	`pyproject.toml`
streamlit (with `snowflake` extra)	unpinned; Snowflake supplies a pre-installed version if no repository is configured	`pyproject.toml` comments
pandas, altair	imported; installed with Streamlit	`streamlit_app.py`
snowflake-snowpark-python	`from snowflake.snowpark.context import get_active_session`	`streamlit_app.py`
uv constraint	`numba>=0.56.0` ; `exclude-newer = "7 days"`	`pyproject.toml`
Snowflake objects	warehouse `COMPUTE_WH` ; compute pool `SYSTEM_COMPUTE_POOL_CPU` (container runtime)	`snowflake.yml`

5.3 Source Data Collection and Ingestion

The six CSV files are UTF-8 with header rows, ISO `YYYY-MM-DD` dates, `True/False` booleans and `Yes/No` flags. Profiling found no blank values and no leading or trailing spaces in any column. The format below governs parsing:

```
CREATE OR REPLACE FILE FORMAT HEALTH_DB.RAW.CSV_FORMAT
    TYPE                         = CSV
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER                  = 1
    NULL_IF                      = ('NULL', 'null', '')
    EMPTY_FIELD_AS_NULL          = TRUE
    DATE_FORMAT                  = 'YYYY-MM-DD';

CREATE OR REPLACE STAGE HEALTH_DB.RAW.HEALTHCARE_STAGE
    FILE_FORMAT = HEALTH_DB.RAW.CSV_FORMAT;
```

5.4 Raw-Layer Implementation

Six typed tables, one per source, are created with `CREATE OR REPLACE TABLE` . Re-running the script empties them, which prevents duplicates from repeated loads. Types: identifiers and text as `VARCHAR` ; dates as `DATE` ; costs as `NUMBER(12,2)` ; satisfaction as `NUMBER(5,2)` ; `IS_WEEKEND` as `BOOLEAN` ; the readmission flag as `VARCHAR` . Example:

```
CREATE OR REPLACE TABLE PATIENT_VISITS_RAW (
    VISIT_ID VARCHAR, PATIENT_ID VARCHAR, DOCTOR_ID VARCHAR, FACILITY_ID VARCHAR,
    DIAGNOSIS_ID VARCHAR, VISIT_DATE DATE, VISIT_TYPE VARCHAR, ADMISSION_STATUS VARCHAR,
    TREATMENT_STATUS VARCHAR, LENGTH_OF_STAY_DAYS NUMBER,
    CONSULTATION_COST NUMBER(12,2), MEDICATION_COST NUMBER(12,2), LAB_COST NUMBER(12,2),
    ROOM_COST NUMBER(12,2), TOTAL_COST NUMBER(12,2), INSURANCE_COVERAGE NUMBER(12,2),
    PATIENT_PAYMENT NUMBER(12,2), PATIENT_SATISFACTION NUMBER(5,2),
    READMISSION_WITHIN_30_DAYS VARCHAR );   -- (formatted for print; same columns as query.sql)
```

A row-count query over all six RAW tables follows the DDL so the load can be confirmed.

5.5 Data Cleaning and Standardisation (STAGING)

Each staging table is a CTAS from its RAW table. Text is trimmed, categorical columns are case-standardised and rows without a key are removed:

```
CREATE OR REPLACE TABLE PATIENT_VISITS AS
SELECT
    TRIM(VISIT_ID)                              AS VISIT_ID,
```

```
    TRIM(PATIENT_ID)                          AS PATIENT_ID,
    ...
    INITCAP(TRIM(VISIT_TYPE))                 AS VISIT_TYPE,
    INITCAP(TRIM(ADMISSION_STATUS))           AS ADMISSION_STATUS,
    INITCAP(TRIM(TREATMENT_STATUS))           AS TREATMENT_STATUS,
    LENGTH_OF_STAY_DAYS, CONSULTATION_COST, ... , PATIENT_SATISFACTION,
    INITCAP(TRIM(READMISSION_WITHIN_30_DAYS)) AS READMISSION_WITHIN_30_DAYS
FROM HEALTH_DB.RAW.PATIENT_VISITS_RAW
WHERE VISIT_ID IS NOT NULL;
```

Three queries then report NULL keys, duplicate keys and row counts for every staging table.

Effect on this dataset (verified locally). The source is already clean, so TRIM and the NULL filter change nothing (all 14,059 rows are kept). `INITCAP` changes two values: `ENT` becomes `Ent` and `Follow-up` becomes `Follow-Up`, because Snowflake lower-cases letters after the first and treats `-` as a word separator. These become the labels shown in the app's specialty filter and visit-type charts.

5.6 Data Transformation and Modelling (CURATED)

Dimensions are `SELECT DISTINCT` copies of the staging tables. The fact table joins visit rows to descriptive attributes:

```
CREATE OR REPLACE TABLE FACT_PATIENT_VISITS AS
SELECT v.VISIT_ID, v.PATIENT_ID, v.DOCTOR_ID, v.FACILITY_ID, v.DIAGNOSIS_ID, v.VISIT_DATE,
       v.VISIT_TYPE, v.ADMISSION_STATUS, v.TREATMENT_STATUS, v.LENGTH_OF_STAY_DAYS,
       v.CONSULTATION_COST, v.MEDICATION_COST, v.LAB_COST, v.ROOM_COST, v.TOTAL_COST,
       v.INSURANCE_COVERAGE, v.PATIENT_PAYMENT, v.PATIENT_SATISFACTION, v.READMISSION_WITHIN_30_DAYS,
       p.FIRST_NAME, p.LAST_NAME, p.GENDER, p.CITY AS PATIENT_CITY, p.INSURANCE_PROVIDER,
       d.DOCTOR_NAME, d.SPECIALTY, d.DEPARTMENT,
       f.FACILITY_NAME, f.CITY AS FACILITY_CITY, f.FACILITY_TYPE,
       dg.DIAGNOSIS_NAME, dg.DIAGNOSIS_CATEGORY
FROM HEALTH_DB.STAGING.PATIENT_VISITS v
LEFT JOIN HEALTH_DB.STAGING.PATIENTS   p  ON v.PATIENT_ID   = p.PATIENT_ID
LEFT JOIN HEALTH_DB.STAGING.DOCTORS    d  ON v.DOCTOR_ID    = d.DOCTOR_ID
LEFT JOIN HEALTH_DB.STAGING.FACILITIES f  ON v.FACILITY_ID  = f.FACILITY_ID
LEFT JOIN HEALTH_DB.STAGING.DIAGNOSES  dg ON v.DIAGNOSIS_ID = dg.DIAGNOSIS_ID;
```

`LEFT JOIN` keeps visits whose reference data is missing. The orphan check that follows counts rows where the joined name is NULL; it returns 0 for this dataset (verified locally). Each key is unique in staging, so the joins do not multiply rows (FACT = 10,000).

5.7 Snowflake SQL Object Inventory

Table 5.2 — Objects created by query.sql (36 objects incl. the Streamlit app)

Schema	Object type	Count	Names
—	Database	1	`HEALTH_DB`
—	Schemas	4	`RAW`, `STAGING`, `CURATED`, `MART`
RAW	Tables	6	`PATIENTS_RAW`, `DOCTORS_RAW`, `FACILITIES_RAW`, `DIAGNOSES_RAW`, `DATE_RAW`, `PATIENT_VISITS_RAW`
RAW	File format, stage	2	`CSV_FORMAT`, `HEALTHCARE_STAGE`
STAGING	Tables	6	`PATIENTS`, `DOCTORS`, `FACILITIES`, `DIAGNOSES`, `DATE`, `PATIENT_VISITS`
CURATED	Tables	6	`DIM_PATIENT`, `DIM_DOCTOR`, `DIM_FACILITY`, `DIM_DIAGNOSIS`, `DIM_DATE`, `FACT_PATIENT_VISITS`
MART	Views	10	see Table 5.3
RAW	Streamlit	1	`ICYNJI7QNBXR2IUI` (from `snowflake.yml`)

`query.sql` also contains 19 diagnostic `SELECT` / `SHOW` statements: 2 `SHOW` commands, row counts at three layers, NULL, duplicate and orphan checks, a fact preview, and previews of all ten views.

5.8 Analytical Queries and KPIs (MART)

Table 5.3 — MART views and their measures

View	Grain	Measures
`VW_EXECUTIVE_KPI`	1 row	visits, distinct patients, doctors, facilities; total cost, insurance coverage, patient payment; avg satisfaction; avg LOS; readmissions
`VW_MONTHLY_VISITS`	month	visits, total cost, avg satisfaction
`VW_DIAGNOSIS_ANALYTICS`	diagnosis	visits, avg cost, avg LOS, avg satisfaction
`VW_FACILITY_ANALYTICS`	facility	visits, total and avg cost, avg satisfaction, avg LOS
`VW_VISIT_TYPE_DISTRIBUTION`	visit type	visits, avg cost, avg satisfaction, avg LOS (sorted by visits)
`VW_TREATMENT_ANALYTICS`	treatment status	visits, avg cost, avg satisfaction
`VW_ADMISSION_STATUS_ANALYTICS`	admission status	visits, avg cost
`VW_PATIENT_VISIT_TYPE_ANALYSIS`	visit type	same as `VW_VISIT_TYPE_DISTRIBUTION`, sorted by avg cost
`VW_DOCTOR_ANALYTICS`	doctor name + specialty + department	visits, avg cost, avg satisfaction, readmissions
`VW_READMISSION_ANALYTICS`	diagnosis	visits, readmissions, readmission rate %

KPI definitions:

KPI	Formula	Value (full dataset, local replica)
Total visits	`COUNT(*)`	10,000
Unique patients	`COUNT(DISTINCT PATIENT_ID)`	2,889
Total cost	`SUM(TOTAL_COST)`	82,818,797.00
Insurance coverage / patient payment	`SUM(...)`	59,918,913.84 / 22,899,883.16 (72.3% / 27.7%)
Average satisfaction	`AVG(PATIENT_SATISFACTION)`	3.83 / 5
Average LOS	`AVG(LENGTH_OF_STAY_DAYS)`	0.97 days (inpatient only: 7.69)
Readmissions / rate	`SUM(CASE WHEN flag='Yes' THEN 1 END)` / ÷ visits × 100	1,203 / 12.03%

Example view (readmission rate):

```
CREATE OR REPLACE VIEW VW_READMISSION_ANALYTICS AS
SELECT DIAGNOSIS_CATEGORY, DIAGNOSIS_NAME, COUNT(*) AS TOTAL_VISITS,
       SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) AS READMISSIONS,
       ROUND(100.0 * SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS READMISSION_RATE_PCT
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2
ORDER BY READMISSION_RATE_PCT DESC;
```

5.9 Streamlit Application Implementation

Data loading with caching. The app reads the fact table once and caches it for 10 minutes:

```
@st.cache_data(ttl=600)
def load_fact():
    session = get_active_session()
    df = session.table("HEALTH_DB.CURATED.FACT_PATIENT_VISITS").to_pandas()
    df.columns = df.columns.str.upper()
    df["VISIT_DATE"] = pd.to_datetime(df["VISIT_DATE"])
    return df
```

Filters. The date range is bounded by the minimum and maximum visit date. The four drop-downs are filled from the distinct values in the data, each with "All" first. Filters are applied one after another to a copy of the data frame:

```
df = df_raw.copy()
if len(date_range) == 2:
    df = df[(df["VISIT_DATE"].dt.date >= date_range[0]) & (df["VISIT_DATE"].dt.date <= date_range[1])]
if sel_facility  != "All": df = df[df["FACILITY_NAME"]      == sel_facility]
if sel_specialty != "All": df = df[df["SPECIALTY"]          == sel_specialty]
if sel_visit     != "All": df = df[df["VISIT_TYPE"]         == sel_visit]
if sel_diag_cat  != "All": df = df[df["DIAGNOSIS_CATEGORY"] == sel_diag_cat]
```

The `len(date_range) == 2` guard covers the moment when the user has picked only the first date of the range.

KPIs with empty-result guards:

```
readmit_rate = (100 * (df["READMISSION_WITHIN_30_DAYS"] == "Yes").sum() / total_visits
                if total_visits > 0 else 0)
```

Visuals: an Altair line (monthly trend, `to_period("M")`), two donuts (`mark_arc(innerRadius=50)`) for visit type and admission status, bars for top-10 diagnoses, facilities (coloured by type), treatment status and readmission rate (top 10), a stacked bar of mean cost components (`melt` to long format), and two `st.dataframe` tables (facility, doctor leaderboard). Charts include tooltips. The footer prints the filtered visit count.

5.10 Integration Between Components

```
CSV ──PUT/upload──▶ RAW.HEALTHCARE_STAGE ──load──▶ RAW.*_RAW ──CTAS──▶ STAGING.* ──CTAS──▶ CURATED.DIM_* / FACT
                                                                                  │            │
                                                                  CREATE VIEW ◀───┘            └──▶ Streamlit
(Snowpark session.table)
                                                                  MART.VW_* ──▶ Snowsight SQL users
```

The app depends only on `CURATED.FACT_PATIENT_VISITS` and its column names. Renaming a fact column breaks the app. MART views also depend on the fact table, but the app does not use them.

5.11 Error Handling, Logging and Validation

Aspect	Implemented	Gap / recommendation
Load errors	Depends on the load method (not scripted)	Use `COPY INTO … ON_ERROR='ABORT_STATEMENT'`; inspect `VALIDATE()` / `COPY_HISTORY`
Data validation	SQL checks for row counts, NULL keys, duplicate keys and orphans (manual review of results)	Make checks fail loudly, e.g., a stored procedure or test that raises when a count is > 0; add the business-rule checks from Phase 6
App robustness	Empty-filter guards for averages and rates; partial date-range guard; `dropna()` on filter options	No `try/except` around the Snowflake read; add `st.error` with a friendly message if the table is missing
Logging	None in code; Snowflake `QUERY_HISTORY` records all executed SQL	Optional: Snowflake event table for app logs

5.12 Business Questions and Visualisation Report

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

Visualisation development turns complex datasets into visual formats that are easy to interpret. The business questions below guided the selection of metrics and charts. The template requires a minimum of 8; ten are provided, including the three questions in the official problem statement. Each lists the implemented Streamlit visual and SQL view.

1. What is the monthly trend in patient visits? - Visualisation: line chart "Monthly Visit Trend" (Streamlit, Row 1); SQL: `VW_MONTHLY_VISITS`. - Finding: 32 months, 257–351 visits per month (lowest February 2026). Visits were 3,728 in 2024, 3,812 in 2025 and 2,460 in January–August 2026.

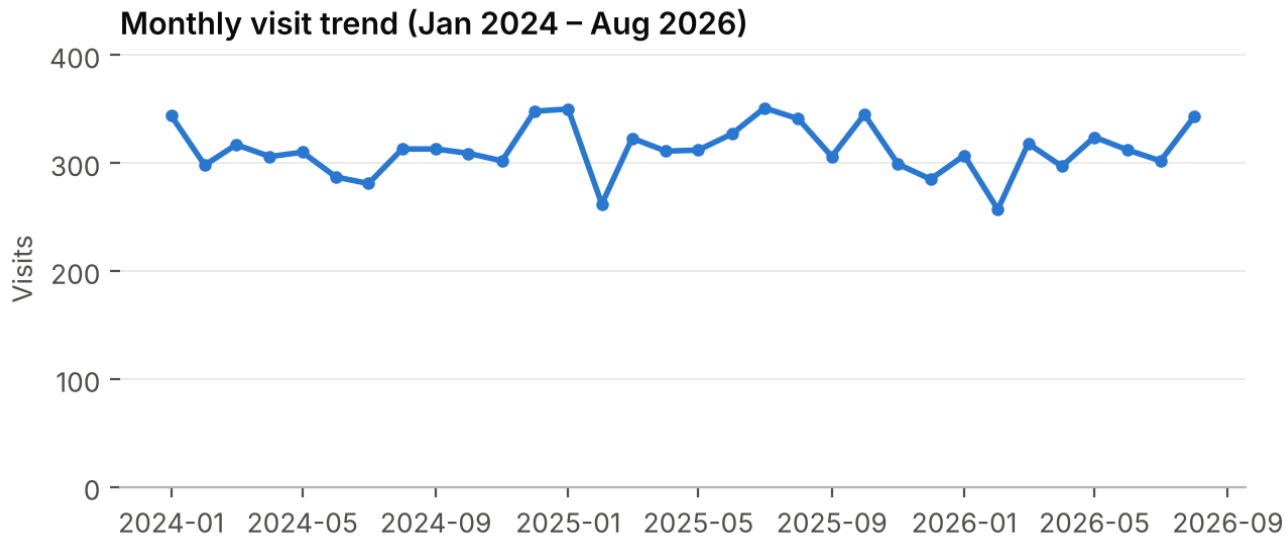


Figure 5.1 — Monthly visit trend (local reproduction from CSVs)

2. How are visits distributed across visit types? - Visualisation: donut "Visit Type Split"; SQL: `VW_VISIT_TYPE_DISTRIBUTION`. - Finding: Outpatient 39.7%, Follow-up 18.1%, Telemedicine 14.9%, Emergency 14.7%, Inpatient 12.6%.

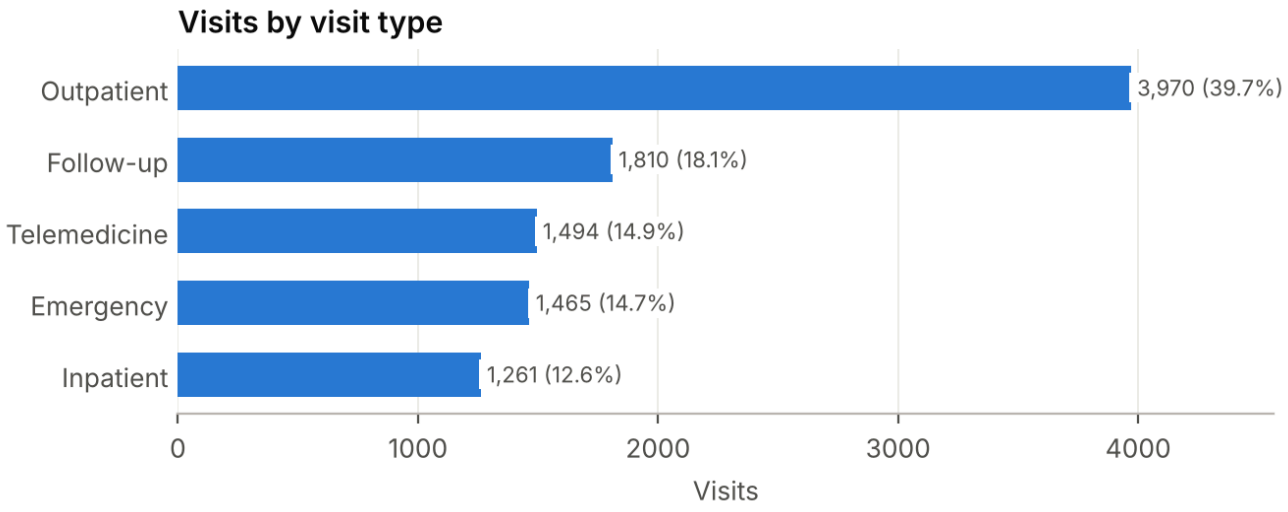


Figure 5.2 — Visits by visit type (bar form used for print; the app uses a donut)

3. Which diagnoses have the highest visit volume? - Visualisation: bar "Top 10 Diagnoses by Volume"; SQL: `VW_DIAGNOSIS_ANALYTICS`. - Finding: Pneumonia (714), Arthritis (700) and Gastritis (690) lead. Volumes are fairly even (637–714 across all 15).

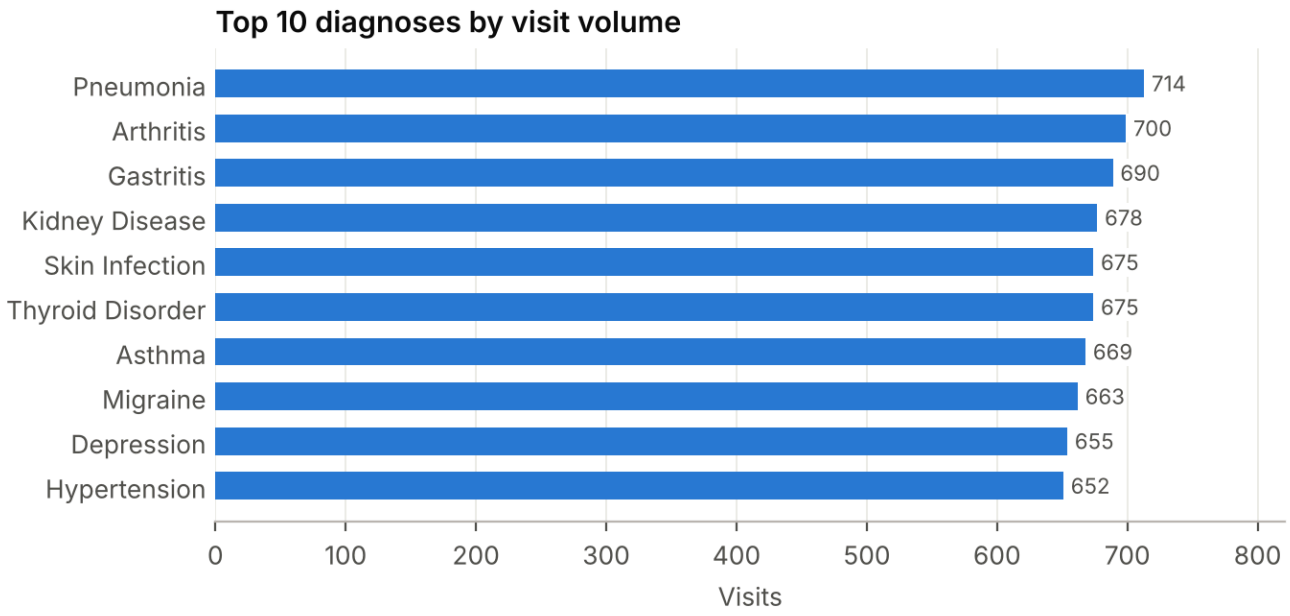


Figure 5.3 — Top 10 diagnoses by visit volume

4. Which diagnoses have the highest 30-day readmission rate? - Visualisation: bar "Readmission Rate by Diagnosis"; SQL: `VW_READMISSION_ANALYTICS` . - Finding: Depression 14.05%, Back Pain 13.98%, Asthma 13.75%, Arthritis 13.43%; Hypertension lowest at 10.43%; overall 12.03%.

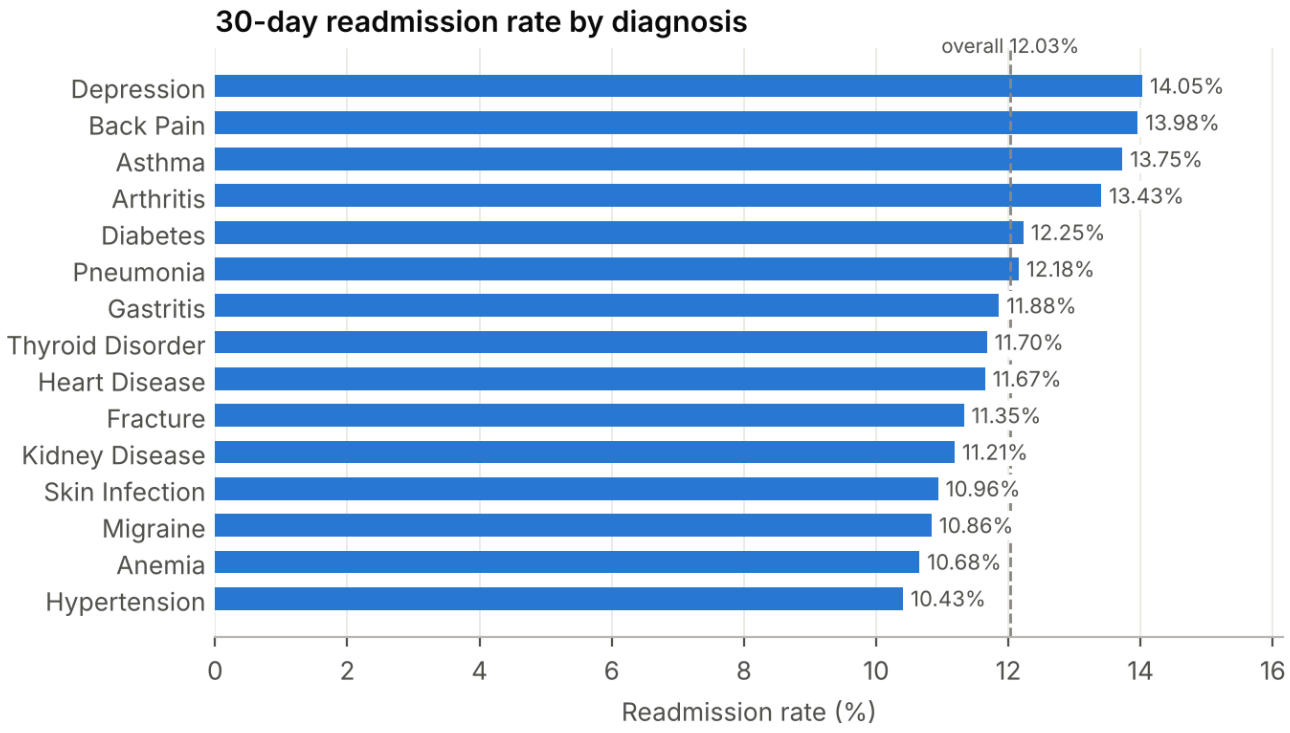


Figure 5.4 — 30-day readmission rate by diagnosis (all 15 shown; the app shows the top 10)

5. How do facilities compare on visit volume and cost per visit? - Visualisation: "Facility Performance" bar plus table; SQL: `VW_FACILITY_ANALYTICS` . - Finding: City General Hospital, Kolkata, has the most visits (1,039). Hope Hospital, Jaipur, has the highest average cost (8,432.07). Apollo View Clinic, Delhi, has the lowest (8,084.86).

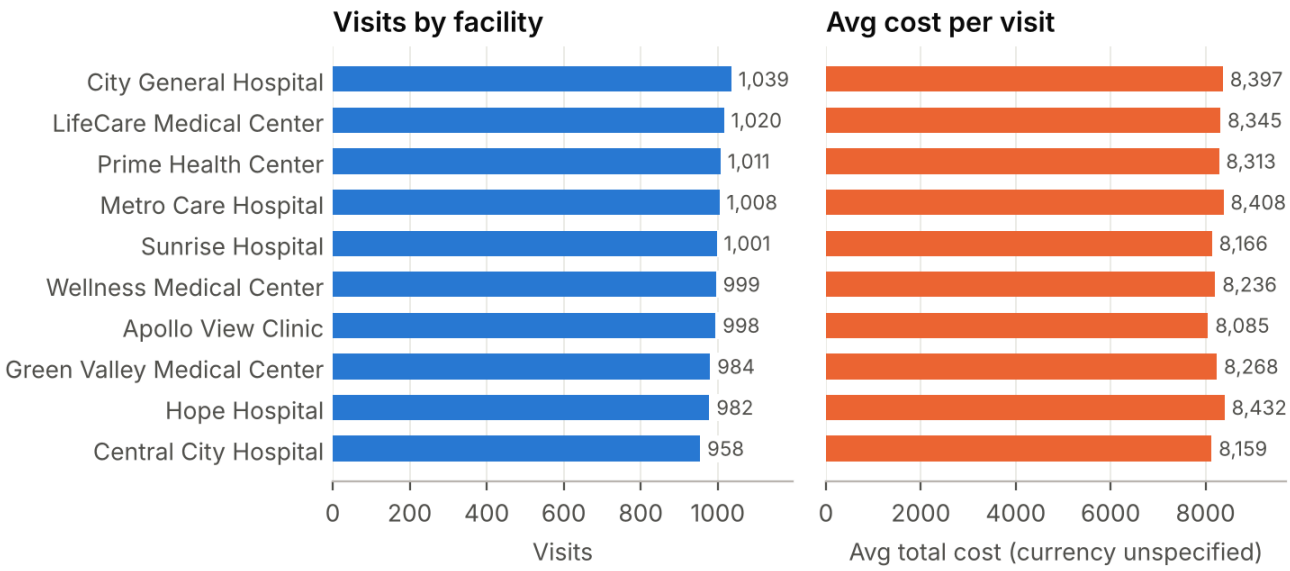


Figure 5.5 — Visits and average cost per visit by facility

6. What drives the cost of a visit for each visit type? - Visualisation: stacked bar "Average Cost Breakdown by Visit Type". - Finding: lab charges are the largest share overall (42.3% of total cost), followed by medication (30.9%). Room cost occurs only for inpatient visits (average 6,518), which nearly doubles the inpatient average to 13,973, against about 7,400–7,550 for other types.

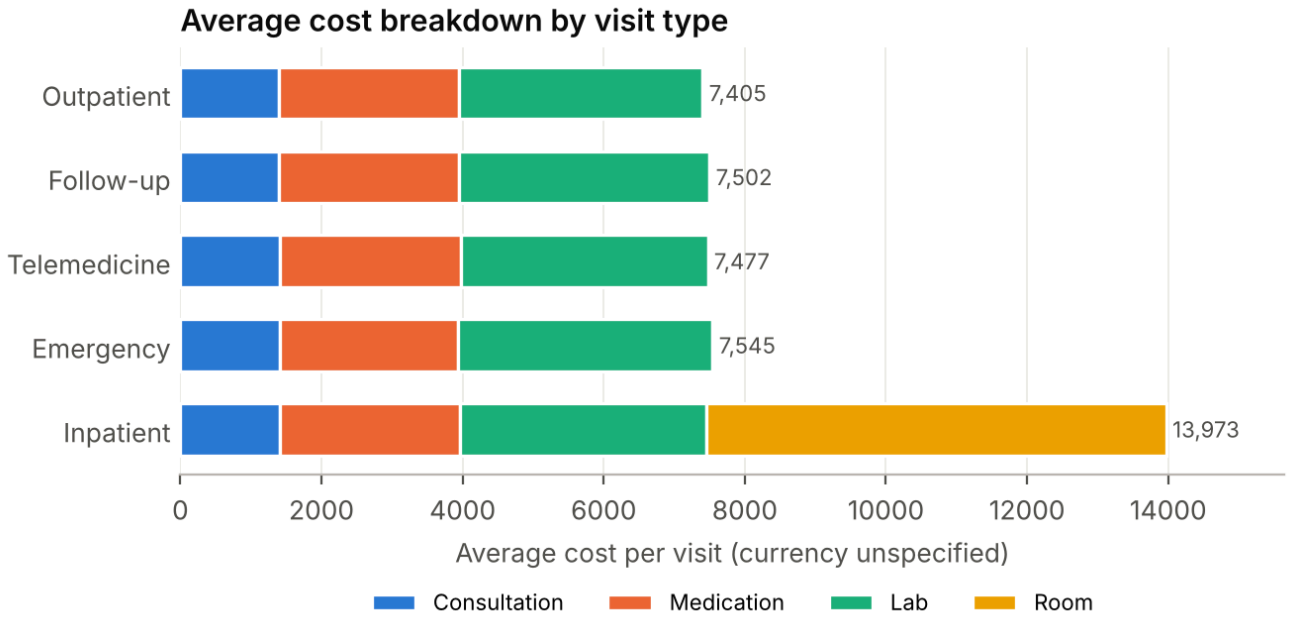


Figure 5.6 — Average cost breakdown by visit type

7. What are the treatment-outcome and admission mixes? - Visualisation: "Treatment Status" bar and "Admission Status" donut; SQL: `VW_TREATMENT_ANALYTICS`, `VW_ADMISSION_STATUS_ANALYTICS`. - Finding: 69.9% of visits are Completed, 18.8% Ongoing, 7.7% Referred and 3.6% Cancelled. 19.1% of visits are Admitted, with an average cost of 11,700 against 7,473 for Discharged.

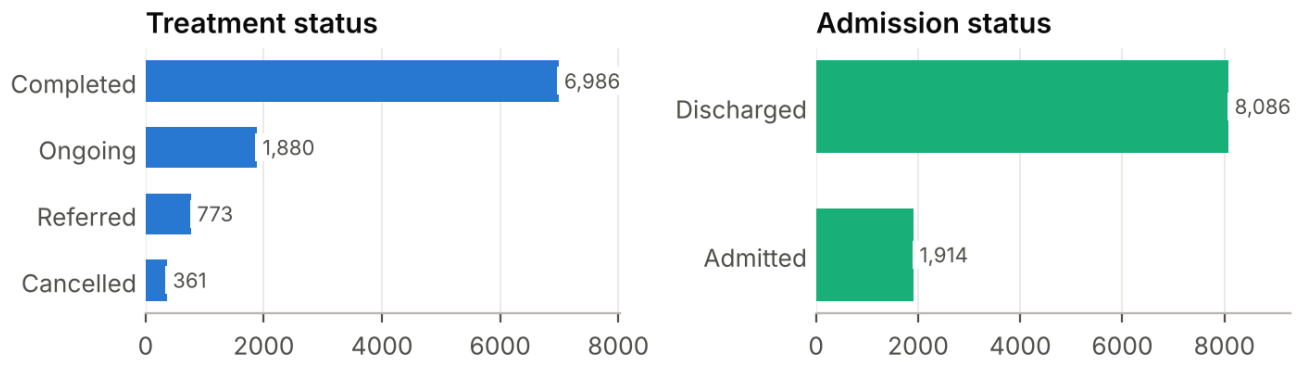


Figure 5.7 — Treatment and admission status

8. How much of the cost is covered by insurance, and does it differ by insurer? - Visualisation: SQL `VW_EXECUTIVE_KPI` (totals). There is no per-insurer chart in the app; the figure below is an additional analysis. - Finding: insurance covers 72.3% of total cost overall, and 72.0–72.7% for every provider, **including "Self Pay"**. This is a data-quality anomaly (Phase 6, TC-DQ-BR-01).

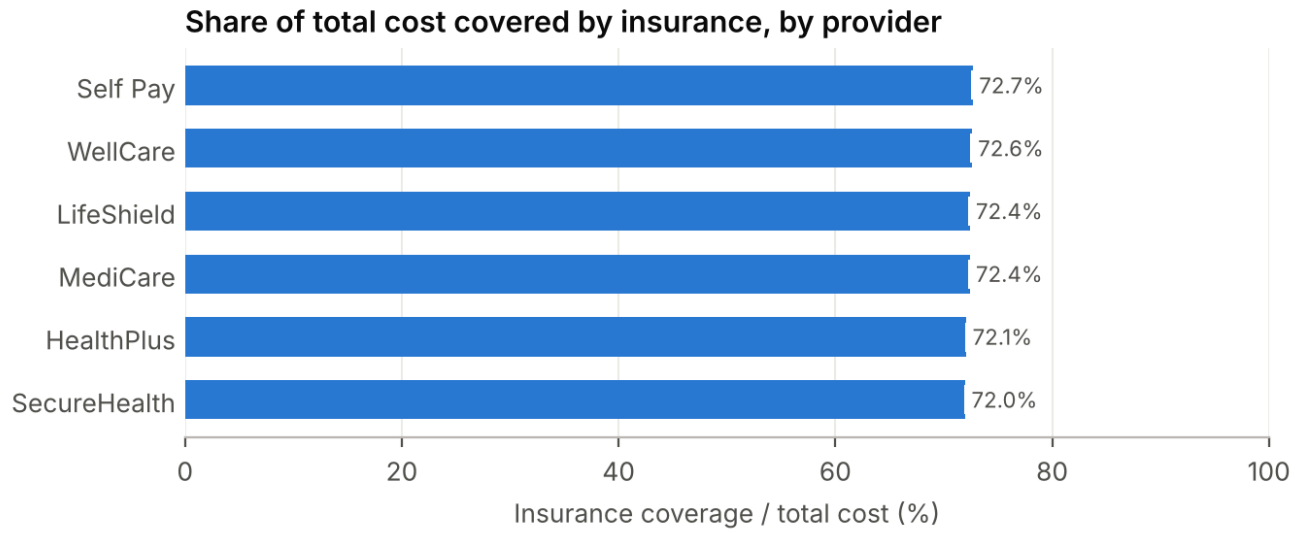


Figure 5.8 — Insurance coverage share by provider (additional analysis, not in app)

9. How satisfied are patients? - Visualisation: KPI card "Avg Satisfaction"; satisfaction columns in the facility and doctor tables. - Finding: average 3.83 / 5. 70.1% of visits score 4 or 5; 12.2% score 1 or 2. **Satisfaction at risk:** Sunrise Hospital has the lowest facility average (3.78) and the highest share of 1–2 scores (13.9%). Asthma is the lowest diagnosis (3.75). Dr. Rohan Singh (D060, 3.65) and Dr. Rohan Gupta (D004, 3.69) are the lowest doctors.

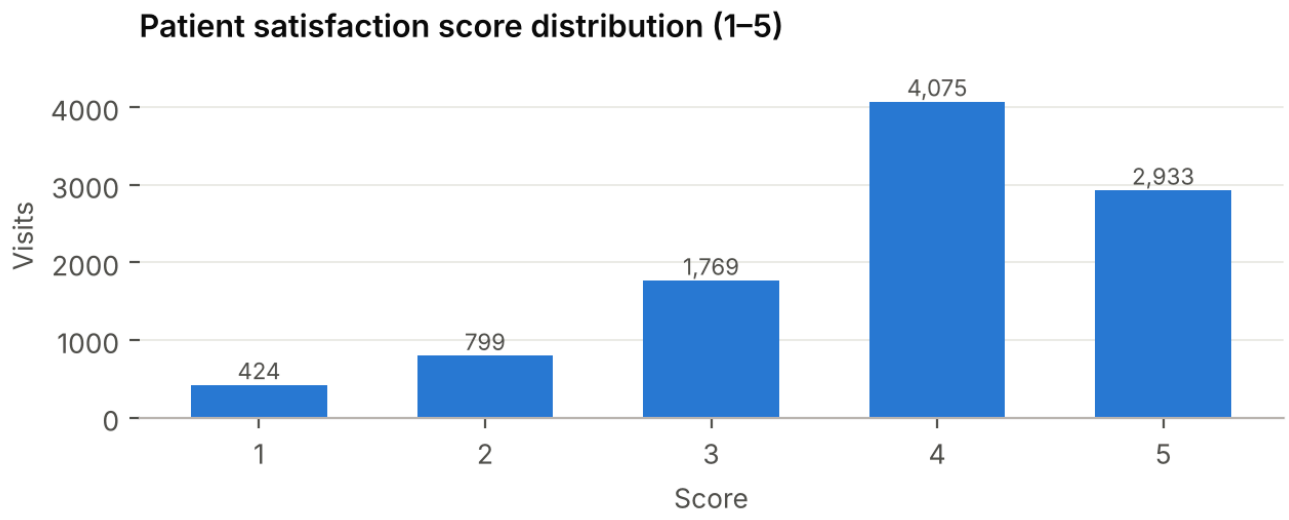


Figure 5.9 — Distribution of patient satisfaction scores (additional analysis, not in app)

10. Which diagnoses drive the longest and most expensive stays? *(question from the official problem statement)* - Visualisation: app filter Visit Type = Inpatient, then the diagnosis charts; SQL: `VW_DIAGNOSIS_ANALYTICS` (avg cost and avg LOS per diagnosis). - Finding (inpatient visits): the longest stays are Fracture (8.60 days), Anemia (8.32) and Diabetes (8.07). The most expensive are Kidney Disease (14,816.82 avg), Hypertension (14,640.75) and Asthma (14,549.67). Across all visit types, Heart Disease has the highest average LOS (1.20 days).

5.13 Dashboard Design

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

Activity 1: Interactive and visually appealing dashboards. How the implemented Streamlit dashboard addresses each design point in the template:

Template design point	Implementation in `streamlit_app.py`
Clear and Intuitive Layout	Wide layout: KPI row on top, then five rows of paired charts, then full-width cost chart and leaderboard (see Phase 3, Table 3.7)
Use Appropriate Visualisations	Line for time trend; bars for ranked comparisons; stacked bar for composition; donuts for 2–5 part shares; tables for detailed rows
Colour and Theming	Altair default categorical palette; `.streamlit/config.toml` exists but holds only comments (no custom theme)
Interactive Filters and Slicers	Date range plus four drop-downs in the sidebar
Drill-Down Capabilities	Filter-driven drill-down (e.g., choose a facility, then specialty); tooltips on every chart; sortable tables. No click-to-drill.
Responsive Design	`use_container_width=True` on all charts and tables; Streamlit columns stack on narrow screens
Custom Visuals and Icons	Not used
Use of Infographics	KPI metric cards with units

Major outcomes (full dataset, no filters):

- **Scale:** 10,000 visits from 2,889 patients across 10 facilities and 60 doctors; total cost 82.82 M (currency unspecified), of which insurance covers 59.92 M (72.3%).
- **Quality:** average satisfaction 3.83 / 5; 30-day readmission rate 12.03% (1,203 visits), highest for Depression (14.05%).

- **Utilisation:** outpatient care dominates (39.7% of visits); average LOS is 0.97 days overall but 7.69 days for inpatient stays.
- **Cost driver:** inpatient visits cost about 1.9× other visit types because of room charges. Lab is the largest cost component overall.
- **Facilities:** visit volume is evenly spread (958–1,039 per facility). Average cost per visit varies by only about 4% (8,085–8,432).

Activity 2: Published dashboard link. *Tableau Public: not applicable (no Tableau workbook in the project).* The Streamlit-in-Snowflake app is private to the Snowflake account. Its Snowsight URL can be shared only with users who have access: **[paste Snowsight app URL or a screen-recording link here]**.

5.14 Story Design

Field	Value
Date	[Submission Date] *(template shows 24 September 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	5 Marks

The template describes a Tableau Story: a sequence of visualisations that tells a narrative. No Tableau story exists in the project. The equivalent below is the narrative order used to present the Streamlit dashboard during the demonstration (Phase 8), with the screenshot to capture for each story point.

Story point	Message	Visual to capture (live app)
1. The big picture	10,000 visits, 82.8 M cost, 3.83 satisfaction, 12.03% readmissions	KPI row, no filters
2. Demand over time	Steady 257–351 visits per month	Monthly Visit Trend
3. Where care happens	Outpatient-heavy mix; facilities evenly loaded	Visit Type Split + Facility Performance
4. What it costs	Inpatient room cost doubles the visit cost	Average Cost Breakdown, filtered to Inpatient
5. Where quality slips	Depression and Back Pain readmit most	Readmission Rate by Diagnosis
6. Who delivers care	Doctor leaderboard for a chosen specialty	Doctor Performance Leaderboard filtered to one specialty

Observations:

- **Inpatient care drives cost:** inpatient visits average 13,973 against about 7,500 for other types, because only inpatient stays incur room charges.
- **Readmissions concentrate in chronic and mental-health conditions:** Depression (14.05%), Back Pain (13.98%), Asthma (13.75%) and Arthritis (13.43%) are above the 12.03% average.
- **Lab tests are the largest cost component:** 42.3% of total cost.
- **Satisfaction is uniform:** facility averages range only from 3.78 to 3.87, so satisfaction does not separate facilities in this dataset.

Activity 2: Published story link. *Tableau Public: not applicable.* **[Optional: link to a recorded demo video]**

5.15 Completed versus Planned Functionality

Table 5.4 — Implementation status (verified against code)

Feature	Status	Evidence / note
Database, schemas, RAW tables, file format, stage	Completed	`query.sql`
Load CSVs into RAW	Partially scripted	No `COPY INTO` in repo; load presumably done in the UI
STAGING cleaning	Completed	`query.sql` ; INITCAP side effects noted

Feature	Status	Evidence / note
CURATED dimensions and fact	Completed	`query.sql` ; `DIM_DATE` unused
MART KPI views (10)	Completed	`query.sql` ; doctor-view grain issue; one redundant view
Validation queries	Completed (manual review)	`query.sql`
Streamlit dashboard, 6 KPIs, 8 charts, 2 tables, 5 filters	Completed	`streamlit_app.py`
Insurance-provider analysis in app	Not implemented	Only totals in `VW_EXECUTIVE_KPI`
Weekday / weekend analysis using `DIM_DATE`	Not implemented	—
Automated refresh (Streams / Tasks)	Planned	Phase 8
RBAC and masking	Planned	Phase 3 §3.9
Automated tests	Added with this documentation	`tests/local_validation.py` , `tests/snowflake_validation.sql`

Phase 6 — Performance Testing

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Template covered:** Project Development Phase — Performance Test (uploaded as `Tableau_.docx`)

> **Honesty statement.** Only the tests marked **Executed (local replica)** were actually run. They were run on 21 October 2026 with `tests/local_validation.py`, which rebuilds the `query.sql` logic in an in-memory SQLite database from the six CSVs. No Snowflake account was available during documentation, so every Snowflake and live-dashboard test is marked **Not Executed**, with instructions to run it. No Snowflake timings or pass rates are claimed.

6.1 Project Performance Testing (template)

Field	Value
Date	10 February 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	[blank on template]

Project team shall fill the following information in the model performance testing template.

Table 6.1 — Performance test template

S.No.	Parameter	Screenshot / Values
1.	Data Rendered	6 source tables, 14,059 rows: patients 3,000; doctors 60; facilities 10; diagnoses 15; date 974; patient_visits 10,000. The app renders all 10,000 fact rows (32 columns) when no filter is set. The footer reads "Data filtered to 10,000 visits". *[Insert screenshot of the unfiltered app.]*
2.	Data Preprocessing	RAW: typed load with `CSV_FORMAT` (NULL_IF, date format). STAGING: `TRIM` on all text, `INITCAP` on 10 categorical columns, `WHERE key IS NOT NULL` on 6 tables. CURATED: `SELECT DISTINCT` dimensions; fact built with 4 `LEFT JOIN`s. App: upper-case column names, `pd.to_datetime(VISIT_DATE)`. Verified result: 0 rows dropped, 0 NULL or duplicate keys, 0 orphans.
3.	Utilization of Filters	5 sidebar filters: Date Range, Facility (10 + All), Doctor Specialty (10 + All), Visit Type (5 + All), Diagnosis Category (10 + All). They combine with AND and drive all 6 KPIs, 8 charts and 2 tables. Filter logic verified locally (TC-APP-FLT-01…03). Live UI test **Not Executed**.
4.	Calculation fields Used	SQL: `ROUND(SUM/AVG …, 2)`, `COUNT(DISTINCT …)`, `CASE WHEN READMISSION_WITHIN_30_DAYS='Yes'`, `READMISSION_RATE_PCT = 100·readmissions/visits`, `DATE_TRUNC('MONTH')`. App (pandas): total visits, unique patients, total cost, avg satisfaction, readmission rate %, avg LOS; monthly period; per-diagnosis readmission %; mean cost components (melted); facility and doctor aggregates.
5.	Dashboard design	**No of Visualizations / Graphs: 8 charts** (monthly trend line; visit-type donut; top-10 diagnoses bar; admission-status donut; facility bar; treatment-status bar; readmission-rate bar; cost-breakdown stacked bar), **plus 2 tables** (facility, doctor leaderboard) **and 6 KPI cards**.
6.	Story Design	**No of Visualizations / Graphs: 0 in Tableau** (no Tableau story in the project). Presentation storyline: 6 story points over the Streamlit visuals (Phase 5, §5.14).

6.2 Testing Objectives and Scope

Objectives: confirm that (1) every source row reaches each layer; (2) keys are complete, unique and referentially intact; (3) transformations behave as designed; (4) KPI views are numerically correct; (5) dashboard filter logic is correct; and (6) performance is acceptable for the data volume.

In scope: `query.sql` (all layers), the six CSVs, the logic in `streamlit_app.py`. **Out of scope:** load testing with concurrent users, security penetration testing, and browser compatibility.

6.3 Test Strategy and Methodology

Table 6.2 — Test levels and environments

Level	What	Environment	Tool	Status
Unit (SQL logic)	Each STAGING, CURATED and MART statement	E1: local replica (SQLite + Python UDF emulating Snowflake `INITCAP`)	`tests/local_validation.py`	**Executed**
Data quality	Completeness, uniqueness, validity, referential integrity, consistency	E1	same	**Executed**
Integration	RAW → STAGING → CURATED → MART row-flow and KPI reconciliation	E1	same + independent pandas recomputation	**Executed**
Functional (dashboard logic)	Filter and KPI calculations	E1 (pandas, same expressions as the app)	same	**Executed**
Platform	Same checks inside Snowflake	E2: Snowflake	`tests/snowflake_validation.sql`	**Not Executed**
Functional (UI)	Widgets, charts and tables in the deployed app	E3: Streamlit in Snowflake	Manual checklist §6.6	**Not Executed**
Performance	Query elapsed time and bytes scanned	E2	`INFORMATION_SCHEMA.QUERY_HISTORY`	**Not Executed** (local timings only)

Replica fidelity: the replica uses the same SQL text as `query.sql`, with two substitutions. `INITCAP` is a Python function implementing Snowflake's documented behaviour (default delimiters include whitespace and `-`; letters after the first are lower-cased). `DATE_TRUNC('MONTH', d)` is replaced by `SUBSTR(d,1,7)`. Snowflake-specific behaviour should still be confirmed with TC-SF-06.

How to run:

```
# from the repository root (Python 3.11+, pandas)
python tests/local_validation.py --json validation_results.json
```

6.4 Executed Test Cases (local replica)

Common preconditions: the six CSVs are in the repository root; Python 3.11+ with pandas is installed. **Common steps:** run the command above; the script loads the CSVs into RAW, builds STAGING, CURATED and MART, then evaluates each check.

Table 6.3 — Executed checks (local replica, 21 Oct 2026)

Test ID	Category	Test (check)	Expected	Actual	Status
TC-ING-01	Ingestion	PATIENTS_RAW row count equals CSV data rows	3000	3000	PASS
TC-ING-02	Ingestion	DOCTORS_RAW row count equals CSV data rows	60	60	PASS
TC-ING-03	Ingestion	FACILITIES_RAW row count equals CSV data rows	10	10	PASS
TC-ING-04	Ingestion	DIAGNOSES_RAW row count equals CSV data rows	15	15	PASS
TC-ING-05	Ingestion	DATE_RAW row count equals CSV data rows	974	974	PASS
TC-ING-06	Ingestion	PATIENT_VISITS_RAW row count equals CSV data rows	10000	10000	PASS
TC-DQ-NUL-01	Completeness	PATIENTS.PATIENT_ID has no NULLs	0	0	PASS
TC-DQ-DUP-01	Uniqueness	PATIENTS.PATIENT_ID is unique	0	0	PASS
TC-TRF-CNT-01	Transformation	STAGING PATIENTS keeps all RAW rows (no unintended filtering)	3000	3000	PASS
TC-DQ-NUL-02	Completeness	DOCTORS.DOCTOR_ID has no NULLs	0	0	PASS
TC-DQ-DUP-02	Uniqueness	DOCTORS.DOCTOR_ID is unique	0	0	PASS

Test ID	Category	Test (check)	Expected	Actual	Status
TC-TRF-CNT-02	Transformation	STAGING DOCTORS keeps all RAW rows (no unintended filtering)	60	60	PASS
TC-DQ-NUL-03	Completeness	FACILITIES.FACILITY_ID has no NULLs	0	0	PASS
TC-DQ-DUP-03	Uniqueness	FACILITIES.FACILITY_ID is unique	0	0	PASS
TC-TRF-CNT-03	Transformation	STAGING FACILITIES keeps all RAW rows (no unintended filtering)	10	10	PASS
TC-DQ-NUL-04	Completeness	DIAGNOSES.DIAGNOSIS_ID has no NULLs	0	0	PASS
TC-DQ-DUP-04	Uniqueness	DIAGNOSES.DIAGNOSIS_ID is unique	0	0	PASS
TC-TRF-CNT-04	Transformation	STAGING DIAGNOSES keeps all RAW rows (no unintended filtering)	15	15	PASS
TC-DQ-NUL-05	Completeness	DATE_STG.DATE has no NULLs	0	0	PASS
TC-DQ-DUP-05	Uniqueness	DATE_STG.DATE is unique	0	0	PASS
TC-TRF-CNT-05	Transformation	STAGING DATE_STG keeps all RAW rows (no unintended filtering)	974	974	PASS
TC-DQ-NUL-06	Completeness	PATIENT_VISITS.VISIT_ID has no NULLs	0	0	PASS
TC-DQ-DUP-06	Uniqueness	PATIENT_VISITS.VISIT_ID is unique	0	0	PASS
TC-TRF-CNT-06	Transformation	STAGING PATIENT_VISITS keeps all RAW rows (no unintended filtering)	10000	10000	PASS
TC-DQ-RI-01	Referential integrity	Every visit PATIENT_ID matches a dimension row	0	0	PASS
TC-DQ-RI-02	Referential integrity	Every visit DOCTOR_ID matches a dimension row	0	0	PASS
TC-DQ-RI-03	Referential integrity	Every visit FACILITY_ID matches a dimension row	0	0	PASS
TC-DQ-RI-04	Referential integrity	Every visit DIAGNOSIS_ID matches a dimension row	0	0	PASS
TC-DQ-RI-05	Referential integrity	Every VISIT_DATE exists in the DATE dimension	0	0	PASS
TC-TRF-FACT-01	Transformation	FACT row count equals STAGING visits (LEFT JOINs do not fan out)	10000	10000	PASS
TC-DQ-VAL-01	Validity	TOTAL_COST = CONSULTATION + MEDICATION + LAB + ROOM	0	0	PASS
TC-DQ-VAL-02	Validity	INSURANCE_COVERAGE + PATIENT_PAYMENT = TOTAL_COST (±0.01)	0	0	PASS
TC-DQ-VAL-03	Validity	No negative cost / payment values	0	0	PASS
TC-DQ-VAL-04	Validity	PATIENT_SATISFACTION within 1-5	0	0	PASS
TC-DQ-VAL-05	Validity	LENGTH_OF_STAY_DAYS >= 0	0	0	PASS
TC-DQ-VAL-06	Validity	READMISSION flag is exactly 'Yes' or 'No' after INITCAP	0	0	PASS
TC-DQ-VAL-07	Validity	INSURANCE_COVERAGE never exceeds TOTAL_COST	0	0	PASS
TC-DQ-VAL-08	Validity	DATE_OF_BIRTH earlier than REGISTRATION_DATE	0	0	PASS
TC-DQ-VAL-09	Validity	No visit dated before the patient's birth	0	0	PASS
TC-DQ-BR-01	Business rule	'Self Pay' patients receive no insurance coverage	0	1578	**FAIL**
TC-DQ-BR-02	Business rule	'Admitted' visits have LENGTH_OF_STAY_DAYS >= 1	0	653	**FAIL**
TC-MART-01	Transformation	VW_DOCTOR_ANALYTICS has one row per DOCTOR_ID	60	59	**FAIL**
TC-MART-02	Transformation	Sum of VW_FACILITY_ANALYTICS visits = total visits	10000	10000	PASS
TC-MART-03	Transformation	Sum of VW_MONTHLY_VISITS visits = total visits	10000	10000	PASS

Test ID	Category	Test (check)	Expected	Actual	Status
TC-TRF-STD-01	Transformation	INITCAP preserves acronym 'ENT' in SPECIALTY	original labels (incl. `ENT`)	`Ent` produced	**FAIL**
TC-TRF-STD-02	Transformation	INITCAP preserves 'Follow-up' in VISIT_TYPE	original labels (incl. `Follow-up`)	`Follow-Up` produced	**FAIL**
TC-KPI-01	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_VISITS matches independent pandas result	10000	10000	PASS
TC-KPI-02	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_PATIENTS matches independent pandas result	2889	2889	PASS
TC-KPI-03	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_COST matches independent pandas result	82818797	82818797	PASS
TC-KPI-04	KPI reconciliation	VW_EXECUTIVE_KPI.AVG_PATIENT_SATISFACTION matches independent pandas result	3.83	3.83	PASS
TC-KPI-05	KPI reconciliation	VW_EXECUTIVE_KPI.AVG_LENGTH_OF_STAY matches independent pandas result	0.97	0.97	PASS
TC-KPI-06	KPI reconciliation	VW_EXECUTIVE_KPI.TOTAL_READMISSIONS matches independent pandas result	1203	1203	PASS
TC-APP-FLT-01	Dashboard logic	Visit Type filter = 'Emergency' row count	1465	1465	PASS
TC-APP-FLT-02	Dashboard logic	Date range 2025-01-01..2025-12-31 row count	3812	3812	PASS
TC-APP-FLT-03	Dashboard logic	Empty filter result yields 0 readmission rate (no divide-by-zero)	0	0	PASS

Summary: 55 checks executed, 50 PASS, 5 FAIL. All structural checks (ingestion, completeness, uniqueness, referential integrity, cost and payer reconciliation, KPI reconciliation, filter logic) pass. The five failures are real findings about the data or the SQL, described in §6.8.

6.4.1 Data completeness, uniqueness, validity and referential integrity (summary)

Dimension	Checks	Result
Completeness (NULL keys)	6 tables	0 NULL keys
Uniqueness (duplicate keys)	6 tables	0 duplicates
Referential integrity	4 FKs + visit date → DATE	0 orphans
Validity (ranges, domains, reconciliations)	9 rules	0 violations
Business-rule plausibility	2 assumed rules	2 violations (data anomalies)

6.4.2 SQL transformation validation

- STAGING keeps 100% of RAW rows in all 6 tables (TC-TRF-CNT-01...06).
- FACT has exactly 10,000 rows: the LEFT JOINs do not fan out (TC-TRF-FACT-01).
- Monthly and facility view totals add back to 10,000 (TC-MART-02/03).
- Six executive KPIs match an independent pandas computation on the raw CSV (TC-KPI-01...06).
- Defects found: INITCAP label changes (TC-TRF-STD-01/02) and doctor-view grain (TC-MART-01).

6.5 Snowflake Test Cases (Not Executed)

Precondition: `query.sql` executed in full and RAW tables loaded. **Steps:** open a worksheet, run the matching block of `tests/snowflake_validation.sql`, and record the output.

Table 6.4 — Snowflake platform tests

Test ID	Test	Expected result	Actual	Status
TC-SF-01	Row counts per layer	10000, 10000, 10000, 3000, 3000, 60, 10, 15, 974	[record]	Not Executed

Test ID	Test	Expected result	Actual	Status
TC-SF-02	Cost reconciliation	0 failures	[record]	Not Executed
TC-SF-03	Payer reconciliation	0 failures	[record]	Not Executed
TC-SF-04	Validity ranges	0 failures	[record]	Not Executed
TC-SF-05	Visit dates in DIM_DATE	0 failures	[record]	Not Executed
TC-SF-06	INITCAP side effects	Replica predicts `Ent , Follow-Up`	[record]	Not Executed
TC-SF-07	Doctor view grain	Replica predicts 59 view rows vs 60 doctors	[record]	Not Executed
TC-SF-08	KPI baseline	10000, 2889, 82818797.00, 3.83, 0.97, 1203	[record]	Not Executed
TC-SF-PERF	Elapsed time per query	Record `ELAPSED_S` , `BYTES_SCANNED`	[record]	Not Executed

6.6 Dashboard Functional and Filter Tests (Not Executed, manual)

Precondition: app deployed in Snowsight and opened by a role with SELECT on `CURATED.FACT_PATIENT_VISITS` .

Table 6.5 — Dashboard UI test cases

Test ID	Steps	Expected result	Actual	Status
TC-UI-01	Open the app with no filters	Six KPIs: 10,000 · 2,889 · $82,818,797 · 3.83 / 5 · 12.0% · 0.97 days	[record]	Not Executed
TC-UI-02	Visit Type = Inpatient	Total Visits 1,261; Avg LOS 7.69 days; room cost appears in breakdown	[record]	Not Executed
TC-UI-03	Visit Type = Emergency	Total Visits 1,465 (matches TC-APP-FLT-01)	[record]	Not Executed
TC-UI-04	Date Range 2025-01-01 → 2025-12-31	Total Visits 3,812 (matches TC-APP-FLT-02)	[record]	Not Executed
TC-UI-05	Pick only a start date in Date Range	No error; data unfiltered by date until the second date is chosen	[record]	Not Executed
TC-UI-06	Facility = City General Hospital	Total Visits 1,039; facility table shows one row	[record]	Not Executed
TC-UI-07	Specialty list	Shows 10 specialties; check whether "Ent" appears (expected defect D-02)	[record]	Not Executed
TC-UI-08	Combine filters to produce zero rows (e.g., narrow date + facility + specialty)	KPIs show 0 / 0.00; no exception; charts empty	[record]	Not Executed
TC-UI-09	Hover over any chart mark	Tooltip shows category and value	[record]	Not Executed
TC-UI-10	Sort the doctor leaderboard by AVG_SATISFACTION	Table re-sorts	[record]	Not Executed
TC-UI-11	Count doctor leaderboard rows (no filters)	Expected 60; replica predicts 59 (defect D-01)	[record]	Not Executed
TC-UI-12	Reload within 10 minutes	Second load faster (cache `ttl=600`)	[record]	Not Executed

6.7 Query Performance and Resource Usage

Measured (local replica, single run, container CPU; NOT Snowflake):

Stage	Elapsed
Load 6 CSVs into RAW (pandas → SQLite)	≈ 0.15 s
Build STAGING + CURATED (12 CTAS)	≈ 0.08 s
Query all 10 MART views	≈ 0.05 s

These numbers show only that the logic is light at this volume (10,000 fact rows). They are not evidence of Snowflake warehouse performance.

To measure in Snowflake (Not Executed): 1. Use warehouse `COMPUTE_WH` at X-Small. Optionally run `ALTER SESSION SET USE_CACHED_RESULT = FALSE;` to avoid result-cache hits. 2. Run `query.sql` sections and `tests/snowflake_validation.sql`. 3. Run the `TC-SF-PERF` query and record `TOTAL_ELAPSED_TIME`, `BYTES_SCANNED` and `ROWS_PRODUCED` per statement. 4. In Snowsight, open the *Query Profile* of `FACT_PATIENT_VISITS` creation and of `VW_DOCTOR_ANALYTICS` to inspect join and aggregate operators. 5. For the app, note first-load time (cold cache) and filtered-interaction time, and record credit usage from *Admin → Cost management*.

Expected behaviour (analytical, not measured): the app performs one full read of a 10,000 × 32 table, then filters in memory, so interactions do not query Snowflake again until the 600-second cache expires. Memory use grows linearly with fact rows, which is the main scalability limit (Phase 8).

6.8 Defects, Limitations and Corrective Actions

Table 6.6 — Defect log

ID	Severity	Found by	Description	Corrective action (proposed)	Status
D-01	Medium	TC-MART-01	`VW_DOCTOR_ANALYTICS` and the app leaderboard group by name + specialty + department, merging D018 and D020 ("Dr. Ananya Reddy", Orthopedics, Specialty Care): 338 visits shown as one doctor	Add `DOCTOR_ID` to `SELECT` / `GROUP BY` in the view and to `groupby([...])` in the app	Open
D-02	Low	TC-TRF-STD-01	`INITCAP` turns acronym `ENT` into `Ent`	`CASE WHEN UPPER(TRIM(SPECIALTY))='ENT' THEN 'ENT' ELSE INITCAP(TRIM(SPECIALTY)) END`, or drop INITCAP for already-clean columns	Open
D-03	Low	TC-TRF-STD-02	`INITCAP` turns `Follow-up` into `Follow-Up`	`INITCAP(TRIM(VISIT_TYPE), ' ')` (space-only delimiter)	Open
D-04	Medium (data)	TC-DQ-BR-01	All 1,578 "Self Pay" visits have insurance coverage (≈72.7%)	Confirm the source semantics; if Self Pay means uninsured, correct upstream or add a STAGING rule plus a DQ alert	Open: needs data owner
D-05	Low (data)	TC-DQ-BR-02	653 Emergency visits are "Admitted" with LOS = 0 and no room cost	Confirm the definition (observation stay?); document or derive LOS	Open: needs data owner
D-06	Low	Code review	`VW_PATIENT_VISIT_TYPE_ANALYSIS` duplicates `VW_VISIT_TYPE_DISTRIBUTION` (only sort differs)	Drop one view	Open
D-07	Low	Code review	`DIM_DATE` is never joined; weekday and weekend analysis unavailable	Join on `VISIT_DATE = DATE` in a view; add a weekend chart	Open
D-08	Info	Code review	App labels costs with `$`, but the source currency is unspecified (Indian cities)	Confirm currency; change labels	Open
D-09	Info	Code review	No `COPY INTO` in `query.sql`; loading is not reproducible from the repository	Add the load commands from Phase 5 §5.1.2	Open

Limitations of this test phase: Snowflake and UI tests are not executed. The business-rule checks rest on assumed semantics. No concurrency or large-volume tests were run.

Phase 8 — Project Demonstration

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Demonstration of Proposed Features · Team Involvement in Demonstration · Project Demo Planning · Communication · Scalability & Future Plan (1 mark each)

> **Availability note.** The files show that the Snowflake objects and the Streamlit app were written, and `snowflake.yml` holds an auto-generated app identifier. A live Snowflake account, deployed app or demo recording could **not** be verified. Expected on-screen values below come from the local replica (Phase 6). Confirm them in your own account before presenting. Team-specific fields are placeholders.

8.1 Presentation-Ready Overview

Problem	Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.
Solution	Snowflake pipeline `RAW → STAGING → CURATED → MART` plus a Streamlit-in-Snowflake dashboard with 6 KPIs, 8 charts, 2 tables and 5 filters.
Data	6 CSVs, 14,059 rows; 10,000 visits; January 2024 to August 2026.
Stack	Snowflake SQL, internal stage and file format, Streamlit in Snowflake, Python 3.11, pandas, Altair, Snowpark.
Evidence	55 automated checks on a local replica: 50 pass, 5 findings documented with fixes.

8.2 Demonstration of Proposed Features

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This document lists every feature proposed during planning and tracks whether each was implemented and demonstrated.

Table 8.1 — Proposed features and status (implementation status verified against code)

S.No	Feature Name	Description	Status (Implemented / Partial / Pending)	Demonstrated (Yes / No)	Remarks
1	Layered Snowflake database	`HEALTH_DB` with RAW, STAGING, CURATED, MART	Implemented	[Yes/No]	`query.sql`
2	Ingestion objects	`CSV_FORMAT` , `HEALTHCARE_STAGE` , 6 typed RAW tables	Implemented	[Yes/No]	—
3	Data load to RAW	Load 6 CSVs into RAW	Partial	[Yes/No]	No `COPY INTO` in repo; UI load assumed
4	Cleaning & standardisation	TRIM, INITCAP, NOT NULL keys	Implemented	[Yes/No]	`ENT → Ent` side effect
5	Star schema	5 dimensions + visit fact	Implemented	[Yes/No]	`DIM_DATE` not joined
6	Data-quality checks	Row counts, NULLs, duplicates, orphans	Implemented	[Yes/No]	Manual review of results
7	KPI views	10 MART views	Implemented	[Yes/No]	Doctor-view grain defect D-01
8	KPI cards	6 metrics	Implemented	[Yes/No]	—

S.No	Feature Name	Description	Status (Implemented / Partial / Pending)	Demonstrated (Yes / No)	Remarks
9	Trend & mix charts	Monthly trend, visit type, top diagnoses, admission, treatment	Implemented	[Yes/No]	—
10	Facility & doctor comparison	Facility chart and table; doctor leaderboard	Implemented	[Yes/No]	—
11	Readmission & cost analysis	Readmission by diagnosis; cost breakdown by visit type	Implemented	[Yes/No]	—
12	Interactive filters	Date, facility, specialty, visit type, diagnosis category	Implemented	[Yes/No]	—
13	Insurance / payer analysis	Coverage versus patient payment by insurer	Partial	[Yes/No]	Totals only (`VW_EXECUTIVE_KPI`)
14	Security & automation	RBAC, masking, scheduled refresh	Pending	No	Future scope

Feature Implementation Summary:

Metric	Value
Total Features Proposed	14
Total Features Implemented	11 fully + 2 partially
Total Features Demonstrated	[count after demo]
Overall Implementation Rate (%)	78.6% fully implemented (11 / 14); 85.7% counting partial features as half (12 / 14)

8.3 Team Involvement in Demonstration

Field	Value
Date	[Demo Date] *(template shows 15 March 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This document records each team member's participation and role during the demonstration. *(Roles are suggestions mapped to the demo plan; fill in actual names and participation.)*

S.No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation (Active / Passive)
1	[Name]	Lead presenter	Introduction & problem	Framing, objectives, wrap-up	[Active/Passive]
2	[Name]	Data engineer	Architecture & pipeline	Layers, SQL objects, data load	[ ]
3	[Name]	Analytics engineer	Data model & KPIs	Star schema, MART views, validation	[ ]
4	[Name]	App developer	Live dashboard	Filters and charts walkthrough	[ ]
5	[Name]	QA	Testing results	Test summary and defects	[ ]
6	[Name]	Q&A coordinator	Q&A	Routes questions	[ ]

Aspect	Details
Team Leader / Coordinator	[Name]
Overall Team Coordination Rating (1–5)	[ ]
Any issues during demo	[record, e.g., warehouse resume delay]

Aspect	Details
How issues were resolved	[record]

8.4 Project Demo Planning

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

Table 8.2 — Demo plan (10 minutes)

S.No	Demo Section	Description	Duration (mins)	Responsible Member
1	Introduction & problem	Persona, pain points, objectives	1.0	[Member 1]
2	Architecture & data model	Architecture diagram; layers; star schema	1.5	[Member 2]
3	Pipeline in Snowflake	Show schemas and objects; run row-count and orphan checks	2.0	[Member 2/3]
4	KPI views	Query `VW_EXECUTIVE_KPI` and `VW_READMISSION_ANALYTICS`	1.0	[Member 3]
5	Live dashboard	KPIs, filters, charts, leaderboard	3.0	[Member 4]
6	Testing, limitations & future	Test summary, defects, roadmap	1.5	[Member 5]

Demo Flow Summary:

Step	Activity	Notes
1	Introduction & Problem Statement	One slide: the official problem statement and its three questions
2	Solution Overview	Architecture figure (Phase 3, Figure 3.1)
3	Live Feature Demonstration	Snowsight worksheet → Streamlit app (script in §8.7)
4	Q&A Session	Use FAQ §8.9; keep this documentation open

8.5 Communication

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

The project files contain no record of team communication. The channels below are a plan; confirm or edit them to match what the team actually used.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	[e.g., WhatsApp / Google Meet]	All members	Yesterday / today / blockers
2	Progress Update	Weekly	[e.g., Google Meet + shared Drive]	Team + mentor	Sprint progress, burndown
3	Issue / Bug Discussion	As Needed	[e.g., GitHub Issues / chat]	Relevant members	Resolve SQL or app defects
4	Stakeholder Review	Bi-Weekly	[e.g., Meet with faculty guide]	Team + [Faculty Guide]	Validate scope and deliverables

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
5	Final Demo Rehearsal	Once	[In person / Meet]	All members	Timing, hand-offs, fallback plan
6	Documentation review	Before submission	Shared Drive / GitHub PR	All members	Fill placeholders, proof-read

S.No	Challenge Faced	Resolution / Action Taken
1	[e.g., differing local setups]	[e.g., used Snowsight only]
2	[ ]	[ ]
3	[ ]	[ ]

8.6 Step-by-Step Setup and Execution (for the demo machine)

1. Log in to Snowsight with a role that owns `HEALTH_DB` (or can create it). Select warehouse `COMPUTE_WH` and run `ALTER WAREHOUSE COMPUTE_WH RESUME IF SUSPENDED;` a minute before the demo to avoid a resume delay.
2. If the database does not exist yet: follow Phase 5 §5.1.2 (create objects → upload CSVs → load → run the rest of `query.sql`).
3. Open the Streamlit app (Projects → Streamlit → "HEALTH") and wait for the first load (cache is cold).
4. Keep a worksheet open with the demo queries in §8.7.
5. Fallback: if Snowflake is unreachable, present the figures in Phase 5 and run `python tests/local_validation.py` locally to show the pipeline logic.

8.7 Demonstration Script (≈ 9 minutes)

0:00 – Introduction (1 min). "Hospital administrators and finance teams need one consolidated, queryable view of patient visits, covering cost, length of stay, satisfaction and readmissions. Today that data sits fragmented across raw operational records. Without a structured warehouse they cannot easily see which facilities and doctors perform well, which diagnoses drive the longest and most expensive stays, or where satisfaction is at risk. We built that warehouse in Snowflake, with a native Streamlit dashboard on top."

1:00 – Architecture (1.5 min). Show Figure 3.1. "CSV files land in an internal stage and load into typed RAW tables. STAGING trims and standardises text and drops rows with missing keys. CURATED holds a star schema with a 10,000-row visit fact and five dimensions. MART holds ten KPI views. The app reads the curated fact through the active Snowpark session, so there are no credentials in code."

2:30 – Pipeline in Snowflake (2 min). In the object explorer, expand `HEALTH_DB` to show the four schemas. Run:

```
SELECT 'FACT_PATIENT_VISITS' AS T, COUNT(*) FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
UNION ALL SELECT 'DIM_PATIENT', COUNT(*) FROM HEALTH_DB.CURATED.DIM_PATIENT;
-- expected (local replica): 10000, 3000

SELECT COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
WHERE FIRST_NAME IS NULL OR DOCTOR_NAME IS NULL OR FACILITY_NAME IS NULL OR DIAGNOSIS_NAME IS NULL;
-- expected: 0
```

4:30 – KPI views (1 min).

```
SELECT * FROM HEALTH_DB.MART.VW_EXECUTIVE_KPI;
-- expected: 10000 | 2889 | 60 | 10 | 82818797.00 | 59918913.84 | 22899883.16 | 3.83 | 0.97 | 1203

SELECT DIAGNOSIS_NAME, READMISSION_RATE_PCT FROM HEALTH_DB.MART.VW_READMISSION_ANALYTICS LIMIT 3;
-- expected: Depression 14.05 | Back Pain 13.98 | Asthma 13.75
```

5:30 – Live dashboard (3 min). 1. Unfiltered KPI row: 10,000 visits; 2,889 patients; total cost; 3.83 / 5; 12.0%; 0.97 days. 2. Set **Visit Type = Inpatient**: visits fall to 1,261, LOS rises to 7.69 days, and room cost appears in the cost breakdown. "Room charges almost double inpatient cost." 3. Reset, then set **Diagnosis Category = Mental Health**: the readmission chart shows Depression on top. 4. Choose a **Facility**: the facility table narrows; point out average cost and satisfaction. 5. Scroll to the **Doctor Performance Leaderboard** and sort by satisfaction.

8:30 – Testing and roadmap (1.5 min). "We executed 55 automated checks on a replica of the pipeline. All structural checks pass. Five findings are documented with fixes: a doctor-grouping bug, two INITCAP label changes, and two source-data anomalies. Next steps: scripted loads with Tasks, RBAC and masking, and SQL push-down for scale."

8.8 Suggested Screenshots to Capture

#	Screen	Purpose / where it is used
S1	Snowsight object explorer showing `HEALTH_DB` and its 4 schemas	Phase 3/5 evidence
S2	Stage file list for `HEALTHCARE_STAGE`	Ingestion evidence
S3	RAW row-count query result	TC-SF-01
S4	STAGING NULL / duplicate check results	Phase 6
S5	Orphan check result (0)	Phase 6
S6	`VW_EXECUTIVE_KPI` result	Phase 7 Results
S7	App unfiltered: header, KPI row, sidebar	Phase 7 §7.1 placeholder
S8	App with Visit Type = Inpatient	Cost and LOS story
S9	Readmission chart with Mental Health filter	Quality story
S10	Facility performance chart and table	Facility comparison
S11	Doctor leaderboard	Doctor comparison
S12	`snowflake_validation.sql` outputs and Query History	Phase 6 Snowflake tests and performance

Save the files under `docs/screenshots/` with the names `S1_object_explorer.png` and so on.

8.9 Frequently Asked Technical Questions

Q1. Why Snowflake rather than MySQL? Storage and compute scale independently, the SQL is standard, the internal stage and file format handle CSV loading, and Streamlit runs natively, so the app needs no separate server or stored credentials.

Q2. Why four schemas? Separation of concerns and traceability. RAW keeps data as loaded, STAGING cleans it, CURATED models it for analysis, and MART holds consumption logic. Any layer can be rebuilt from the one before.

Q3. What is the grain of the fact table? One row per patient visit (`VISIT_ID`), 10,000 rows.

Q4. Why LEFT JOIN in the fact build? So that a visit is never lost when reference data is missing. The orphan check then counts any unmatched rows. There are 0 in this data.

Q5. Why is the fact table denormalised? The app and views can read one table with no joins. The trade-off is redundancy, and dimension changes require a rebuild.

Q6. Is DIM_DATE used? It is built but not yet joined. Monthly trends use `DATE_TRUNC` on `VISIT_DATE` . Joining it would enable weekday and weekend analysis.

Q7. How is the readmission rate calculated? `100 × count(READMISSION_WITHIN_30_DAYS = 'Yes') ÷ count(visits)` over the filtered set. The flag comes pre-computed in the source.

Q8. How does the app authenticate? `get_active_session()` returns the Snowpark session of the running Streamlit-in-Snowflake app. No passwords or keys appear in code.

Q9. Why does the app use pandas instead of the MART views? For simplicity: it loads the fact table once (cached for 600 s) and recomputes metrics for any filter combination in memory. For larger data, we would push filters and aggregations to SQL.

Q10. How did you test without Snowflake? `tests/local_validation.py` rebuilds the same SQL in SQLite, emulating Snowflake's `INITCAP` , and runs 55 checks. A Snowflake version of the checks is in `tests/snowflake_validation.sql` .

Q11. What defects did you find? Two doctors with identical name, specialty and department merge in the leaderboard. `INITCAP` changes `ENT` to `Ent` and `Follow-up` to `Follow-Up` . "Self Pay" visits show insurance coverage. Some admitted emergency visits have 0-day LOS.

Q12. Is patient data protected? The data appears synthetic. For real data we would add least-privilege roles, masking policies on names and date of birth, and remove names from the fact table.

Q13. How would you automate refresh? Load new files with Snowpipe or a scheduled `COPY INTO`, then rebuild STAGING and CURATED with a chain of Snowflake Tasks, or use Streams for incremental processing.

8.10 Known Limitations

Load not scripted · no RBAC or masking · app loads the full fact table into memory · KPI logic duplicated between views and app · `DIM_DATE` unused · INITCAP side effects · doctor-view grain defect · source anomalies (Self Pay coverage, zero-LOS admissions) · currency unspecified · Snowflake and UI tests not executed · synthetic data.

8.11 Scalability & Future Plan

Field	Value
Date	[Submission Date] *(template shows 15 March 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This section explains how the solution can scale to larger user bases, more data and more features, and gives the roadmap.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Manual, unscripted data load and full rebuilds	Refresh depends on a person; not reproducible from the repository	High
2	No RBAC or masking; PII in the fact table	Unsuitable for real patient data	High
3	App loads the whole fact table into pandas	Memory and latency grow with volume	Medium
4	Doctor-view grain and INITCAP defects	Misleading leaderboard and labels	Medium
5	KPI logic duplicated in SQL and pandas	Risk of divergence	Low

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single app on `COMPUTE_WH`; per-user cache	Multi-cluster warehouse (auto-scale) for concurrency; separate warehouse for ELT and BI
2	Data Storage	6 tables, ~14 K rows, full `CREATE OR REPLACE` each run	Incremental loads with Streams and `MERGE`; clustering on `VISIT_DATE` at large volumes; Time Travel for recovery
3	Performance	Full fact read, then pandas filters	Push filters into Snowpark or SQL (`WHERE` and `GROUP BY` in the warehouse); use MART views; dynamic tables for pre-aggregates
4	Security	Development role; no grants or masking	Role hierarchy (`HEALTH_ENGINEER` / `ANALYST` / `APP_VIEWER`), masking and row-access policies, app moved out of `RAW`

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Fix D-01 to D-03; script `COPY INTO`; run Snowflake and UI tests; RBAC and masking	[Next 2 weeks, estimate]	Correct, reproducible, secure baseline

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 3	Tasks or Snowpipe automation; Streams and `MERGE` incremental model; `DIM_DATE` weekday and seasonality views; insurer and payer dashboard page	[1–2 months, estimate]	Hands-off refresh; deeper finance insight
Phase 4	SCD Type 2 dimensions; Snowpark ML readmission-risk model; alerts when readmission rate exceeds a threshold; CI running the test scripts on every commit	[3+ months, estimate]	Predictive, monitored, production-grade analytics

Part III — Appendices

Appendix A and B reproduce the project source files unchanged. Appendix C is generated from the DDL and the CSV data. Setup instructions are in Phase 5 §5.1 and the repository README. Test procedures are in Phase 6 and TEST_CASES.md.

Appendix A — query.sql (complete)

```
CREATE DATABASE IF NOT EXISTS HEALTH_DB;

USE DATABASE HEALTH_DB;

CREATE SCHEMA IF NOT EXISTS RAW;
CREATE SCHEMA IF NOT EXISTS STAGING;
CREATE SCHEMA IF NOT EXISTS CURATED;
CREATE SCHEMA IF NOT EXISTS MART;


USE SCHEMA HEALTH_DB.RAW;

CREATE OR REPLACE TABLE PATIENTS_RAW (
    PATIENT_ID          VARCHAR,
    FIRST_NAME          VARCHAR,
    LAST_NAME           VARCHAR,
    GENDER              VARCHAR,
    DATE_OF_BIRTH       DATE,
    CITY                VARCHAR,
    INSURANCE_PROVIDER  VARCHAR,
    REGISTRATION_DATE   DATE
);

CREATE OR REPLACE TABLE DOCTORS_RAW (
    DOCTOR_ID        VARCHAR,
    DOCTOR_NAME      VARCHAR,
    SPECIALTY        VARCHAR,
    EXPERIENCE_YEARS NUMBER,
    DEPARTMENT       VARCHAR
);

CREATE OR REPLACE TABLE FACILITIES_RAW (
    FACILITY_ID   VARCHAR,
    FACILITY_NAME VARCHAR,
    CITY          VARCHAR,
    FACILITY_TYPE VARCHAR
);

CREATE OR REPLACE TABLE DIAGNOSES_RAW (
    DIAGNOSIS_ID       VARCHAR,
    DIAGNOSIS_NAME     VARCHAR,
    DIAGNOSIS_CATEGORY VARCHAR
);

CREATE OR REPLACE TABLE DATE_RAW (
    DATE        DATE,
    YEAR        NUMBER,
    QUARTER     VARCHAR,
    MONTH       NUMBER,
    MONTH_NAME  VARCHAR,
    WEEK        NUMBER,
    DAY_OF_WEEK VARCHAR,
    IS_WEEKEND  BOOLEAN
);

CREATE OR REPLACE TABLE PATIENT_VISITS_RAW (
    VISIT_ID                    VARCHAR,
    PATIENT_ID                  VARCHAR,
    DOCTOR_ID                   VARCHAR,
    FACILITY_ID                 VARCHAR,
    DIAGNOSIS_ID                VARCHAR,
    VISIT_DATE                  DATE,
    VISIT_TYPE                  VARCHAR,
    ADMISSION_STATUS            VARCHAR,
    TREATMENT_STATUS            VARCHAR,
    LENGTH_OF_STAY_DAYS         NUMBER,
    CONSULTATION_COST           NUMBER(12,2),
    MEDICATION_COST             NUMBER(12,2),
    LAB_COST                    NUMBER(12,2),
    ROOM_COST                   NUMBER(12,2),
    TOTAL_COST                  NUMBER(12,2),
    INSURANCE_COVERAGE          NUMBER(12,2),
    PATIENT_PAYMENT             NUMBER(12,2),
    PATIENT_SATISFACTION        NUMBER(5,2),
```

```
    READMISSION_WITHIN_30_DAYS  VARCHAR
);

SHOW TABLES IN SCHEMA HEALTH_DB.RAW;


USE SCHEMA HEALTH_DB.RAW;

CREATE OR REPLACE FILE FORMAT HEALTH_DB.RAW.CSV_FORMAT
    TYPE                         = CSV
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    SKIP_HEADER                  = 1
    NULL_IF                      = ('NULL', 'null', '')
    EMPTY_FIELD_AS_NULL          = TRUE
    DATE_FORMAT                  = 'YYYY-MM-DD';

CREATE OR REPLACE STAGE HEALTH_DB.RAW.HEALTHCARE_STAGE
    FILE_FORMAT = HEALTH_DB.RAW.CSV_FORMAT;

SHOW STAGES IN SCHEMA HEALTH_DB.RAW;


SELECT 'PATIENTS_RAW'       AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.PATIENTS_RAW       UNION ALL
SELECT 'DOCTORS_RAW'        AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.DOCTORS_RAW        UNION ALL
SELECT 'FACILITIES_RAW'     AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.FACILITIES_RAW     UNION ALL
SELECT 'DIAGNOSES_RAW'      AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.DIAGNOSES_RAW      UNION ALL
SELECT 'DATE_RAW'           AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.DATE_RAW           UNION ALL
SELECT 'PATIENT_VISITS_RAW' AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.RAW.PATIENT_VISITS_RAW;


USE SCHEMA HEALTH_DB.STAGING;

CREATE OR REPLACE TABLE PATIENTS AS
SELECT
    TRIM(PATIENT_ID)                    AS PATIENT_ID,
    TRIM(FIRST_NAME)                    AS FIRST_NAME,
    TRIM(LAST_NAME)                     AS LAST_NAME,
    INITCAP(TRIM(GENDER))               AS GENDER,
    DATE_OF_BIRTH,
    INITCAP(TRIM(CITY))                 AS CITY,
    TRIM(INSURANCE_PROVIDER)            AS INSURANCE_PROVIDER,
    REGISTRATION_DATE
FROM HEALTH_DB.RAW.PATIENTS_RAW
WHERE PATIENT_ID IS NOT NULL;

CREATE OR REPLACE TABLE DOCTORS AS
SELECT
    TRIM(DOCTOR_ID)                     AS DOCTOR_ID,
    TRIM(DOCTOR_NAME)                   AS DOCTOR_NAME,
    INITCAP(TRIM(SPECIALTY))            AS SPECIALTY,
    EXPERIENCE_YEARS,
    INITCAP(TRIM(DEPARTMENT))           AS DEPARTMENT
FROM HEALTH_DB.RAW.DOCTORS_RAW
WHERE DOCTOR_ID IS NOT NULL;

CREATE OR REPLACE TABLE FACILITIES AS
SELECT
    TRIM(FACILITY_ID)                   AS FACILITY_ID,
    TRIM(FACILITY_NAME)                 AS FACILITY_NAME,
    INITCAP(TRIM(CITY))                 AS CITY,
    INITCAP(TRIM(FACILITY_TYPE))        AS FACILITY_TYPE
FROM HEALTH_DB.RAW.FACILITIES_RAW
WHERE FACILITY_ID IS NOT NULL;

CREATE OR REPLACE TABLE DIAGNOSES AS
SELECT
    TRIM(DIAGNOSIS_ID)                  AS DIAGNOSIS_ID,
    TRIM(DIAGNOSIS_NAME)                AS DIAGNOSIS_NAME,
    INITCAP(TRIM(DIAGNOSIS_CATEGORY))   AS DIAGNOSIS_CATEGORY
FROM HEALTH_DB.RAW.DIAGNOSES_RAW
WHERE DIAGNOSIS_ID IS NOT NULL;

CREATE OR REPLACE TABLE DATE AS
SELECT
    DATE,
    YEAR,
    QUARTER,
    MONTH,
    MONTH_NAME,
```

```
    WEEK,
    DAY_OF_WEEK,
    IS_WEEKEND
FROM HEALTH_DB.RAW.DATE_RAW
WHERE DATE IS NOT NULL;

CREATE OR REPLACE TABLE PATIENT_VISITS AS
SELECT
    TRIM(VISIT_ID)                                  AS VISIT_ID,
    TRIM(PATIENT_ID)                                AS PATIENT_ID,
    TRIM(DOCTOR_ID)                                 AS DOCTOR_ID,
    TRIM(FACILITY_ID)                               AS FACILITY_ID,
    TRIM(DIAGNOSIS_ID)                              AS DIAGNOSIS_ID,
    VISIT_DATE,
    INITCAP(TRIM(VISIT_TYPE))                       AS VISIT_TYPE,
    INITCAP(TRIM(ADMISSION_STATUS))                 AS ADMISSION_STATUS,
    INITCAP(TRIM(TREATMENT_STATUS))                 AS TREATMENT_STATUS,
    LENGTH_OF_STAY_DAYS,
    CONSULTATION_COST,
    MEDICATION_COST,
    LAB_COST,
    ROOM_COST,
    TOTAL_COST,
    INSURANCE_COVERAGE,
    PATIENT_PAYMENT,
    PATIENT_SATISFACTION,
    INITCAP(TRIM(READMISSION_WITHIN_30_DAYS))       AS READMISSION_WITHIN_30_DAYS
FROM HEALTH_DB.RAW.PATIENT_VISITS_RAW
WHERE VISIT_ID IS NOT NULL;

SELECT 'PATIENTS'       AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.PATIENTS       WHERE PATIENT_ID
IS NULL UNION ALL
SELECT 'DOCTORS'        AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.DOCTORS        WHERE DOCTOR_ID
IS NULL UNION ALL
SELECT 'FACILITIES'     AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.FACILITIES     WHERE FACILITY_ID
IS NULL UNION ALL
SELECT 'DIAGNOSES'      AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.DIAGNOSES      WHERE DIAGNOSIS_ID
IS NULL UNION ALL
SELECT 'DATE'           AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.DATE           WHERE DATE
IS NULL UNION ALL
SELECT 'PATIENT_VISITS' AS TABLE_NAME, COUNT(*) AS NULL_RECORDS FROM HEALTH_DB.STAGING.PATIENT_VISITS WHERE VISIT_ID
IS NULL;

SELECT 'PATIENTS'       AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT PATIENT_ID   FROM
HEALTH_DB.STAGING.PATIENTS       GROUP BY PATIENT_ID   HAVING COUNT(*) > 1) UNION ALL
SELECT 'DOCTORS'        AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT DOCTOR_ID    FROM
HEALTH_DB.STAGING.DOCTORS        GROUP BY DOCTOR_ID    HAVING COUNT(*) > 1) UNION ALL
SELECT 'FACILITIES'     AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT FACILITY_ID  FROM
HEALTH_DB.STAGING.FACILITIES     GROUP BY FACILITY_ID  HAVING COUNT(*) > 1) UNION ALL
SELECT 'DIAGNOSES'      AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT DIAGNOSIS_ID FROM
HEALTH_DB.STAGING.DIAGNOSES      GROUP BY DIAGNOSIS_ID HAVING COUNT(*) > 1) UNION ALL
SELECT 'DATE'           AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT DATE         FROM HEALTH_DB.STAGING.DATE
GROUP BY DATE         HAVING COUNT(*) > 1) UNION ALL
SELECT 'PATIENT_VISITS' AS TABLE_NAME, COUNT(*) AS DUPLICATE_RECORDS FROM (SELECT VISIT_ID     FROM
HEALTH_DB.STAGING.PATIENT_VISITS GROUP BY VISIT_ID     HAVING COUNT(*) > 1);

SELECT 'PATIENTS'       AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.PATIENTS       UNION ALL
SELECT 'DOCTORS'        AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.DOCTORS        UNION ALL
SELECT 'FACILITIES'     AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.FACILITIES     UNION ALL
SELECT 'DIAGNOSES'      AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.DIAGNOSES      UNION ALL
SELECT 'DATE'           AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.DATE           UNION ALL
SELECT 'PATIENT_VISITS' AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.STAGING.PATIENT_VISITS;


USE SCHEMA HEALTH_DB.CURATED;

CREATE OR REPLACE TABLE DIM_PATIENT AS
SELECT DISTINCT
    PATIENT_ID,
    FIRST_NAME,
    LAST_NAME,
    GENDER,
    DATE_OF_BIRTH,
    CITY,
    INSURANCE_PROVIDER,
    REGISTRATION_DATE
FROM HEALTH_DB.STAGING.PATIENTS;

CREATE OR REPLACE TABLE DIM_DOCTOR AS
```

```
SELECT DISTINCT
    DOCTOR_ID,
    DOCTOR_NAME,
    SPECIALTY,
    EXPERIENCE_YEARS,
    DEPARTMENT
FROM HEALTH_DB.STAGING.DOCTORS;

CREATE OR REPLACE TABLE DIM_FACILITY AS
SELECT DISTINCT
    FACILITY_ID,
    FACILITY_NAME,
    CITY,
    FACILITY_TYPE
FROM HEALTH_DB.STAGING.FACILITIES;

CREATE OR REPLACE TABLE DIM_DIAGNOSIS AS
SELECT DISTINCT
    DIAGNOSIS_ID,
    DIAGNOSIS_NAME,
    DIAGNOSIS_CATEGORY
FROM HEALTH_DB.STAGING.DIAGNOSES;

CREATE OR REPLACE TABLE DIM_DATE AS
SELECT DISTINCT
    DATE,
    YEAR,
    QUARTER,
    MONTH,
    MONTH_NAME,
    WEEK,
    DAY_OF_WEEK,
    IS_WEEKEND
FROM HEALTH_DB.STAGING.DATE;

CREATE OR REPLACE TABLE FACT_PATIENT_VISITS AS
SELECT
    v.VISIT_ID,
    v.PATIENT_ID,
    v.DOCTOR_ID,
    v.FACILITY_ID,
    v.DIAGNOSIS_ID,
    v.VISIT_DATE,
    v.VISIT_TYPE,
    v.ADMISSION_STATUS,
    v.TREATMENT_STATUS,
    v.LENGTH_OF_STAY_DAYS,
    v.CONSULTATION_COST,
    v.MEDICATION_COST,
    v.LAB_COST,
    v.ROOM_COST,
    v.TOTAL_COST,
    v.INSURANCE_COVERAGE,
    v.PATIENT_PAYMENT,
    v.PATIENT_SATISFACTION,
    v.READMISSION_WITHIN_30_DAYS,
    p.FIRST_NAME,
    p.LAST_NAME,
    p.GENDER,
    p.CITY            AS PATIENT_CITY,
    p.INSURANCE_PROVIDER,
    d.DOCTOR_NAME,
    d.SPECIALTY,
    d.DEPARTMENT,
    f.FACILITY_NAME,
    f.CITY            AS FACILITY_CITY,
    f.FACILITY_TYPE,
    dg.DIAGNOSIS_NAME,
    dg.DIAGNOSIS_CATEGORY
FROM HEALTH_DB.STAGING.PATIENT_VISITS v
LEFT JOIN HEALTH_DB.STAGING.PATIENTS   p  ON v.PATIENT_ID   = p.PATIENT_ID
LEFT JOIN HEALTH_DB.STAGING.DOCTORS    d  ON v.DOCTOR_ID    = d.DOCTOR_ID
LEFT JOIN HEALTH_DB.STAGING.FACILITIES f  ON v.FACILITY_ID  = f.FACILITY_ID
LEFT JOIN HEALTH_DB.STAGING.DIAGNOSES  dg ON v.DIAGNOSIS_ID = dg.DIAGNOSIS_ID;

SELECT 'DIM_PATIENT'         AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.DIM_PATIENT        UNION ALL
SELECT 'DIM_DOCTOR'          AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.DIM_DOCTOR         UNION ALL
SELECT 'DIM_FACILITY'        AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.DIM_FACILITY       UNION ALL
SELECT 'DIM_DIAGNOSIS'       AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.DIM_DIAGNOSIS      UNION ALL
```

```
SELECT 'DIM_DATE'            AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.DIM_DATE           UNION ALL
SELECT 'FACT_PATIENT_VISITS' AS TABLE_NAME, COUNT(*) AS ROW_COUNT FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS;

SELECT 'Unmatched PATIENT_ID'   AS CHECK_NAME, COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS WHERE
FIRST_NAME    IS NULL UNION ALL
SELECT 'Unmatched DOCTOR_ID'    AS CHECK_NAME, COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS WHERE
DOCTOR_NAME   IS NULL UNION ALL
SELECT 'Unmatched FACILITY_ID'  AS CHECK_NAME, COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS WHERE
FACILITY_NAME IS NULL UNION ALL
SELECT 'Unmatched DIAGNOSIS_ID' AS CHECK_NAME, COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS WHERE
DIAGNOSIS_NAME IS NULL;

SELECT * FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS LIMIT 5;


USE SCHEMA HEALTH_DB.MART;

CREATE OR REPLACE VIEW VW_EXECUTIVE_KPI AS
SELECT
    COUNT(*)                            AS TOTAL_VISITS,
    COUNT(DISTINCT PATIENT_ID)          AS TOTAL_PATIENTS,
    COUNT(DISTINCT DOCTOR_ID)           AS TOTAL_DOCTORS,
    COUNT(DISTINCT FACILITY_ID)         AS TOTAL_FACILITIES,
    ROUND(SUM(TOTAL_COST), 2)           AS TOTAL_COST,
    ROUND(SUM(INSURANCE_COVERAGE), 2)   AS TOTAL_INSURANCE_COVERAGE,
    ROUND(SUM(PATIENT_PAYMENT), 2)      AS TOTAL_PATIENT_PAYMENT,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_PATIENT_SATISFACTION,
    ROUND(AVG(LENGTH_OF_STAY_DAYS), 2)  AS AVG_LENGTH_OF_STAY,
    SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) AS TOTAL_READMISSIONS
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS;

CREATE OR REPLACE VIEW VW_MONTHLY_VISITS AS
SELECT
    DATE_TRUNC('MONTH', VISIT_DATE)     AS VISIT_MONTH,
    COUNT(*)                            AS TOTAL_VISITS,
    ROUND(SUM(TOTAL_COST), 2)           AS TOTAL_COST,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1
ORDER BY 1;

CREATE OR REPLACE VIEW VW_DIAGNOSIS_ANALYTICS AS
SELECT
    DIAGNOSIS_NAME,
    DIAGNOSIS_CATEGORY,
    COUNT(*)                            AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_COST,
    ROUND(AVG(LENGTH_OF_STAY_DAYS), 2)  AS AVG_LENGTH_OF_STAY,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_FACILITY_ANALYTICS AS
SELECT
    FACILITY_NAME,
    FACILITY_CITY,
    FACILITY_TYPE,
    COUNT(VISIT_ID)                     AS TOTAL_VISITS,
    ROUND(SUM(TOTAL_COST), 2)           AS TOTAL_COST,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_COST_PER_VISIT,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION,
    ROUND(AVG(LENGTH_OF_STAY_DAYS), 2)  AS AVG_LENGTH_OF_STAY
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2, 3
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_VISIT_TYPE_DISTRIBUTION AS
SELECT
    VISIT_TYPE,
    COUNT(*)                            AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_VISIT_COST,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION,
    ROUND(AVG(LENGTH_OF_STAY_DAYS), 2)  AS AVG_LENGTH_OF_STAY
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_TREATMENT_ANALYTICS AS
```

```
SELECT
    TREATMENT_STATUS,
    COUNT(*)                            AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_COST,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_ADMISSION_STATUS_ANALYTICS AS
SELECT
    ADMISSION_STATUS,
    COUNT(*)                            AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_COST
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_PATIENT_VISIT_TYPE_ANALYSIS AS
SELECT
    VISIT_TYPE,
    COUNT(VISIT_ID)                     AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_VISIT_COST,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION,
    ROUND(AVG(LENGTH_OF_STAY_DAYS), 2)  AS AVG_LENGTH_OF_STAY
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1
ORDER BY AVG_VISIT_COST DESC;

CREATE OR REPLACE VIEW VW_DOCTOR_ANALYTICS AS
SELECT
    DOCTOR_NAME,
    SPECIALTY,
    DEPARTMENT,
    COUNT(VISIT_ID)                     AS TOTAL_VISITS,
    ROUND(AVG(TOTAL_COST), 2)           AS AVG_COST_PER_VISIT,
    ROUND(AVG(PATIENT_SATISFACTION), 2) AS AVG_SATISFACTION,
    SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) AS TOTAL_READMISSIONS
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2, 3
ORDER BY TOTAL_VISITS DESC;

CREATE OR REPLACE VIEW VW_READMISSION_ANALYTICS AS
SELECT
    DIAGNOSIS_CATEGORY,
    DIAGNOSIS_NAME,
    COUNT(*)                            AS TOTAL_VISITS,
    SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) AS READMISSIONS,
    ROUND(
        100.0 * SUM(CASE WHEN READMISSION_WITHIN_30_DAYS = 'Yes' THEN 1 ELSE 0 END) / COUNT(*),
        2
    )                                   AS READMISSION_RATE_PCT
FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
GROUP BY 1, 2
ORDER BY READMISSION_RATE_PCT DESC;

SELECT * FROM HEALTH_DB.MART.VW_EXECUTIVE_KPI;
SELECT * FROM HEALTH_DB.MART.VW_MONTHLY_VISITS          LIMIT 5;
SELECT * FROM HEALTH_DB.MART.VW_DIAGNOSIS_ANALYTICS     LIMIT 5;
SELECT * FROM HEALTH_DB.MART.VW_FACILITY_ANALYTICS      LIMIT 5;
SELECT * FROM HEALTH_DB.MART.VW_VISIT_TYPE_DISTRIBUTION;
SELECT * FROM HEALTH_DB.MART.VW_TREATMENT_ANALYTICS;
SELECT * FROM HEALTH_DB.MART.VW_ADMISSION_STATUS_ANALYTICS;
SELECT * FROM HEALTH_DB.MART.VW_PATIENT_VISIT_TYPE_ANALYSIS;
SELECT * FROM HEALTH_DB.MART.VW_DOCTOR_ANALYTICS        LIMIT 5;
SELECT * FROM HEALTH_DB.MART.VW_READMISSION_ANALYTICS   LIMIT 5;
```

Appendix B — Streamlit application and deployment files

B.1 Health/streamlit_app.py

```
import streamlit as st
import pandas as pd
import altair as alt
from snowflake.snowpark.context import get_active_session

st.set_page_config(
    page_title="Healthcare Intelligence Hub",
    layout="wide",
    initial_sidebar_state="expanded"
)

# ── Connection (Snowflake native session) ──────────────────
@st.cache_data(ttl=600)
def load_fact():
    session = get_active_session()
    df = session.table("HEALTH_DB.CURATED.FACT_PATIENT_VISITS").to_pandas()
    df.columns = df.columns.str.upper()
    df["VISIT_DATE"] = pd.to_datetime(df["VISIT_DATE"])
    return df

df_raw = load_fact()

# ── Sidebar Filters ────────────────────────────────────────
st.sidebar.title("Filters")

date_min = df_raw["VISIT_DATE"].min().date()
date_max = df_raw["VISIT_DATE"].max().date()
date_range = st.sidebar.date_input(
    "Date Range", [date_min, date_max],
    min_value=date_min, max_value=date_max
)

facilities  = ["All"] + sorted(df_raw["FACILITY_NAME"].dropna().unique().tolist())
specialties = ["All"] + sorted(df_raw["SPECIALTY"].dropna().unique().tolist())
visit_types = ["All"] + sorted(df_raw["VISIT_TYPE"].dropna().unique().tolist())
diag_cats   = ["All"] + sorted(df_raw["DIAGNOSIS_CATEGORY"].dropna().unique().tolist())

sel_facility  = st.sidebar.selectbox("Facility",           facilities)
sel_specialty = st.sidebar.selectbox("Doctor Specialty",   specialties)
sel_visit     = st.sidebar.selectbox("Visit Type",         visit_types)
sel_diag_cat  = st.sidebar.selectbox("Diagnosis Category", diag_cats)

# ── Apply Filters ──────────────────────────────────────────
df = df_raw.copy()
if len(date_range) == 2:
    df = df[
        (df["VISIT_DATE"].dt.date >= date_range[0]) &
        (df["VISIT_DATE"].dt.date <= date_range[1])
    ]
if sel_facility  != "All": df = df[df["FACILITY_NAME"]      == sel_facility]
if sel_specialty != "All": df = df[df["SPECIALTY"]          == sel_specialty]
if sel_visit     != "All": df = df[df["VISIT_TYPE"]         == sel_visit]
if sel_diag_cat  != "All": df = df[df["DIAGNOSIS_CATEGORY"] == sel_diag_cat]

# ── Header ─────────────────────────────────────────────────
st.title("Healthcare Intelligence Hub")
st.caption("Patient, Cost & Care Analytics")
st.divider()

# ── KPI Cards ──────────────────────────────────────────────
total_visits   = len(df)
total_patients = df["PATIENT_ID"].nunique()
total_cost     = df["TOTAL_COST"].sum()
avg_sat        = df["PATIENT_SATISFACTION"].mean() if total_visits > 0 else 0
readmit_rate   = (
    100 * (df["READMISSION_WITHIN_30_DAYS"] == "Yes").sum() / total_visits
    if total_visits > 0 else 0
)
avg_los = df["LENGTH_OF_STAY_DAYS"].mean() if total_visits > 0 else 0
```

```
c1, c2, c3, c4, c5, c6 = st.columns(6)
c1.metric("Total Visits",       f"{total_visits:,}")
c2.metric("Unique Patients",    f"{total_patients:,}")
c3.metric("Total Cost",         f"${total_cost:,.0f}")
c4.metric("Avg Satisfaction",   f"{avg_sat:.2f} / 5")
c5.metric("Readmission Rate",   f"{readmit_rate:.1f}%")
c6.metric("Avg Length of Stay", f"{avg_los:.2f} days")

st.divider()

# ── Row 1: Monthly Trend + Visit Type Split ──────────────────
col1, col2 = st.columns([2, 1])

with col1:
    st.subheader("Monthly Visit Trend")
    monthly = (
        df.groupby(df["VISIT_DATE"].dt.to_period("M"))
        .agg(TOTAL_VISITS=("VISIT_ID", "count"))
        .reset_index()
    )
    monthly["VISIT_DATE"] = monthly["VISIT_DATE"].dt.to_timestamp()
    chart = alt.Chart(monthly).mark_line(point=True).encode(
        x=alt.X("VISIT_DATE:T", title="Month"),
        y=alt.Y("TOTAL_VISITS:Q", title="Total Visits")
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)

with col2:
    st.subheader("Visit Type Split")
    vt = df.groupby("VISIT_TYPE")["VISIT_ID"].count().reset_index()
    vt.columns = ["VISIT_TYPE", "COUNT"]
    chart = alt.Chart(vt).mark_arc(innerRadius=50).encode(
        theta=alt.Theta("COUNT:Q"),
        color=alt.Color("VISIT_TYPE:N", title="Visit Type"),
        tooltip=["VISIT_TYPE", "COUNT"]
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)

# ── Row 2: Top Diagnoses + Admission Status ──────────────────
col1, col2 = st.columns([2, 1])

with col1:
    st.subheader("Top 10 Diagnoses by Volume")
    diag = (
        df.groupby("DIAGNOSIS_NAME")["VISIT_ID"].count()
        .reset_index()
        .rename(columns={"VISIT_ID": "VISITS"})
        .sort_values("VISITS", ascending=False)
        .head(10)
    )
    chart = alt.Chart(diag).mark_bar().encode(
        x=alt.X("VISITS:Q", title="Total Visits"),
        y=alt.Y("DIAGNOSIS_NAME:N", sort="-x", title=""),
        tooltip=["DIAGNOSIS_NAME", "VISITS"]
    ).properties(height=320)
    st.altair_chart(chart, use_container_width=True)

with col2:
    st.subheader("Admission Status")
    adm = df.groupby("ADMISSION_STATUS")["VISIT_ID"].count().reset_index()
    adm.columns = ["STATUS", "COUNT"]
    chart = alt.Chart(adm).mark_arc(innerRadius=50).encode(
        theta=alt.Theta("COUNT:Q"),
        color=alt.Color("STATUS:N", title="Status"),
        tooltip=["STATUS", "COUNT"]
    ).properties(height=320)
    st.altair_chart(chart, use_container_width=True)

# ── Row 3: Facility Performance ──────────────────────────────
st.subheader("Facility Performance")
facility_df = (
    df.groupby(["FACILITY_NAME", "FACILITY_CITY", "FACILITY_TYPE"])
    .agg(
        TOTAL_VISITS    =("VISIT_ID",            "count"),
        TOTAL_COST      =("TOTAL_COST",          "sum"),
        AVG_COST        =("TOTAL_COST",          "mean"),
        AVG_SATISFACTION=("PATIENT_SATISFACTION","mean"),
        AVG_LOS         =("LENGTH_OF_STAY_DAYS", "mean"),
    )
```

```
    .reset_index()
    .sort_values("TOTAL_VISITS", ascending=False)
    .round(2)
)

col1, col2 = st.columns([1, 1])
with col1:
    chart = alt.Chart(facility_df).mark_bar().encode(
        x=alt.X("FACILITY_NAME:N", sort="-y", title="", axis=alt.Axis(labelAngle=-30)),
        y=alt.Y("TOTAL_VISITS:Q", title="Total Visits"),
        color=alt.Color("FACILITY_TYPE:N", title="Type"),
        tooltip=["FACILITY_NAME", "FACILITY_TYPE", "TOTAL_VISITS", "AVG_COST"]
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)
with col2:
    st.dataframe(
        facility_df[["FACILITY_NAME", "FACILITY_CITY", "FACILITY_TYPE",
                     "TOTAL_VISITS", "AVG_COST", "AVG_SATISFACTION"]],
        use_container_width=True, hide_index=True
    )

# ── Row 4: Treatment Status + Readmission Rate ──────────────────
col1, col2 = st.columns(2)

with col1:
    st.subheader("Treatment Status")
    treat = df.groupby("TREATMENT_STATUS")["VISIT_ID"].count().reset_index()
    treat.columns = ["STATUS", "COUNT"]
    chart = alt.Chart(treat).mark_bar().encode(
        x=alt.X("STATUS:N", title="Treatment Status"),
        y=alt.Y("COUNT:Q", title="Total Visits"),
        tooltip=["STATUS", "COUNT"]
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)

with col2:
    st.subheader("Readmission Rate by Diagnosis")
    readm = df.groupby("DIAGNOSIS_NAME").agg(
        TOTAL       =("VISIT_ID", "count"),
        READMISSIONS=("READMISSION_WITHIN_30_DAYS", lambda x: (x == "Yes").sum())
    ).reset_index()
    readm["READMISSION_RATE_PCT"] = (100 * readm["READMISSIONS"] / readm["TOTAL"]).round(2)
    readm = readm.sort_values("READMISSION_RATE_PCT", ascending=False).head(10)
    chart = alt.Chart(readm).mark_bar().encode(
        x=alt.X("READMISSION_RATE_PCT:Q", title="Readmission Rate (%)"),
        y=alt.Y("DIAGNOSIS_NAME:N", sort="-x", title=""),
        tooltip=["DIAGNOSIS_NAME", "READMISSION_RATE_PCT", "READMISSIONS"]
    ).properties(height=300)
    st.altair_chart(chart, use_container_width=True)

# ── Row 5: Cost Breakdown by Visit Type ─────────────────────────
st.subheader("Average Cost Breakdown by Visit Type")
cost_cols = ["CONSULTATION_COST", "MEDICATION_COST", "LAB_COST", "ROOM_COST"]
cost_df = df.groupby("VISIT_TYPE")[cost_cols].mean().round(2).reset_index()
cost_melted = cost_df.melt(id_vars="VISIT_TYPE", var_name="COST_TYPE", value_name="AVG_COST")
chart = alt.Chart(cost_melted).mark_bar().encode(
    x=alt.X("VISIT_TYPE:N", title="Visit Type"),
    y=alt.Y("AVG_COST:Q", title="Avg Cost ($)"),
    color=alt.Color("COST_TYPE:N", title="Cost Type"),
    tooltip=["VISIT_TYPE", "COST_TYPE", "AVG_COST"]
).properties(height=300)
st.altair_chart(chart, use_container_width=True)

# ── Row 6: Doctor Leaderboard ────────────────────────────────────
st.subheader("Doctor Performance Leaderboard")
doctor_df = (
    df.groupby(["DOCTOR_NAME", "SPECIALTY", "DEPARTMENT"])
    .agg(
        TOTAL_VISITS    =("VISIT_ID",                   "count"),
        AVG_COST        =("TOTAL_COST",                 "mean"),
        AVG_SATISFACTION=("PATIENT_SATISFACTION",       "mean"),
        READMISSIONS    =("READMISSION_WITHIN_30_DAYS", lambda x: (x == "Yes").sum())
    )
    .reset_index()
    .sort_values("TOTAL_VISITS", ascending=False)
    .round(2)
)
st.dataframe(doctor_df, use_container_width=True, hide_index=True)
```

```
st.divider()
st.caption(f"Data filtered to {total_visits:,} visits | HEALTH_DB.CURATED.FACT_PATIENT_VISITS")
```

B.2 Health/snowflake.yml

```
definition_version: 2
entities:
  streamlit_app:
    type: streamlit
    meta:
      use_mixins:
        - streamlit_shared
    identifier:
      name: ICYNJI7QNBXR2IUI
      database: HEALTH_DB
      schema: RAW
mixins:
  streamlit_shared:
    artifacts:
      - pyproject.toml
      - streamlit_app.py
      - .streamlit/config.toml
    title: HEALTH
    query_warehouse: COMPUTE_WH
    compute_pool: SYSTEM_COMPUTE_POOL_CPU
    main_file: streamlit_app.py
```

B.3 Health/pyproject.toml

```
# Streamlit in Snowflake apps run with a pre-installed version of the Streamlit library.
# To use a different Streamlit version or packages not pre-installed,
# configure a package source available in this deployment, such as an
# artifact repository or an External Access Integration to an approved repository.
# Artifact repositories: (https://docs.snowflake.com/en/developer-guide/streamlit/app-development/dependency-
management#snowflake-artifact-repository)
# External Access Integrations: (https://docs.snowflake.com/en/developer-guide/streamlit/additional-features#external-
network-access-in-sis)
# If a package repository server is not reachable and the pyproject.toml file has not been modified, the app will fall
back to the default version of the Streamlit library.

[project]
name = "streamlit-app"
requires-python = "~=3.11.0"
version = "0.0.1"
description = ""
dependencies = [
    "streamlit[snowflake]"
]

[tool.uv]
constraint-dependencies = ["numba>=0.56.0"]
exclude-newer = "7 days"

[tool.uv.exclude-newer-package]
streamlit = false
```

Appendix C — Data Dictionary

Generated from the `CREATE TABLE` statements in `query.sql` and profiled against the six CSV files (all values verified). Keys are **logical**: `query.sql` declares no PRIMARY KEY or FOREIGN KEY constraints. Monetary currency is **not specified** in the source (the app labels it `$`).

Layer overview

Source CSV	RAW table	STAGING table	CURATED table	Rows	Columns
`patients.csv`	`RAW.PATIENTS_RAW`	`STAGING.PATIENTS`	`CURATED.DIM_PATIENT`	3,000	8
`doctors.csv`	`RAW.DOCTORS_RAW`	`STAGING.DOCTORS`	`CURATED.DIM_DOCTOR`	60	5
`facilities.csv`	`RAW.FACILITIES_RAW`	`STAGING.FACILITIES`	`CURATED.DIM_FACILITY`	10	4
`diagnoses.csv`	`RAW.DIAGNOSES_RAW`	`STAGING.DIAGNOSES`	`CURATED.DIM_DIAGNOSIS`	15	3
`date.csv`	`RAW.DATE_RAW`	`STAGING.DATE`	`CURATED.DIM_DATE`	974	8
`patient_visits.csv`	`RAW.PATIENT_VISITS_RAW`	`STAGING.PATIENT_VISITS`	`CURATED.FACT_PATIENT_VISITS`	10,000	19

STAGING and CURATED dimension tables keep the RAW column names and types. STAGING applies the transformation shown in the *STAGING rule* column, and the dimensions are `SELECT DISTINCT` copies of STAGING.

Source / RAW tables

`RAW.PATIENTS_RAW` → `STAGING.PATIENTS` → `CURATED.DIM_PATIENT`

Source: `patients.csv` · 3,000 rows · key: `PATIENT_ID`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`PATIENT_ID`	VARCHAR	PK	Patient business key (format `P00001`)	TRIM	3,000	0	e.g. P00001
`FIRST_NAME`	VARCHAR		Patient first name (PII)	TRIM	20	0	e.g. Ananya
`LAST_NAME`	VARCHAR		Patient last name (PII)	TRIM	15	0	e.g. Brown
`GENDER`	VARCHAR		Patient gender	INITCAP(TRIM)	3	0	Female, Male, Other
`DATE_OF_BIRTH`	DATE		Date of birth (PII)	pass-through	2,822	0	1945-01-11 – 2015-12-21
`CITY`	VARCHAR		City	INITCAP(TRIM)	10	0	e.g. Ahmedabad
`INSURANCE_PROVIDER`	VARCHAR		Insurer name, or `Self Pay`	TRIM	6	0	HealthPlus, LifeShield, MediCare, SecureHealth, Self Pay, WellCare
`REGISTRATION_DATE`	DATE		Date the patient registered	pass-through	1,445	0	2021-01-01 – 2025-12-31

`RAW.DOCTORS_RAW` → `STAGING.DOCTORS` → `CURATED.DIM_DOCTOR`

Source: `doctors.csv` · 60 rows · key: `DOCTOR_ID`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`DOCTOR_ID`	VARCHAR	PK	Doctor business key (format `D001`)	TRIM	60	0	e.g. D001
`DOCTOR_NAME`	VARCHAR		Doctor display name (not unique: 55 names for 60 IDs)	TRIM	55	0	e.g. Dr. Ishita Sharma
`SPECIALTY`	VARCHAR		Medical specialty	INITCAP(TRIM)	10	0	e.g. Neurology
`EXPERIENCE_YEARS`	NUMBER		Years of experience	pass-through	26	0	2 – 30
`DEPARTMENT`	VARCHAR		Hospital department	INITCAP(TRIM)	4	0	Emergency, Inpatient, Outpatient, Specialty Care

`RAW.FACILITIES_RAW` → `STAGING.FACILITIES` → `CURATED.DIM_FACILITY`

Source: `facilities.csv` · 10 rows · key: `FACILITY_ID`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`FACILITY_ID`	VARCHAR	PK	Facility business key (format `F001`)	TRIM	10	0	e.g. F001
`FACILITY_NAME`	VARCHAR		Facility name	TRIM	10	0	e.g. Central City Hospital
`CITY`	VARCHAR		City	INITCAP(TRIM)	10	0	e.g. Hyderabad
`FACILITY_TYPE`	VARCHAR		Clinic / Hospital / Specialty Center	INITCAP(TRIM)	3	0	Clinic, Hospital, Specialty Center

`RAW.DIAGNOSES_RAW` → `STAGING.DIAGNOSES` → `CURATED.DIM_DIAGNOSIS`

Source: `diagnoses.csv` · 15 rows · key: `DIAGNOSIS_ID`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`DIAGNOSIS_ID`	VARCHAR	PK	Diagnosis business key (format `DIA001`)	TRIM	15	0	e.g. DIA001
`DIAGNOSIS_NAME`	VARCHAR		Diagnosis	TRIM	15	0	e.g. Hypertension
`DIAGNOSIS_CATEGORY`	VARCHAR		Clinical category of the diagnosis	INITCAP(TRIM)	10	0	e.g. Cardiovascular

`RAW.DATE_RAW` → `STAGING.DATE` → `CURATED.DIM_DATE`

Source: `date.csv` · 974 rows · key: `DATE`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`DATE`	DATE	PK	Calendar date (date-dimension key)	pass-through	974	0	2024-01-01 – 2026-08-31
`YEAR`	NUMBER		Calendar year	pass-through	3	0	2024, 2025, 2026
`QUARTER`	VARCHAR		Quarter label `Q1` – `Q4`	pass-through	4	0	Q1, Q2, Q3, Q4
`MONTH`	NUMBER		Month number 1–12	pass-through	12	0	1 – 12
`MONTH_NAME`	VARCHAR		Month name	pass-through	12	0	e.g. January
`WEEK`	NUMBER		Week number	pass-through	52	0	1 – 52
`DAY_OF_WEEK`	VARCHAR		Weekday name	pass-through	7	0	e.g. Monday
`IS_WEEKEND`	BOOLEAN		TRUE on Saturday/Sunday	pass-through	2	0	False, True

`RAW.PATIENT_VISITS_RAW` → `STAGING.PATIENT_VISITS` → `CURATED.FACT_PATIENT_VISITS`

Source: `patient_visits.csv` · 10,000 rows · key: `VISIT_ID`

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
`VISIT_ID`	VARCHAR	PK	Visit business key (format `V000001`); fact grain	TRIM	10,000	0	e.g. V000001
`PATIENT_ID`	VARCHAR	FK → DIM_PATIENT	Patient business key (format `P00001`)	TRIM	2,889	0	e.g. P02842
`DOCTOR_ID`	VARCHAR	FK → DIM_DOCTOR	Doctor business key (format `D001`)	TRIM	60	0	e.g. D004

Column	Type	Key	Description	STAGING rule	Distinct	NULL/blank	Example / range
FACILITY_ID	VARCHAR	FK → DIM_FACILITY	Facility business key (format F001)	TRIM	10	0	e.g. F006
DIAGNOSIS_ID	VARCHAR	FK → DIM_DIAGNOSIS	Diagnosis business key (format DIA001)	TRIM	15	0	e.g. DIA001
VISIT_DATE	DATE	FK → DIM_DATE.DATE (logical)	Date of visit (FK to date dimension)	pass-through	974	0	2024-01-01 – 2026-08-31
VISIT_TYPE	VARCHAR		Outpatient / Follow-up / Telemedicine / Emergency / Inpatient	INITCAP(TRIM)	5	0	Emergency, Follow-up, Inpatient, Outpatient, Telemedicine
ADMISSION_STATUS	VARCHAR		Admitted / Discharged	INITCAP(TRIM)	2	0	Admitted, Discharged
TREATMENT_STATUS	VARCHAR		Completed / Ongoing / Referred / Cancelled	INITCAP(TRIM)	4	0	Cancelled, Completed, Ongoing, Referred
LENGTH_OF_STAY_DAYS	NUMBER		Length of stay in days (0 for non-inpatient)	pass-through	15	0	0 – 14
CONSULTATION_COST	NUMBER(12,2)		Consultation charge	pass-through	2,176	0	300 – 2500
MEDICATION_COST	NUMBER(12,2)		Medication charge	pass-through	4,279	0	100 – 5000
LAB_COST	NUMBER(12,2)		Laboratory charge	pass-through	5,350	0	0 – 7000
ROOM_COST	NUMBER(12,2)		Room charge (non-zero only for inpatient)	pass-through	1,190	0	0 – 11982
TOTAL_COST	NUMBER(12,2)		Sum of the four cost components (verified)	pass-through	6,748	0	973 – 23642
INSURANCE_COVERAGE	NUMBER(12,2)		Amount paid by insurer	pass-through	9,931	0	643.88 – 20257
PATIENT_PAYMENT	NUMBER(12,2)		Amount paid by patient; coverage + payment = total (verified)	pass-through	9,900	0	106.67 – 10837.4
PATIENT_SATISFACTION	NUMBER(5,2)		Satisfaction score 1–5 (integers in data)	pass-through	5	0	1, 2, 3, 4, 5
READMISSION_WITHIN_30_DAYS	VARCHAR		Yes / No : readmitted within 30 days	INITCAP(TRIM)	2	0	No, Yes

CURATED.FACT_PATIENT_VISITS

Grain: **one row per visit** (10,000 rows). Built by `LEFT JOIN` of `STAGING.PATIENT_VISITS` to patients, doctors, facilities and diagnoses.

#	Column	Origin	Notes
1	`VISIT_ID`	`STAGING.PATIENT_VISITS`	Visit business key (format `V000001`); fact grain
2	`PATIENT_ID`	`STAGING.PATIENT_VISITS`	Patient business key (format `P00001`)
3	`DOCTOR_ID`	`STAGING.PATIENT_VISITS`	Doctor business key (format `D001`)
4	`FACILITY_ID`	`STAGING.PATIENT_VISITS`	Facility business key (format `F001`)
5	`DIAGNOSIS_ID`	`STAGING.PATIENT_VISITS`	Diagnosis business key (format `DIA001`)
6	`VISIT_DATE`	`STAGING.PATIENT_VISITS`	Date of visit (FK to date dimension)
7	`VISIT_TYPE`	`STAGING.PATIENT_VISITS`	Outpatient / Follow-up / Telemedicine / Emergency / Inpatient
8	`ADMISSION_STATUS`	`STAGING.PATIENT_VISITS`	Admitted / Discharged
9	`TREATMENT_STATUS`	`STAGING.PATIENT_VISITS`	Completed / Ongoing / Referred / Cancelled
10	`LENGTH_OF_STAY_DAYS`	`STAGING.PATIENT_VISITS`	Length of stay in days (0 for non-inpatient)
11	`CONSULTATION_COST`	`STAGING.PATIENT_VISITS`	Consultation charge
12	`MEDICATION_COST`	`STAGING.PATIENT_VISITS`	Medication charge
13	`LAB_COST`	`STAGING.PATIENT_VISITS`	Laboratory charge
14	`ROOM_COST`	`STAGING.PATIENT_VISITS`	Room charge (non-zero only for inpatient)
15	`TOTAL_COST`	`STAGING.PATIENT_VISITS`	Sum of the four cost components (verified)
16	`INSURANCE_COVERAGE`	`STAGING.PATIENT_VISITS`	Amount paid by insurer
17	`PATIENT_PAYMENT`	`STAGING.PATIENT_VISITS`	Amount paid by patient; coverage + payment = total (verified)
18	`PATIENT_SATISFACTION`	`STAGING.PATIENT_VISITS`	Satisfaction score 1–5 (integers in data)
19	`READMISSION_WITHIN_30_DAYS`	`STAGING.PATIENT_VISITS`	`Yes` / `No` : readmitted within 30 days
20	`FIRST_NAME`	`STAGING.PATIENTS` (denormalised)	Patient first name (PII)
21	`LAST_NAME`	`STAGING.PATIENTS` (denormalised)	Patient last name (PII)
22	`GENDER`	`STAGING.PATIENTS` (denormalised)	Patient gender
23	`PATIENT_CITY`	`STAGING.PATIENTS.CITY` (denormalised)	City
24	`INSURANCE_PROVIDER`	`STAGING.PATIENTS` (denormalised)	Insurer name, or `Self Pay`
25	`DOCTOR_NAME`	`STAGING.DOCTORS` (denormalised)	Doctor display name (not unique: 55 names for 60 IDs)
26	`SPECIALTY`	`STAGING.DOCTORS` (denormalised)	Medical specialty
27	`DEPARTMENT`	`STAGING.DOCTORS` (denormalised)	Hospital department
28	`FACILITY_NAME`	`STAGING.FACILITIES` (denormalised)	Facility name
29	`FACILITY_CITY`	`STAGING.FACILITIES.CITY` (denormalised)	City
30	`FACILITY_TYPE`	`STAGING.FACILITIES` (denormalised)	Clinic / Hospital / Specialty Center
31	`DIAGNOSIS_NAME`	`STAGING.DIAGNOSES` (denormalised)	Diagnosis
32	`DIAGNOSIS_CATEGORY`	`STAGING.DIAGNOSES` (denormalised)	Clinical category of the diagnosis

Total: 32 columns.

MART views

All views read `CURATED.FACT_PATIENT_VISITS`.

View	Group-by (grain)	Output columns
`MART.VW_EXECUTIVE_KPI`	whole table (1 row)	`TOTAL_VISITS`, `TOTAL_PATIENTS`, `TOTAL_DOCTORS`, `TOTAL_FACILITIES`, `TOTAL_COST`, `TOTAL_INSURANCE_COVERAGE`, `TOTAL_PATIENT_PAYMENT`, `AVG_PATIENT_SATISFACTION`, `AVG_LENGTH_OF_STAY`, `TOTAL_READMISSIONS`
`MART.VW_MONTHLY_VISITS`	month: DATE_TRUNC('MONTH', VISIT_DATE)	`VISIT_MONTH`, `TOTAL_VISITS`, `TOTAL_COST`, `AVG_SATISFACTION`
`MART.VW_DIAGNOSIS_ANALYTICS`	DIAGNOSIS_NAME, DIAGNOSIS_CATEGORY	`DIAGNOSIS_NAME`, `DIAGNOSIS_CATEGORY`, `TOTAL_VISITS`, `AVG_COST`, `AVG_LENGTH_OF_STAY`, `AVG_SATISFACTION`
`MART.VW_FACILITY_ANALYTICS`	FACILITY_NAME, FACILITY_CITY, FACILITY_TYPE	`FACILITY_NAME`, `FACILITY_CITY`, `FACILITY_TYPE`, `TOTAL_VISITS`, `TOTAL_COST`, `AVG_COST_PER_VISIT`, `AVG_SATISFACTION`, `AVG_LENGTH_OF_STAY`
`MART.VW_VISIT_TYPE_DISTRIBUTION`	VISIT_TYPE	`VISIT_TYPE`, `TOTAL_VISITS`, `AVG_VISIT_COST`, `AVG_SATISFACTION`, `AVG_LENGTH_OF_STAY`
`MART.VW_TREATMENT_ANALYTICS`	TREATMENT_STATUS	`TREATMENT_STATUS`, `TOTAL_VISITS`, `AVG_COST`, `AVG_SATISFACTION`
`MART.VW_ADMISSION_STATUS_ANALYTICS`	ADMISSION_STATUS	`ADMISSION_STATUS`, `TOTAL_VISITS`, `AVG_COST`
`MART.VW_PATIENT_VISIT_TYPE_ANALYSIS`	VISIT_TYPE	`VISIT_TYPE`, `TOTAL_VISITS`, `AVG_VISIT_COST`, `AVG_SATISFACTION`, `AVG_LENGTH_OF_STAY`
`MART.VW_DOCTOR_ANALYTICS`	DOCTOR_NAME, SPECIALTY, DEPARTMENT	`DOCTOR_NAME`, `SPECIALTY`, `DEPARTMENT`, `TOTAL_VISITS`, `AVG_COST_PER_VISIT`, `AVG_SATISFACTION`, `TOTAL_READMISSIONS`
`MART.VW_READMISSION_ANALYTICS`	DIAGNOSIS_CATEGORY, DIAGNOSIS_NAME	`DIAGNOSIS_CATEGORY`, `DIAGNOSIS_NAME`, `TOTAL_VISITS`, `READMISSIONS`, `READMISSION_RATE_PCT`

Relationships

From (child)	Column	To (parent)	Cardinality	Verified orphans
FACT_PATIENT_VISITS	PATIENT_ID	DIM_PATIENT.PATIENT_ID	N : 1	0
FACT_PATIENT_VISITS	DOCTOR_ID	DIM_DOCTOR.DOCTOR_ID	N : 1	0
FACT_PATIENT_VISITS	FACILITY_ID	DIM_FACILITY.FACILITY_ID	N : 1	0
FACT_PATIENT_VISITS	DIAGNOSIS_ID	DIM_DIAGNOSIS.DIAGNOSIS_ID	N : 1	0
FACT_PATIENT_VISITS	VISIT_DATE	DIM_DATE.DATE	N : 1 (logical; not joined in code)	0

Unknown or unverified information

- Currency of monetary columns; meaning of `Self Pay` alongside non-zero `INSURANCE_COVERAGE`.
- How `READMISSION_WITHIN_30_DAYS` was derived upstream.
- Whether `ADMISSION_STATUS = 'Admitted'` with `LENGTH_OF_STAY_DAYS = 0` (653 emergency visits) is intended.
- Snowflake column types after load were not inspected live; types above are the declared DDL types.